ENTERPRISE ARCHITECTURE REFERENCE

Canvas Marketing OS

Architecture, Product & Operating Model — reverse-engineered from source

A complete technical and commercial reference for the Canvas Marketing OS platform, derived entirely from the source tree. No prior documentation was assumed; the code is treated as the single source of truth. Every claim cites a file. Inference is marked **[INFERRED]**.

18	917	8	27	23
DOCUMENTS	FILES READ	SERVICES	TABLES / 5 SCHEMAS	AGENT PACKAGES

Repository rooipiet-tech/Canvas-Marketing-OS **Branch** claude/platform-reverse-engineering-19ytw9 **Prepared** 2 September 2026

Contents

00

Executive Summary

*Reverse-engineered from source. Every claim below cites a file. Where a statement is inference rather than something the code states, it is marked **[INFERRED]**.*

1. What this product is

Canvas Marketing OS (CMOS) is a governed, agent-native execution platform for marketing operations. It is not a marketing tool with AI features bolted on. It is a *control plane* that lets autonomous AI agents perform real marketing work — market scanning, brief writing, content drafting, QA, scheduling, publishing — while every action they take is metered, policy- gated, consent-checked, content-hash-bound, human-approvable, auditable and reversible by a kill switch.

The single most revealing fact in the repository: there are **eight backend services**, and only *one* of them (`services/orchestrator`) is about doing marketing work. The other seven exist to *govern* it:

SERVICE	EXISTS TO
`services/orchestrator`	Do the work (decompose loops, dispatch tasks, call agents)
`services/model-gateway`	Meter cost, route models, block PII before it reaches a provider
`services/gatekeeper`	Decide whether an action is allowed; issue signed authorisation tokens
`services/publisher`	Verify that authorisation before anything reaches the outside world
`services/vault`	Be the system of record, with taxonomy, consent and retention on every object
`services/registry`	Version, sign and evaluate the agent definitions themselves
`services/analytics-ingest`	Close the loop: measure what the agents produced
`services/telemetry-lib`	Make every step of the above observable, with PII structurally excluded

That ratio — **7 governance/measurement services to 1 execution service** — is the product. What has actually been built is *AI governance infrastructure that happens to be pointed at marketing*.

And even "one execution service" overstates it. The orchestrator is a generic DAG executor; it contains no marketing knowledge of its own. **Every line of marketing logic in the platform lives in five handler functions in a single file** — `services/orchestrator/orchestrator/dispatch.py` , lines 443–1023, **395 lines of executable Python out of 21,182 non-test Python lines (1.9%)**. The domain knowledge those handlers act on ships as five content files staged into the container image (`services/orchestrator/Dockerfile` L102–106) plus the permission register — **607 lines** in total. That is the complete marketing payload of the running system.

2. Who it is for

Read literally off the code, there are three distinct user populations:

(a) The autonomous agents themselves. This is the primary "user". The codebase repeatedly uses the term *agent-native* as a design requirement (`services/gatekeeper/main.py` : "Agent-native by construction (AC-17): everything a human can observe here, an internal agent caller can observe over plain HTTP"; `console/README.md` : "send `Accept: application/json` on any GET route"). Every governance surface has a machine-readable equivalent. There is no human-only step in the critical path.

(b) The operator / marketing ops lead. Served by `console/` — one externally-reachable web surface with six read screens (task queue, trace timeline, approval inbox, Vault search, cost ledger, kill switch) and exactly one write action: `POST /kill-switch/toggle` (`console/app/routes_write.py` — "The ONLY mutating route in the entire console app").

(c) The approver. A named human who receives an Adaptive Card in Microsoft Teams (`services/gatekeeper/app/teams_client.py`) or an inbox row, and clicks a single-use, 24-hour-expiring, Entra-ID-authenticated deep link (`services/gatekeeper/app/routers/approval_action.py`). Critically: *possession of the link is never identity* — the approver recorded is the Easy-Auth principal on the request (`services/gatekeeper/app/auth.py`).

The *customer* the marketing output is aimed at is documented in `docs/positioning.md` : **the office of the CFO in multi-entity groups**, South Africa and Southern Africa. The operating company is Canvas Intelligence, a Chartered-Accountant-founded Microsoft Fabric data engineering firm.

3. What business problems it solves

Four, in descending order of how much code is dedicated to them:

Problem 1 — "We cannot let AI touch the outside world without control." Solved by a five-layer defence chain, each layer independently auditable: autonomy policy (`services/gatekeeper/policy/autonomy.yaml` , fail-closed at level 0) → human approval (`governance.approval_inbox`) → signed gate token (`contracts/gate-token/spec.md` , RS256, `jti` , `exp` , content-hash-bound) → independent hash re-computation at the boundary (`services/publisher/app/hashing.py`) → single-use `jti` ledger in Postgres (`services/publisher/app/jti_ledger.py`). Plus a kill switch checked uncached on *every* decision and *every* publish (`services/gatekeeper/app/kill_switch.py`).

Problem 2 — "We cannot let client/personal data leak to a US LLM provider." Solved by a redaction firewall that runs *before* any provider adapter call (`services/model-gateway/redaction.py`), scanning every non-system message role and the `tools[]` passthrough against patterns from a frozen contract (`contracts/model-gateway/redaction-rules.yaml` : SA ID numbers, SA phone numbers, emails, name-shapes). Every block writes a `gate_decisions` audit row. Plus consent gating at the data layer: any Vault write carrying `data_subject_ref` is rejected 403 unless an active `consent_register` row matches subject+channel+purpose (`services/vault/vault/consent.py`).

Problem 3 — "We cannot let AI spend money unpredictably." Solved by per-function daily budgets (`services/model-gateway/policy/budgets.yaml`) with a **downgrade-don't-block** soft breach (opus→sonnet→haiku) and a hard-breach that queues the request as an escalated `gate_decisions` row and returns 429 with that row's id (`services/model-gateway/budget.py` , `completion.py`). Every completion writes three `costs` rows — usd, tokens, ms (`services/model-gateway/metering.py`) — giving an unbroken roll-up chain `costs → agent_runs → campaigns` .

Problem 4 — "We cannot let AI say something we can't defend." Solved by a default-deny client-naming register (`docs/permission-register.yaml` — *nothing is CLEARED today*, and absence blocks identically to explicit UNCLEARED, proven by a self-test in `functions/02-brand-steward-qa/permission_check.py`), plus a Brand Steward QA agent (function 02) whose `pass: false` verdict is *terminal* — the orchestrator transitions the task to `FAILED` with reason `qa_blocked` and **never calls** `advance_dependents` , so the downstream approval task can never see the asset (`services/orchestrator/orchestrator/dispatch.py` , `qa_review_handler`).

4. What category of software this is

The honest answer is that it spans three Gartner categories and is not cleanly any one of them. See `08-product-positioning.md` for the full argument. In one line:

> **An AI Agent Orchestration and Governance Platform with a vertical marketing-operations application layer.**

The governance half (gatekeeper, model-gateway, publisher, vault, registry) is horizontal and domain-agnostic. The application half (23 function packages, 3 loop definitions, 3 MCP servers) is marketing-specific. The boundary between them is clean enough that the governance half could be lifted out — which is the strategic option this codebase has accidentally created.

5. How the platform works — the one-paragraph version

A **Logic App** fires a recurrence trigger on a schedule (`infra/modules/scheduling/`) and drops a **heartbeat event** onto an Azure Service Bus `event` queue. The **orchestrator**'s background worker loop picks it up, loads the matching **loop definition** YAML (`services/orchestrator/loops/*.yaml`), validates it as a DAG (`loop_loader.py` , Kahn's algorithm), and **deterministically decomposes** it into tasks whose ids are `uuid5(event_id, task_id)` — so the same heartbeat always produces the same task ids. Each task is persisted to `task_state` and published as a **task envelope** carrying *metadata only, never content* (`contracts/service-bus/spec.md`). The worker consumes those envelopes; `dispatch.py` routes each `task_type` to a handler. A handler reads its predecessor's `result_ref` by walking the `depends_on` lineage, loads a **prompt** from a function package (`functions/NN-*/prompt.md`), calls the **model-gateway** (which routes, redaction-scans, budget-checks, calls Anthropic, and meters three cost rows), parses the JSON response, writes the artefact to the **Vault** (content-addressed blob + taxonomy + retention policy), records a small `result_ref` , and advances its dependents. When a task reaches `request-approval` , it calls the **Gatekeeper**'s `/gate-check` , which evaluates the autonomy policy, writes a `gate_decisions` row, creates a single-use approval link, and posts a Teams card. A human clicks; the approver is taken from Easy Auth, not the link. The next gate-check finds the approval and issues a **signed gate token** binding `content_hash` + `function_id` in canonical JSON. The **Publisher** verifies the signature with a pinned algorithm, re-checks the kill switch, **independently recomputes the hash over the raw bytes**, cross-checks the Vault's own stored hash, burns the `jti` , and only then — and only if not in dry-run — calls Buffer via **mcp-buffer**, which can only ever create *drafts*. Every step emits an OpenTelemetry span with five mandatory attributes from a closed enum. Overnight, **analytics-ingest** pulls Buffer / GA4 / Search Console / LinkedIn metrics, reconciles UTM tags, computes four KPI rollups including *cost per accepted asset*, and exports a validated JSON payload to blob for Microsoft Fabric.

That paragraph is the product. Everything else is detail.

6. Current state, honestly

- **Live and proven end-to-end:** the daily signal loop runs in Azure (`cmos-dev`), makes real Anthropic calls, meters real costs, and produces real approval cards. `.compound/index.md` L-0027 records the capstone: "the full deploy-infra → deploy-gateway → live completion → metering chain is proven end to end."

- **Deliberately dry-run:** publishing. `PUBLISHER_DRY_RUN` defaults true (`services/publisher/app/config.py`), and a proof-circuit-tagged asset is *forced* dry-run regardless of the flag. Nothing has been published to a live channel by this system.
- **Stubbed:** `services/publisher/app/vault_adapter.py` is an in-memory spy, not a real Vault write. `functions/task-worker/` is a health-check placeholder. The console's Gatekeeper client is still `mock` because no REST wrapper exists over `kill_switch.py` / `approval_inbox.py` (`console/README.md` documents this precisely).
- **Not built:** authentication on the Vault API (network isolation only — `docs/accepted-risks.md`), authorisation-by-role on the console (any authenticated tenant user reaches the kill switch), multi-tenancy, and any notion of a customer other than Canvas Intelligence itself.

7. The single most important structural observation

This codebase was **built by an AI agent loop, about an AI agent loop.**

`.compound/learnings/` holds 79 numbered, classified, cross-referenced learnings with strengthening/recurrence annotations. `README.md` documents a worktree-per-session development model. Commit messages carry finding codes (`F-CASCADE-QA-BLOCKED` , `F-INGEST-PUBLIC-SOURCE`). Code comments cite "heartbeat round 17" and "deploy-loop-e2e-smoke #33" — live production incidents, root-caused inline, with the fix's reasoning preserved next to the fix.

That is not a code smell. It is an **organisational memory system**, and it is arguably a more differentiated asset than the marketing platform itself. See `07-operating-model.md` .

01

System Architecture

Reverse-engineered from `infra/` , `services/` , `mcp/` , `console/` , `contracts/` *and* `.github/workflows/` *. Diagrams are Mermaid.*

---

1. Architectural style

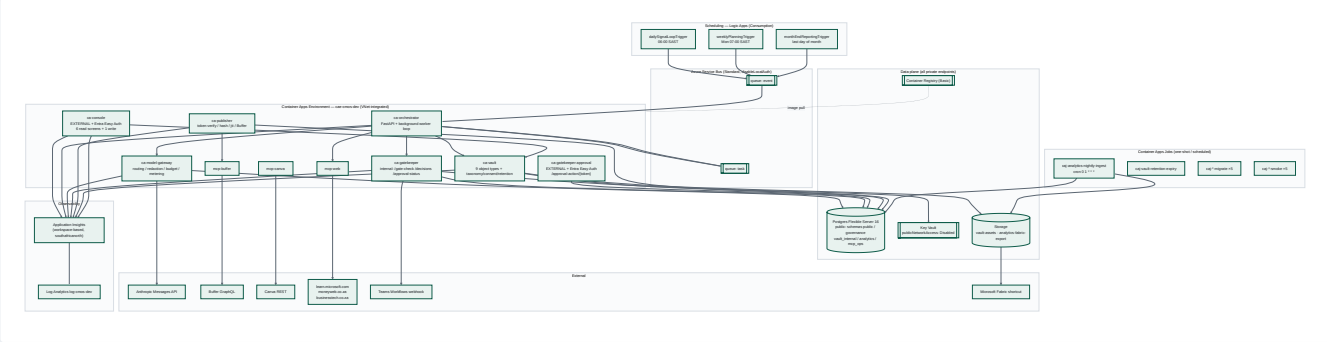
Three styles are layered, deliberately:

1. **Contract-first, frozen-interface architecture.** `contracts/` holds ten files hash-pinned in `contracts/.frozen-v1.sha256` and enforced by `scripts/validate_contracts.py` in CI. Services read these contracts *at runtime*, not just at build time — `services/model-gateway/completion.py` validates every request against the `CompletionRequest` schema read out of `contracts/model-gateway/openapi.yaml` ; `redaction.py` compiles its patterns from `contracts/model-gateway/redaction-rules.yaml` ; `orchestrator/loop_loader.py` validates loop YAML against `contracts/orchestrator/loop-definition.schema.json` . **The contract is not documentation — it is the running validator.**

2. **Choreographed, event-driven services.** Two Service Bus queues (`event` , `task`) carry all inter-service work. No service calls another to *start* work; they publish. Synchronous HTTP is used only for *queries and decisions* (gateway completions, gate checks, Vault CRUD).

3. **Policy-as-data.** Every governance decision is driven by a YAML file that a human can read and diff: `gatekeeper/policy/autonomy.yaml` , `model-gateway/policy/routing.yaml` , `model-gateway/policy/budgets.yaml` , `contracts/model-gateway/redaction-rules.yaml` , `docs/permission-register.yaml` , `orchestrator/loops/*.yaml` , `functions/09-*/fetch_sources.yaml` . Changing platform behaviour is a reviewed one-line data change, not a code change. `routing.yaml` 's own header states this explicitly: *"a model upgrade is one reviewed line, not a code change."*

2. The whole system



3. Layer-by-layer

3.1 Frontend / UI

There is exactly **one** human UI: `console/` . It is a server-rendered FastAPI + Jinja2 app. There is no SPA, no React, no build step, no JavaScript framework — seven templates in `console/app/templates/` (`base.html` , `tasks.html` , `trace.html` , `approvals.html` , `vault_search.html` , `costs.html` , `kill_switch.html`).

The defining design choice is **dual-surface rendering**: `console/app/rendering.py` 's `render_or_json(request, template, data, templates)` serves the *same* `data` dict as HTML or JSON based on the `Accept` header. `console/app/services.py` is the single source of truth for both. This is what makes the console agent-native — an agent and a human get byte-equivalent information.

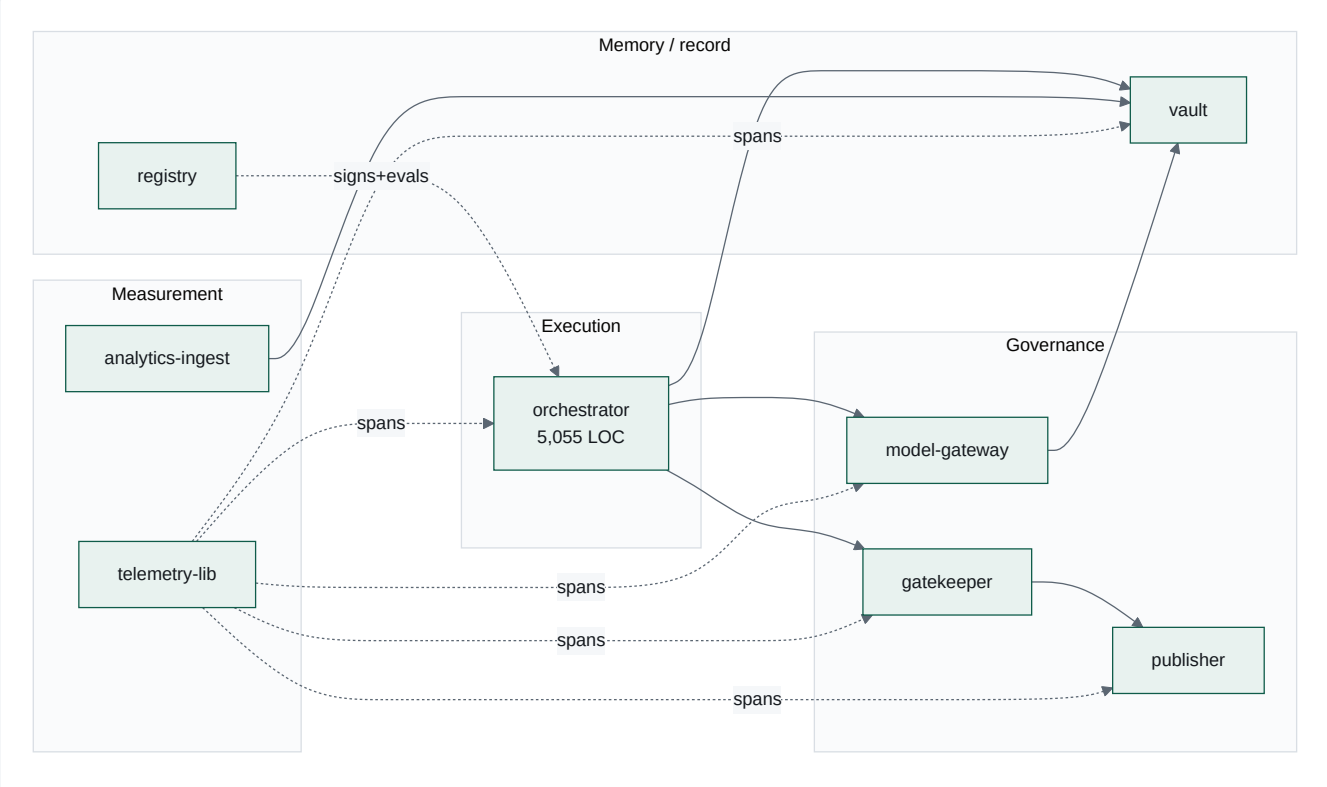
Route inventory (`console/app/routes_reads.py` , `routes_write.py`):

METHOD	PATH	SCREEN
GET	`/`	redirect → `/tasks`
GET	`/tasks`	Task queue (agent runs)
GET	`/tasks/{task_ref}/trace`	Trace timeline (App Insights KQL)
GET	`/approvals`	Approval inbox (read-only)
GET	`/vault-search`	Taxonomy-filtered Vault search
GET	`/costs`	Cost ledger (`group_by=function|day`)
GET	`/kill-switch`	Kill-switch state + last audit entry
POST	`/kill-switch/toggle`	**the only mutating route**
GET	`/health`	unauthenticated probe

Two hardening details worth noting: `require_principal` is applied as a FastAPI `Depends` on *every* route as a **code-level backstop** to the Bicep-wired Easy Auth layer (`RISK-003` — defence in depth against infra drift); and the toggle route accepts both JSON (agent) and form-urlencoded (browser), with an Origin/Referer same-origin check applied only to the form path as CSRF defence (`_same_origin_or_reject`).

3.2 Backend services

All eight are Python 3.12. Six are FastAPI HTTP services; two (`registry` , `telemetry-lib`) are libraries/CLI toolchains.



Ingress posture (`infra/modules/**`):

APP	INGRESS	AUTH
`ca-console`	**external**	Container Apps Easy Auth (Entra ID), FIC-secretless
`ca-gatekeeper-approval`	**external**	Easy Auth, `unauthenticatedClientAction=Return401`
`ca-gatekeeper`	internal	none (network isolation)
`ca-publisher`	internal	none
`ca-model-gateway`	internal	none
`ca-vault`	internal	**none — documented accepted risk**
`mcp-web` / `mcp-buffer` / `mcp-canva`	internal	none

Only **two** surfaces are reachable from the internet, and both are Entra- protected. The route separation between `ca-gatekeeper` (internal, holds `/gate-check`) and `ca-gatekeeper-approval` (external, holds only `/approval-action`) is enforced by *two separate FastAPI app objects* (`main.py` vs `approval_main.py`) — the docstring is explicit: *"Two separate app objects means route separation cannot be lost to a misconfigured flag."*

3.3 Database

One Postgres 16 Flexible Server (`Standard_B1ms`, Burstable, `publicNetworkAccess: Disabled`, private endpoint into `snet-pe`), with **five schemas** owned by different services:

SCHEMA	OWNER	TABLES	MIGRATION
`public` (9 tables)	Vault (frozen)	signals, opportunity_cards, briefs, assets, campaigns, agent_runs, gate_decisions, costs, consent_register	`contracts/vault-schema/schema.sql` (hash-frozen)
`public` (2 tables)	Orchestrator (additive)	task_state, task_transitions	`services/orchestrator/migrations/0001-0004`
`vault_internal` (7)	Vault sidecar	object_taxonomy, consent_linkage, audit_log, retention_policy, retention_run, access_log, utilisation_daily	`services/vault/migrations/0001`
`public` (3 tables)	Vault (additive v2)	option_cards, approval_decisions, standing_permissions — the newer, general "ratification model," coexisting with `approval_inbox` / `gate_decisions`	`services/vault/migrations/0002-0003`
`governance` (6)	Gatekeeper + Publisher	schema_migrations, kill_switches, approval_inbox, approval_actions, publish_attempts, jti_ledger	`infra/modules/governance/migrations/0001`
`analytics` (11)	analytics-ingest	4 raw fact tables, utm_campaign_map, utm_quarantine, scheduled_posts, 4 kpi_rollup_*	`services/analytics-ingest/migrations/0001`
`mcp_ops` (1 table)	MCP servers	tool_calls	`mcp/mcp_ops/schema.sql`

The `vault_internal` split is the single most consequential schema decision. `contracts/vault-schema/schema.sql` is hash-frozen, so taxonomy, consent linkage, retention and utilisation *could not* be added as columns. They went into a sidecar schema instead, and the Vault API stitches them back together with a `LEFT JOIN` on every read (`services/vault/vault/routers/objects.py`, `fetch_object`). This is recorded as an accepted risk with a deferred v2 consolidation path (`docs/accepted-risks.md`).

Every migration is applied by an in-VNet **Container Apps Job**, and every one base64-encodes its SQL because *Container Apps collapses a literal `$$` in secret values to `$`*, which corrupts PL/pgSQL dollar-quoting (learning L-0012). CI reproduces that exact encoding round-trip before applying (`.github/workflows/ci.yml`).

3.4 Queues

Two Service Bus queues, both `lockDuration: PT1M`, `maxDeliveryCount: 10`, `deadLetteringOnMessageExpiration: true` (`infra/modules/service-bus.bicep`).

- **`event`** carries two discriminated message shapes: `HeartbeatEvent` (has `event_type: "heartbeat"`) and `DeadLetterAlert` (has `alert_version`). `worker.py::_event_message_kind` routes between them — a fix (`F-EVENTQ-DISCRIMINATE`) added after every dead-letter produced 8 Pydantic errors in the logs.
- **`task`** carries `TaskEnvelope` — **metadata only**. This is the compensating control for running Standard SKU without a private endpoint: `contracts/service-bus/spec.md` states the blast radius of the public endpoint is limited to task metadata because content is referenced by id and fetched from the Vault.

The retry state machine is entirely application-level and deliberately decoupled from the transport. `services/orchestrator/orchestrator/state_machine.py`: `retry_count` lives in `task_state`, backoff is `2 * 2^(attempt-1) + jitter`, dead-letter fires at exactly the 3rd failure, and the whole thing is idempotent against redelivery (short-circuits if already `dead_lettered`).

3.5 Scheduling

Five Consumption Logic Apps *(was three; updated 17 Aug 2026)*, each with its own SystemAssigned identity and its own Service Bus Data Sender role assignment (`infra/modules/scheduling/*.bicep`), on `South Africa Standard Time` :

LOGIC APP	FIRES	LOOP
`la-daily-signal-loop-trigger`	06:00 SAST daily	`daily-signal-loop`
`la-weekly-planning-trigger`	**07:00 SAST daily** — see below	`weekly-content-loop`
`la-month-end-reporting-trigger`	last day of month	`month-end-reporting`
`la-publish-trigger`	per Bicep	`publish-loop`
`la-source-discovery-trigger`	per Bicep	`source-discovery-loop`

Plus one Schedule-triggered Container Apps Job for analytics (`caj-analytics-nightly-ingest` , cron `0 1 * * *` = 03:00 SAST).

`weekly-content-loop` **fires daily, and that is deliberate.** The loop id and its `monday-` … `friday-` task-id prefixes are **dependency-chain names, not a schedule**: one heartbeat decomposes the whole graph and runs it as fast as `depends_on` allows, so a daily fire produces one *complete* Mon–Fri content cycle per day, not one weekday's slice per day. The trigger carried a `TEMPORARY` marker and a commented-out weekly block until 17 Aug 2026; both are gone, because the revert they promised was never coming. Sends every reader down the wrong path otherwise — it did exactly that once.

`services/orchestrator/loops/nightly-analytics-ingest-loop.yaml` is explicitly **documentary** — its own header states the real trigger is the Container Apps Job, and the loop file exists as registry metadata. This is an honest, unusual piece of self-documentation. **[INFERRED]** it also signals an intent to eventually unify all scheduling under the orchestrator.

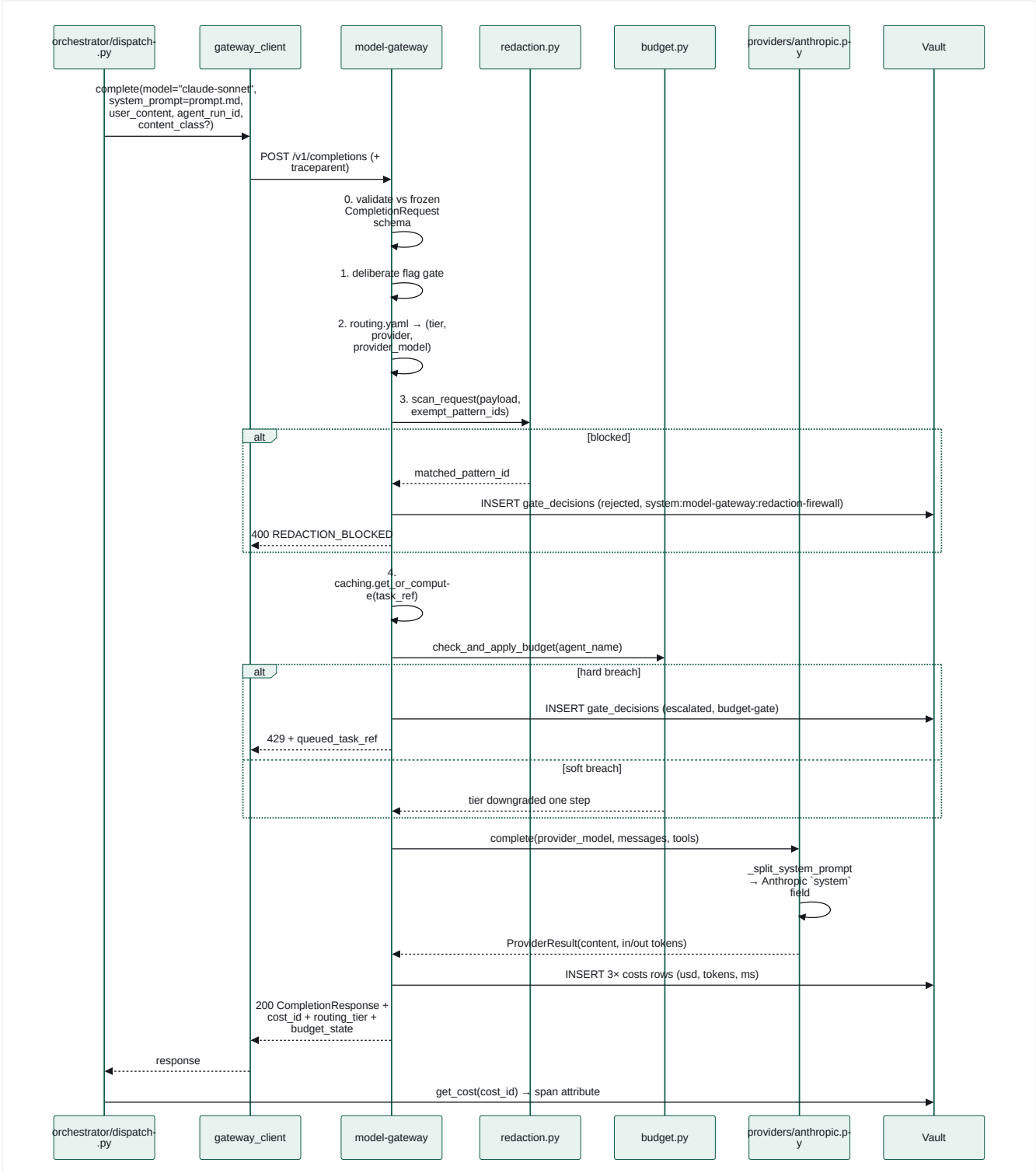
3.6 AI services

How handlers are registered. `DISPATCH_TABLE` maps `task_type` → handler and covers **39 task_types**. Most are literal entries, but the eleven S10 intelligence scanners are built by a factory and merged in as a **dict spread**:

```
SCANNER_TASKS: dict[str, tuple[str, str, str]] = {
    # task_type: (function_id, default profile_id, agent_name)
    "competitor-discovery-scan": ("10-competitor-discovery-scanner", ...),
    ...
}
SCANNER_HANDLERS = {t: _make_scanner_handler(...) for t in SCANNER_TASKS}
DISPATCH_TABLE = { **SCANNER_HANDLERS, "ingest-signals": ..., ... }
```

Their scan scope is data, in `functions/_shared/scan-profiles.yaml` . They deliberately do **not** write straight to `opportunity_cards` : the eleven share three listening scopes, so one event legitimately surfaces several times, and de-duplication is `dedupe-signal-cards` ' job.

> **Reading this table programmatically:** a grep for `"task-type":` inside `DISPATCH_TABLE` misses all eleven factory-registered scanners and reports them as unwired no-ops. Resolve `SCANNER_TASKS` first. This has already produced one false audit result.



Provider extensibility is one register call. `providers/base.py` is a one-method `Protocol`; `providers/registry.py` is a name→class map; `routing.yaml` names the provider as a string. No vendor name appears in `config.py`, `completion.py`, or `routing.py`. Adding a second LLM vendor is a new module + one `register()` + a YAML edit.

3.7 Memory

There are **four distinct memory systems**, and they are not interchangeable:

MEMORY	STORE	LIFETIME	PURPOSE
Task state / run memory	`task_state.result_ref` (jsonb)	run	How a task hands a *pointer* (not content) to its successor. Resolved by `resolve_lineage_result()` walking `depends_on` up to 6 hops
Artefact memory	Vault `public` + blobs	retention-class bounded (30d/1y/3y/legal hold)	The durable system of record: signals, briefs, assets, agent_runs
Institutional memory	`.compound/learnings/**` (79 files)	permanent	Cross-session engineering knowledge, classified `architecture/conventions/known-hard/security`
Strategic memory	`docs/positioning.md`	manual	The "Tier-2 strategy source of truth" that function prompts quote verbatim

There is no vector store, no embeddings, no RAG, and no conversational memory anywhere in this repository. Agent context is assembled deterministically from: a static `prompt.md` + a structured `user_content` JSON built by the dispatch handler + content fetched by id from the Vault. That is a significant architectural stance — see `05-ai-architecture.md`.

3.8 Knowledge

Knowledge enters the system through exactly one door: **mcp-web's `fetch_url` tool**, restricted to an allowlist. The sources are named in `functions/09-market-intelligence-director/fetch_sources.yaml` (3 domains: `learn.microsoft.com`, `moneyweb.co.za`, `businesstech.co.za`), and the allowlist is mirrored into `MCP_WEB_ALLOWLIST` in `infra/main.bicep`. `web_search` is declared in function 09's `tools.yaml` but **not implemented** in mcp-web — the fetch_sources.yaml header documents this as a config-only future activation path.

3.9 Storage

- **Content-addressed blob storage** for assets (`services/vault/vault/storage.py`): blob name = SHA-256 hex of the bytes. `exists()` before write, so identical bytes never duplicate; a concurrent `ResourceExistsError` is treated as successful dedup. Deletion is reference-counted against `assets.content_hash`.
- **`analytics-fabric-export`** container for the nightly Fabric payload.
- Managed identity only — no storage keys, no connection strings.

3.10 Reporting

Two independent reporting paths:

1. **Operational** — `console/costs`, `console/tasks`, `console/tasks/{ref}/trace`, plus `GET /utilisation/rollup` on the Vault (daily reads per object_class per caller_service).
2. **Analytical** — `analytics-ingest` nightly: 4 sources ingested → UTM reconciliation (unmatched → `analytics.utm_quarantine` with a reason) → 4 KPI rollups → schema-validated Fabric export (`analytics/contracts/fabric-nightly-export.schema.json`) → blob → Power BI starter dataset (`analytics/powerbi/analytics-dataset.json`).

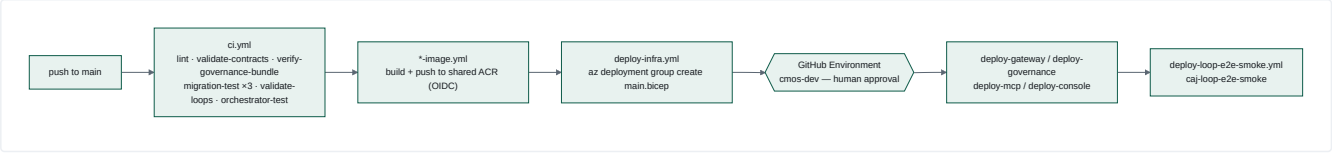
The four KPIs are the platform's own scorecard: `engagement_rate_by_post_archetype`, `publishing_reliability`, **`cost_per_accepted_asset`**, `vault_utilisation`. The third one is the tell — it is a *unit-economics-of-AI-labour* metric, computed as `SUM(costs) / COUNT(assets WHERE approval_state='approved')` joined through `agent_runs.agent_name` (`analytics_ingest/rollup.py`).

3.11 Messaging / notifications

CHANNEL	MECHANISM	STATE
Teams approval card	Adaptive Card v1.4 via Power Automate Workflows webhook (`gatekeeper/app/teams_client.py`)	**Flag-gated** — `TEAMS_WEBHOOK_URL` absent by default; falls back to the inbox row
Teams brief-ready	`orchestrator/teams_notify.py`	Flag-gated, same webhook
Approval inbox	`governance.approval_inbox` row	**Always written** — the row *is* the delivery when no webhook
Dead-letter alert	`DeadLetterAlert` on the `event` queue	Emitted and observable; nothing consumes it yet
Email / SMS / Slack	—	**Not built**

Classic O365 connector webhooks were retired May 2026, so the code targets Workflows HTTP triggers with Adaptive Cards, and uses `Action.OpenUrl` deep links rather than submit-style postbacks — deliberately, because *"a submit-style postback would make 'who clicked' a claim of the card payload rather than an authenticated identity"*.

3.12 Deployment



13 workflows. Zero client secrets anywhere — **OIDC federated identity only** (`AZURE_CLIENT_ID` / `AZURE_TENANT_ID` / `AZURE_SUBSCRIPTION_ID`). ACR admin user is disabled; image pull is by managed identity with an explicit `AcrPull` role assignment.

Two patterns here are hard-won and worth naming (they appear as learnings L-0018, L-0048, L-0049, L-0060, L-0061):

- **Image bootstrap contract**: every Bicep `containerImage` param defaults to a *public MCR placeholder*, and the deploy workflow's preflight resolves the app's *current live image* — but only if `latestReadyRevisionName` is non-empty. Otherwise it re-bootstraps.
- **Never declare `registries[]` on a new `Microsoft.App/jobs` resource** — a user-assigned identity cannot be newly attached and simultaneously referenced for ACR pull in the same create.

3.13 Observability

`services/telemetry-lib` is a shared package adopted by all six services. Its distinguishing feature is that the span attribute API is a **closed enum, not a dict**:

```
class SpanAttributeKey(str, Enum):
    FUNCTION_ID, TASK_REF, MODEL, REGISTRY_VERSION, COST   # required on every span
    STATUS, DURATION_MS, ERROR_CODE                        # optional allowlist
```

`set_span_attribute()` raises `ValueError` on an unrecognised key **and on any string value over `MAX_TEXT_LEN = 200`** — free-text-shaped values are structurally rejected before they reach the exporter. This mirrors the Service Bus redaction convention into telemetry. It is the same principle applied a third time (queues, prompts, spans): *content never travels; only identifiers do.*

W3C `traceparent` is injected on every outbound call (`orchestrator/clients/*`) and joined on every inbound request (each service's `telemetry_wiring.py`), so one `task_ref` stitches the whole chain in App Insights.

4. Cross-cutting invariants (the platform's real "architecture")

These seven rules hold everywhere, and are individually test-enforced:

1. **Fail closed.** Autonomy default level 0; permission register default deny; `VaultLookupError` refuses to publish; missing consent → 403; unknown gate-token alg → reject.
2. **Content never travels; ids do.** Queues, spans, `result_ref` , `metadata` bags.
3. **Append-only audit.** `gate_decisions` has no `updated_at` *by design*; `task_transitions` , `approval_actions` , `publish_attempts` , `vault_internal.audit_log` are insert-only. Rejection audits are written on an *isolated connection* (`write_audit_isolated`) so they survive the transaction rollback that the rejection itself causes.
4. **Uncached where correctness depends on freshness.** Kill switch is a fresh SELECT on every gate decision and every publish; `/status` reads the DB with no cache. Autonomy policy *is* cached — deliberately, because it is a build-time artefact.
5. **Never guess a hostname.** Every FQDN is resolved live via `az containerapp show` or an env override (learning L-0025).
6. **Idempotent migrations, applied twice in CI.**
7. **Two implementations, one behaviour, proven by a parity test.** The kill switch is duplicated in gatekeeper and publisher (they share no library); `test_kill_switch_parity.py` loads *both files* and asserts identical behaviour across the full scope matrix.

02

Module Catalogue

Every module in the repository. "Maturity" is assessed on evidence in the code: tests present, deployed, proven live, or stubbed.

Maturity scale used throughout:

- **L4 Proven live** — deployed and verified against `cmos-dev` with evidence in code comments/learnings
- **L3 Deployed** — Bicep + workflow exist, unit-tested, not independently proven end-to-end
- **L2 Built & tested** — real implementation with tests, not wired to infra
- **L1 Scaffold** — shape exists, behaviour stubbed
- **L0 Declared only** — named but not implemented

M1 — Orchestrator (`services/orchestrator`)

Purpose	The execution engine. Turns a schedule tick into a completed, governed multi-agent workflow.
Business capability	Marketing Work Orchestration; Autonomous Task Execution
Maturity	**L4 Proven live** — `ca-orchestrator` , 19 test modules, real production incident history in comments

Features

- Loop registry: loads `loops/*.yaml` on startup, JSON-Schema-validates each, checks acyclicity via Kahn's algorithm (`loop_loader.py`)
- Deterministic decomposition: `uuid5(heartbeat.event_id, loop_task_id)` — same heartbeat ⇒ same task ids, enabling golden-file tests and smoke-test correlation (`decompose.py`)
- Dependency-ordered dispatch with `dispatchable` gating (`db.advance_dependents`)
- Application-level retry/backoff/dead-letter state machine, decoupled from Service Bus `maxDeliveryCount` (`state_machine.py`)
- Cascade dead-letter: a task blocked on a permanently-failed dependency dead-letters *immediately* instead of burning ~15 min through requeue + 3-strike (`DependencyDeadLetteredError`)
- Five real task handlers + a byte-identical legacy pass-through for every other task_type (`dispatch.py`)
- Lineage resolution: BFS up `depends_on` to find the nearest ancestor carrying a `result_ref` — transparently walks *past* unhandled pass-through stages
- Agent-native run state: `GET /runs/{task_ref}` returns every stage, span presence, and the **real** human approval status (distinct from the request-approval task's own always-COMPLETED state)

Pages / surfaces: `GET /health` , `GET /status` , `GET /runs/{task_ref}`

Data used: `task_state` , `task_transitions` (owned); Vault via REST; `governance.approval_inbox` via Gatekeeper

Dependencies: Service Bus, Postgres, model-gateway, Vault, Gatekeeper, mcp-web, `functions/02,09,42` (staged into the image via `FUNCTIONS_DIR`), `contracts/orchestrator/*` (staged via `CONTRACTS_DIR`)

Users: Logic Apps (producer), itself (consumer), console + smoke jobs (readers)

Inputs: `HeartbeatEvent` , `TaskEnvelope` · **Outputs**: `TaskEnvelope` , `DeadLetterAlert` , Vault objects, gate-check requests

Missing

- No task cancellation, pause, or replay-from-step
- No priority handling — `TaskEnvelope.priority` is in the contract but never read
- `_retry_or_dead_letter` re-invokes handlers that are **not idempotent** — the docstring admits duplicate Vault rows are possible
- No dynamic loop registration: loops are baked into the image
- `DeadLetterAlert` is emitted but nothing consumes it — no alerting, no paging

---

M2 — Model Gateway (`services/model-gateway`)

Purpose	The single, mandatory chokepoint between every agent and every LLM provider.
Business capability	AI Cost Control; Data-Loss Prevention; Model Portability
Maturity	**L4 Proven live** — real Anthropic calls + real metering proven (learning L-0027)

Features (in execution order, `completion.py`)

1. **Step 0** — Runtime validation against the frozen `CompletionRequest` schema — *read out of the contract file, never hand-copied*
2. `deliberate` reasoning-hint feature flag → explicit `NOT_IMPLEMENTED`, never silently ignored
3. Routing: logical id → (risk tier, provider, provider_model) from `policy/routing.yaml`
4. **Redaction firewall** with per-content-class pattern exemptions
5. `task_ref` idempotency cache — in-flight futures + bounded `OrderedDict` (10k), so two concurrent requests for one `task_ref` produce one upstream call
6. Budget: soft breach downgrades a tier, hard breach queues + 429
7. Provider call, then 3 `costs` rows
8. One structured JSON log line per request, on every path

Startup behaviour worth noting: `_validate_routing_against_live_models()` calls Anthropic's `/v1/models` once per process start and **logs** (never raises) if a configured `provider_model` has been retired. Born from learning L-0026, where every originally-configured model id turned out to be retired.

Data used: Vault `costs`, `gate_decisions` (writes only)

Missing

- Cache is process-local — multi-replica double-spend is possible and explicitly declared out of scope
- No streaming, no tool-use loop, no multi-turn conversation
- `PRICE_PER_MTOK` is a hardcoded dict — prices drift silently
- Only one provider implemented despite the extension point

M3 — Gatekeeper (`services/gatekeeper`)

Purpose	Decide whether an agent may perform an action; obtain and record human approval; mint the authorisation token.
Business capability	Autonomy Policy Management; Human-in-the-Loop Approval; Emergency Stop
Maturity	**L4 Proven live** — `caj-governance-smoke` exercises the real level-1 path

Features

- Autonomy levels 0–4 mapped from `(function_id, action_class)`, fail-closed default 0 (`policy/autonomy.yaml`)
- Exactly one `gate_decisions` row on **every** branch, with a distinct `reason` string per branch
- Kill switch checked **first**, uncached, on every request
- Single-use, 24h-expiring approval links; consumption is an atomic conditional `UPDATE ... AND link_consumed_at IS NULL`
- Approver identity from Easy Auth headers, never from link possession
- Four distinguishable click outcomes, each with its own `approval_actions` audit row: `approved`, `rejected`, `link_expired`, `link_already_used`
- Gate token: RS256, Key Vault-held signing key, `function_id` + `content_hash` packed as **canonical JSON** in the `resource` claim (because the frozen schema sets `additionalProperties: false`)
- `GET /approval-status` — reports the *real* pending/approved/rejected/expired state, which `GET /decisions/{id}` structurally cannot

Surfaces: internal `POST /gate-check`, `GET /decisions/{id}`, `GET /approval-status`, `GET /healthz` ; external `GET /approval-action/{link_token}?choice=`

Missing

- Level 2 ("elevated") is implemented identically to level 1 — quorum/second-approver is reserved, not built
- No REST API over kill switches or the approval inbox — this is why the console still runs its Gatekeeper client in `mock` mode
- No delegation, escalation timeout, or approval reminder
- No per-function kill switch UI (the table supports `scope='function'` ; nothing surfaces it)

Deliberate absence worth documenting: there is *no* `smoke.*` or `test.*` entry in `autonomy.yaml`, and a test enforces that no `publish`-class entry sits above level 2. A previous `smoke.governance_cycle` level-4 entry was removed because `/gate-check` authenticates no caller — it was a standing auto-approve backdoor (learning L-0029).

M4 — Publisher (`services/publisher`)

Purpose	The last gate before the outside world. Verifies authorisation and refuses with a recorded reason.
Business capability	Publication Control; Non-repudiation
Maturity	**L3 Deployed** — 21 test modules; the actual Vault write is a **stub**

The refusal matrix — one `publish_attempts` row on every branch:

CONDITION	OUTCOME	REASON
no token	rejected	`token_absent`
alg not pinned (incl. `none`, HS256 confusion)	rejected	`invalid_alg`
expired	rejected	`token_expired`
malformed / forged / non-canonical `resource`	rejected	`token_invalid`
kill switch active	rejected	`kill_switch_active:<scope>[:fn]`
recomputed hash ≠ bound hash	rejected	`content_hash_mismatch`
`asset_id` given, Vault lookup failed	rejected	`vault_lookup_failed`
Vault's stored hash disagrees	rejected	`content_hash_mismatch`
`jti` already consumed	rejected	`token_replayed`
live mode, Buffer queue ≥ 10	rejected	`buffer_queue_cap_exceeded`
dry-run (default, or proof-circuit forced)	published	`published_dry_run`
all checks pass, live	published	`published`

The ordering is itself a security control and is documented as such: kill switch *after* token verification but *before* jti consumption (so a pre-issued token cannot outlive an operator flipping the switch); Vault cross-check before the jti burn (so a failed lookup doesn't spend the token); jti burned last (so a token refused for any other reason is not wasted).

Missing

- `vault_adapter.py` is `StubVaultRecordingAdapter` — an in-memory list. **The "publish record" is never persisted to the Vault.** This is the single largest functional gap in the governance chain.
- Only LinkedIn via Buffer. `BUFFER_LINKEDIN_CHANNEL_ID` is hardcoded in config despite the weekly loop's YAML carrying three channel ids.
- No scheduling — `create_draft` only, by design (mcp-buffer hardcodes `status="draft"` server-side)

M5 — Vault (`services/vault`)

Purpose	The system of record for every business object, with governance metadata mandatory on write.
Business capability	Master Data Management; Consent Management; Records Retention
Maturity	**L4 Proven live** — deployed, smoke-tested; **zero authentication**

Features

- Generic CRUD factory: 9 object types declared *declaratively* in `models.py` (`OBJECT_TYPES` registry), one router factory generates all of them. `assets` is the only special case (content-addressed blobs).
- **6 mandatory taxonomy fields on every write**: `vertical`, `function_id`, `campaign`, `evidence_grade`, `consent_status`, `retention_class`. Missing/invalid → 422 naming the exact field, *with an audit row*.
- Taxonomy fields are **immutable post-create** — PATCH attempting a change → 422.

- Consent gate: a write carrying `data_subject_ref` requires `consent_channel` + `consent_purpose` and an active matching `consent_register` row, else 403 + audit. On success the object is durably linked to the consent row (`consent_linkage`).
- Single-statement batched create: object row + taxonomy row + retention row in one writable-CTE INSERT.
- Content-addressed dedup blob storage.
- Retention sweep: `expires_at <= now()` → delete + audit, **fails closed on blob-delete error** (row left for retry), `SELECT ... FOR UPDATE SKIP LOCKED` inside an explicit outer transaction so the locks actually hold.
- Utilisation: every GET writes an `access_log` row keyed by `X-Caller-Service` ; a daily rollup upserts `utilisation_daily` .

Retention classes (`retention.py`): `ephemeral_30d` (30d), `standard_1y` (365d), `extended_3y` (1095d), `legal_hold` (year-9999 sentinel, never swept).

Missing / risks

- **No authentication or authorisation on any endpoint.** Network isolation is the only control. Documented as an accepted risk not yet approved by the budget owner.
- `X-Caller-Service` is self-asserted and explicitly *not* a trust boundary.
- List endpoints support only limit/offset — no server-side taxonomy filter, which is why the console filters in Python after fetching everything.
- No soft delete, no versioning API beyond the `predecessor_asset_id` chain, no bulk export, no search.

M6 — Registry (`services/registry`)

Purpose	Treat agent definitions as versioned, signed, testable software artefacts.
Business capability	AI Asset Lifecycle Management; Model/Prompt Governance
Maturity	**L2 Built & tested** — CLI toolchain + CI workflow; **not consumed at runtime**

Features

- `validate_package.py` — 13 named rules over the function-package shape, including `prompt-missing-json-output-contract` , added after function 42's prompt never told the model to return JSON and every production call failed to parse (`F-PROMPT-OUTPUT-CONTRACT` ; a repo-wide sweep found 6 of 23 packages affected)
- `build_registry.py` — canonical-JSON manifest, byte-identically reproducible, Ed25519-signed with a detached signature (deliberately *not* an OCI image)
- `eval_harness.py` — golden eval sets graded against a **mocked gateway driven by each package's own** `prompt.md` , so deleting a rule from a prompt fails the task that grades it. `fixtures/regression/42-linkedin-post-writer-broken/` is a copy with the roof-line rule removed, and it must fail by task id.
- `safety_suite.py` — deterministic brand rules with paired good/bad fixtures
- `lint_rubrics.py` — rejects empty or subjective rubrics

The critical gap: nothing in the running platform reads the registry. `dispatch.py` reads `prompt.md` straight off disk via `functions_dir()` . The `registry_version` span attribute exists in `telemetry-lib` but is populated by each service's own wiring, not by the registry. **The signature is never verified at runtime.**

Also: `registry.yml` CI hardcodes the three original package paths (02/09/42) and does not auto-discover the other 20 (documented in `docs/function-register-coverage.md`).

M7 — Analytics Ingest (`services/analytics-ingest`)

Purpose	Close the loop — measure what the agents actually produced.
Business capability	Marketing Performance Measurement; AI Unit Economics
Maturity	**L3 Deployed** — real nightly job; 3 of 4 sources are fixture-backed

Dual-mode: Buffer goes live when `BUFFER_API_KEY` resolves; GA4, Search Console and LinkedIn are fixture-only (`tests/fixtures/*.json`). The pattern is "goes live automatically once real credentials exist, zero code change" — which learning L-0074 flags as a design that needs explicit verification.

Pipeline (`cli.py nightly`): ingest 4 sources → reconcile every `utm_campaign` against `utm_campaign_map` (unmatched → `utm_quarantine` with a reason) → 4 KPI rollups → schema-validated Fabric export → blob upload.

Idempotency convention is explicit and split by table class: raw fact tables use `UNIQUE(source, day, natural_row_id)` + `ON CONFLICT DO NOTHING` (never clobbers a hand-corrected row); rollup tables use `UNIQUE(day, kpi_name, dims)` + `ON CONFLICT DO UPDATE` (refresh cleanly).

Missing: no attribution model beyond UTM matching; no funnel/pipeline metrics; no CRM integration; no revenue linkage; Power BI dataset is a starter definition with no provisioned workspace.

M8 — Telemetry Lib (`services/telemetry-lib`)

Purpose	Uniform, PII-safe distributed tracing.
Maturity	**L4 Proven live** — adopted by all six services; each has a `test_telemetry_wiring.py`

Closed-enum attribute keys, 5 mandatory attributes, 200-char rejection of free-text values, W3C trace propagation. `emit_synthetic_trace.py` exists so a deploy can prove ingestion works.

M9 — Console (`console/`)

Purpose	The human operator surface.
Business capability	Operational Oversight; Emergency Control
Maturity	**L3 Deployed** — live behind Easy Auth; **reads mostly mock data**

Six read screens, one write action. Dual-surface HTML/JSON. Decimal-only arithmetic in the cost ledger (never float) so a byte-for-byte comparison against an independent aggregation is achievable.

The material gap: `GATEKEEPER_API_MODE` is `mock`. The approval-inbox screen and the kill-switch screen are reading a fixture, not production, because Gatekeeper exposes no REST route over those tables. `console/README.md` documents this honestly and corrects an earlier "config-only cutover" claim.

`VAULT_API_MODE` can be flipped to `real` with two env vars — that cutover *is* config-only and was field-by-field re-verified against the merged contract.

M10 — MCP Tool Plane (`mcp/`)

Purpose	Give agents a governed, uniform way to touch external systems.
Business capability	Tool Governance; Integration Abstraction
Maturity	**L3 Deployed** — three apps, 11 test modules, in-VNet smoke job

Three servers, one protocol (`POST /mcp` , JSON-RPC 2.0: `initialize` , `tools/list` , `tools/call`), no external MCP SDK — a from-scratch FastAPI scaffold (`mcp/common/mcp_common/protocol.py`).

SERVER	TOOLS	GUARDRAIL
`mcp-web`	`fetch_url`	Host allowlist checked *before* any network call; sliding-window rate limiter
`mcp-buffer`	`list_queue` , `get_post` , `create_draft`	**No publish path exists in the manifest.** `create_draft` accepts no status/mode/state argument; the server hardcodes `status="draft"` . A test greps tool names and descriptions against `publish|share.?now|send.?now|go.?live` .
`mcp-canva`	`create_design_from_template` , `bulk_create_from_csv` , `export_design`	**Template-locked** — `template_id` required on both creation tools; no free-form design path

Every tool call is best-effort logged to `mcp_ops.tool_calls` through a bounded shared connection pool — a Postgres outage must never take down a tool call.

Fixture-first by default: live mode activates purely on the presence of a credential env var. No code or config edit.

Missing: `web_search` is declared in function 09's tools.yaml but not implemented; none of the 10 pytest markers are wired into `ci.yml` (documented as a known operational gap).

M11 — Function Definition Packages (`functions/NN-*`)

23 packages. Each is a five-file contract: `prompt.md` · `skill.md` · `tools.yaml` · `schema.json` · `evals/*.json` (+ optional `tool_check.py` , `permission_check.py`).

GROUP	PACKAGES	PURPOSE
QA gate	02	Brand Steward QA — the publish gate
Market intelligence	09	Signal scanning with source attribution
Competitive intelligence	10, 11, 12, 13, 16	Discovery, change monitoring, positioning, content performance, Fabric ecosystem
Vertical intelligence	18-01…18-06	Logistics/fleet, mining/industrial, manufacturing, construction, FMCG/beverage, financial services
Response strategy	25	Severity-ranked competitive response plan
Advocacy	26	Consent-gated testimonial harvesting
Content studio	39, 41, 42, 43, 45, 46, 47, 52	Story editor, research brief, LinkedIn post, executive ghostwriter, carousel, newsletter, case study, repurposer

Maturity: L2 for 23 of them; L4 for exactly 3. Only functions 02, 09 and 42 are staged into the orchestrator image and actually invoked by `dispatch.py`. The other 20 exist as validated, eval-tested packages that the loop YAML references by `task_type` — but those task_types fall through to `legacy_task_pass_through`, which transitions RUNNING → COMPLETED and does nothing. **This is the single biggest execution gap in the platform.**

Notably rigorous details:

- Function 18-03 (Manufacturing) is *deliberately proof-light* because `positioning.md` §4 doesn't name manufacturing as a proof vertical — its evals default `evidence_grade` to `light`, never `strong`.
- Functions 26 and 47 ship their own `permission_check.py` exercising the default-deny path directly rather than assuming it.
- Every `function_id` used for gate-checks is one of the four real `autonomy.yaml` pairs — never an invented identifier, which would fail-closed forever.

M12 — Contracts (`contracts/`)

Ten hash-frozen files + the actively-developed `vault-api.yaml` (1,623 lines, 23 paths, 38 schemas) and `function-definition/tools.schema.json`.

`scripts/validate_contracts.py` validates OpenAPI 3.1 structure, JSON Schema Draft 2020-12 validity, presence of `/vN/` version anchors, the gate-token security-claim requirements, **and** a `check_no_internal_leak` assertion that `vault-api.yaml` never mentions `vault_internal` — the contract must not leak the sidecar-schema implementation detail.

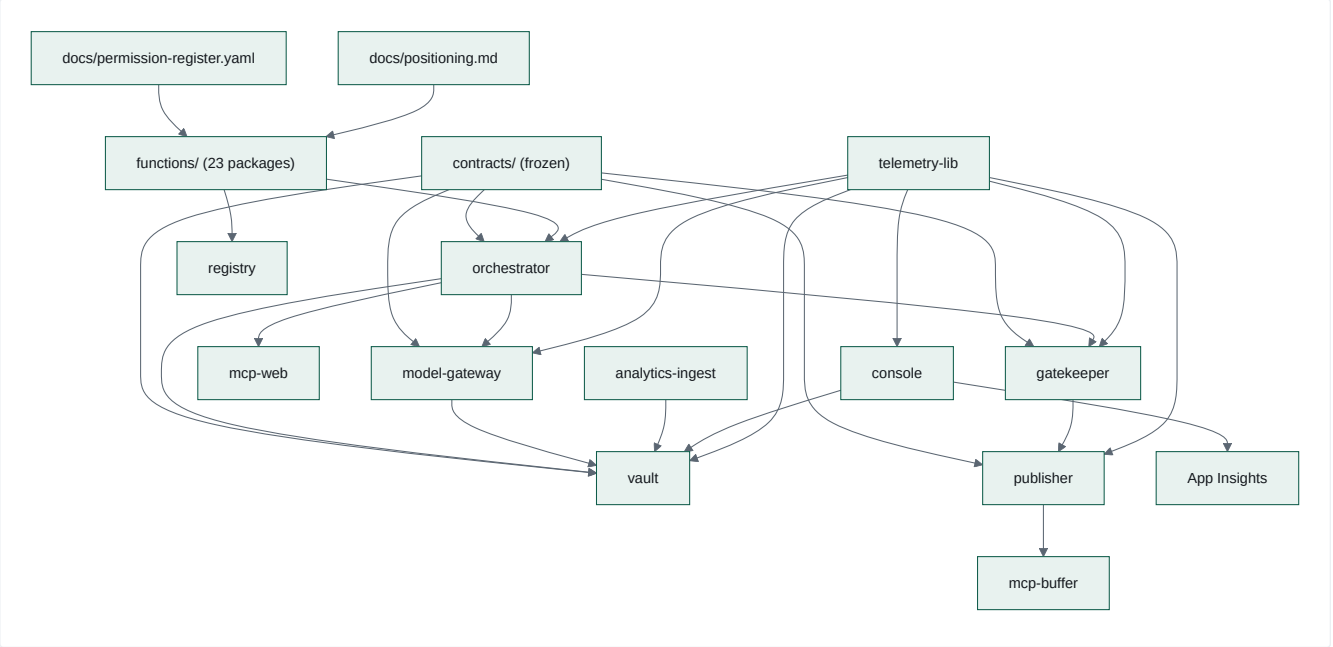
Maturity: L4 — this is the most mature module in the repository.

M13 — Compound Learning System (`.compound/`)

79 numbered learnings across four classes (`architecture` 26, `conventions` 38, `known-hard` 9, `security` 6), each with an id, a class, a one-line statement, and a status carrying strengthening/recurrence history.

This is not documentation of the product. It is **documentation of how to build this product**, accumulated across sessions. It has no code dependencies and no runtime role — and it is plausibly the most valuable artefact in the repository. See `07-operating-model.md`.

Module dependency map



Note the two **dashed** relationships that *should* exist but don't: `registry → orchestrator` (signatures never verified at runtime) and `console → gatekeeper` (no REST API to call).

03

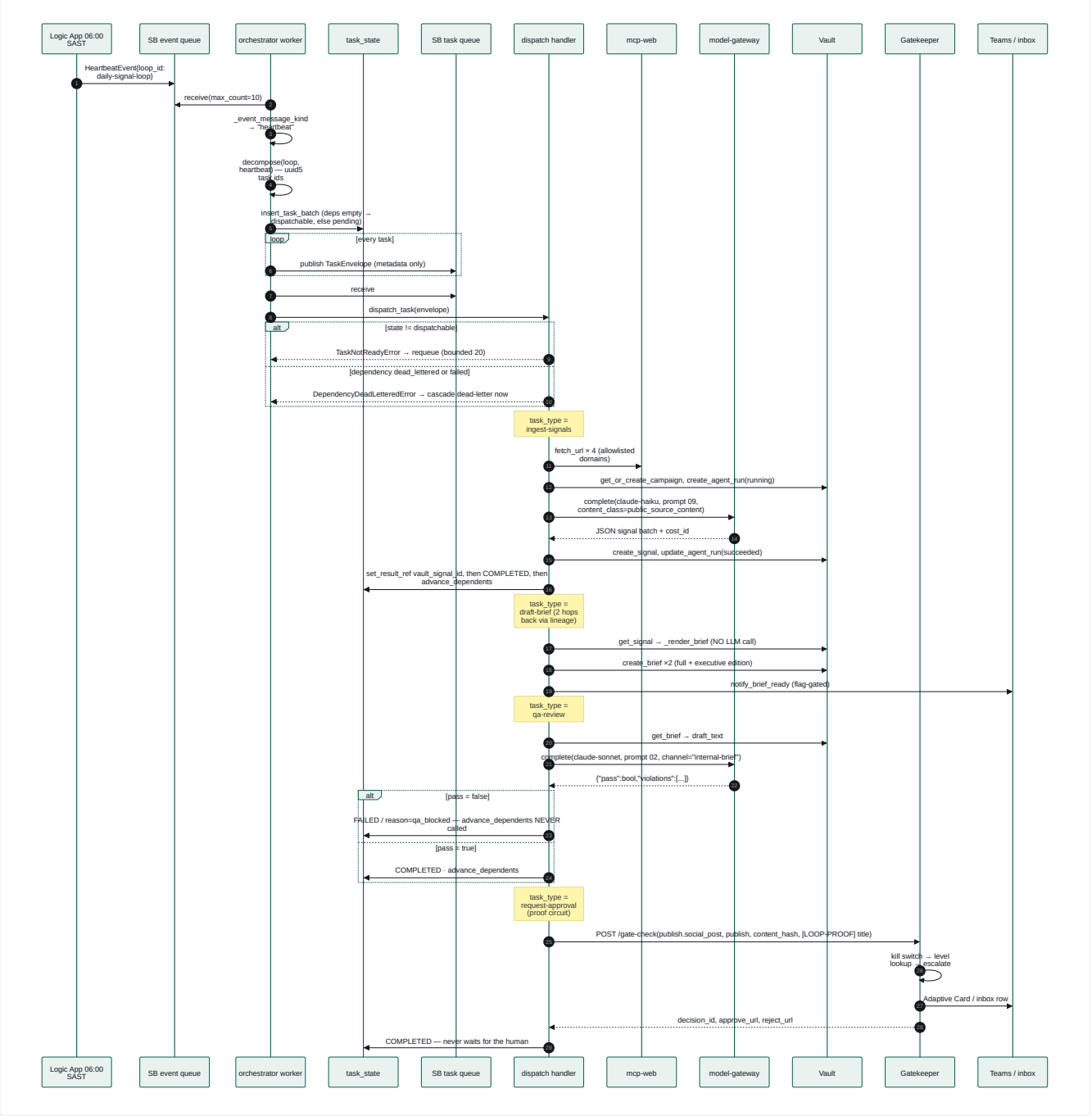
User Journeys

Every flow that exists in code. Flows that a typical SaaS would have but this platform does not are listed at the end — because their absence is itself a finding.

Journey inventory

#	JOURNEY	ACTOR	EXISTS?
J1	Daily signal loop (ingest → brief → QA → approval)	System	✅ L4 proven live
J2	Weekly content studio (Mon–Fri)	System	⚠️ L2 — DAG defined, 8 of 9 stages pass-through
J3	Nightly analytics ingestion	System	✅ L3 deployed
J4	Human approval of a publish	Approver	✅ L4
J5	Operator monitors a run	Operator	✅ L3 (partly mock data)
J6	Operator pulls the emergency stop	Operator	✅ L3
J7	Agent inspects run state programmatically	Agent	✅ L4
J8	Author/change a function definition	Engineer	✅ L2
J9	Deploy a change	Engineer	✅ L4
J10	Knowledge ingestion	System	✅ L4 (narrow)
J11	Retention expiry / erasure	System	✅ L3
J12	Bootstrap console authentication	Admin	✅ manual runbook
—	Registration / sign-up	—	❌ does not exist
—	Tenant onboarding	—	❌ does not exist
—	Project creation by a user	—	❌ does not exist
—	Conversational agent chat	—	❌ does not exist
—	Campaign creation by a user	—	❌ does not exist

J1 — The daily signal loop (the flagship journey)



Key journey properties visible in the code:

- The loop DAG has **24 tasks**: the original 5-stage chain, an 11-way intelligence fan-out off `ingest`, a dedupe join, response strategising, two brief-rollup nodes, and a 3-task S8 proof circuit.
- Of those 24, **exactly 4 task types do real work** (`ingest-signals`, `draft-brief`, `qa-review`, `draft-content`, `request-approval` — 5 handlers). The 11 fan-out scanners, the dedupe, the strategist and both rollups all hit `legacy_task_pass_through`: RUNNING → COMPLETED, no work.
- `request-approval` **completes as soon as** `/gate-check` **responds**. It never polls. The human decision arrives asynchronously and is discovered later via `GET /runs/{task_ref}` → Gatekeeper `/approval-status`.
- `draft-brief` performs **no LLM call** — it is deterministic rendering (`_render_brief`), and cites sources by **domain only**, never the bare URL, so that function 02's customer-facing link rules don't fire on an internal citation.

The S8 "Proof Circuit" — a genuinely interesting pattern

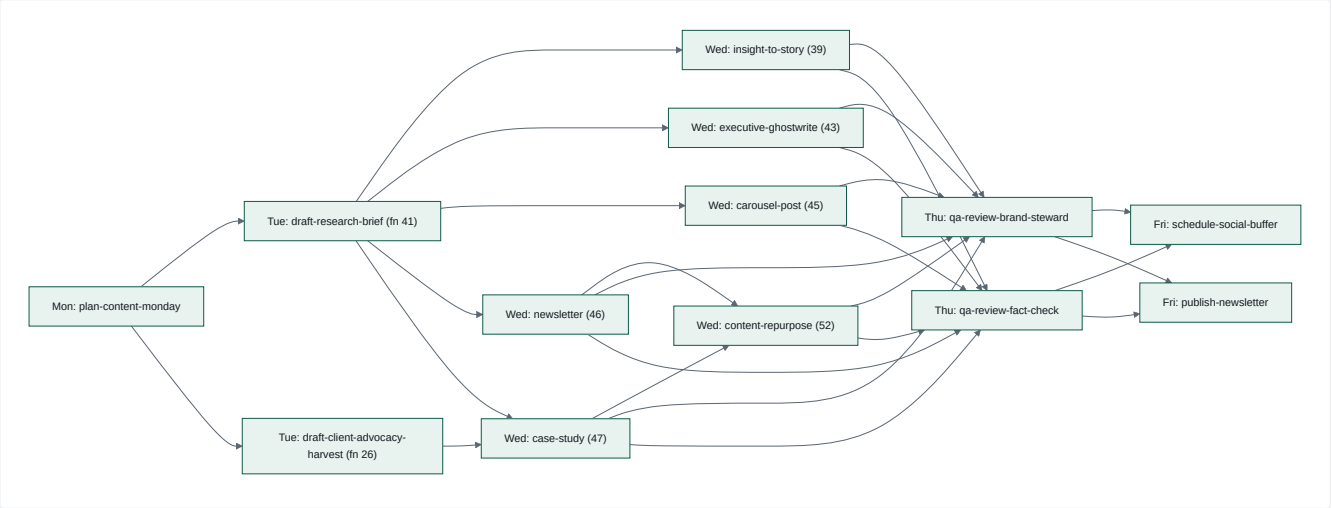
Three tasks in the daily loop carry `params.proof_circuit: true`, which `worker._task_metadata` promotes onto the envelope's `metadata` bag. Every handler that sees it tags its Vault `agent_run` with `agent_name = "loop-proof-circuit"` and stamps the approval card's `preview_title` / `preview_reference` with `[LOOP-PROOF]`.

The payoff: `services/publisher/app/vault_lookup.py` resolves an `asset_id` → `agent_run.agent_name`, and if it equals `AGENT_NAME_LOOP_PROOF`, publishing is **forced dry-run regardless of** `PUBLISHER_DRY_RUN`. The two services share no library, so the constant is duplicated — and a test in *each* service asserts they stay equal.

This is an end-to-end production smoke test that exercises the real path, against the real platform, with a structural guarantee that it can never actually publish. That is a genuinely strong safety pattern, and it has a name in the industry: a *canary transaction with a poison pill*.

J2 — The weekly content studio

`services/orchestrator/loops/weekly-content-loop.yaml` — a Mon–Fri DAG:



Dual-verdict gating is the notable design: Friday's two publishing tasks both `depends_on` *both* Thursday verdicts. Nothing ships unless Brand Steward QA **and** a fact-check verdict both pass.

Reality check: none of these 15 task_types is in `DISPATCH_TABLE`. Every one falls through to `legacy_task_pass_through`. The DAG, the dependency semantics, the day-prefixed task ids, the Buffer channel ids and the weekly cap (`buffer_weekly_post_cap: 8`, below Buffer Free's 10) are all real and validated — **but no content is produced.** The weekly loop is a fully-specified, correctly-gated, currently-empty pipeline.

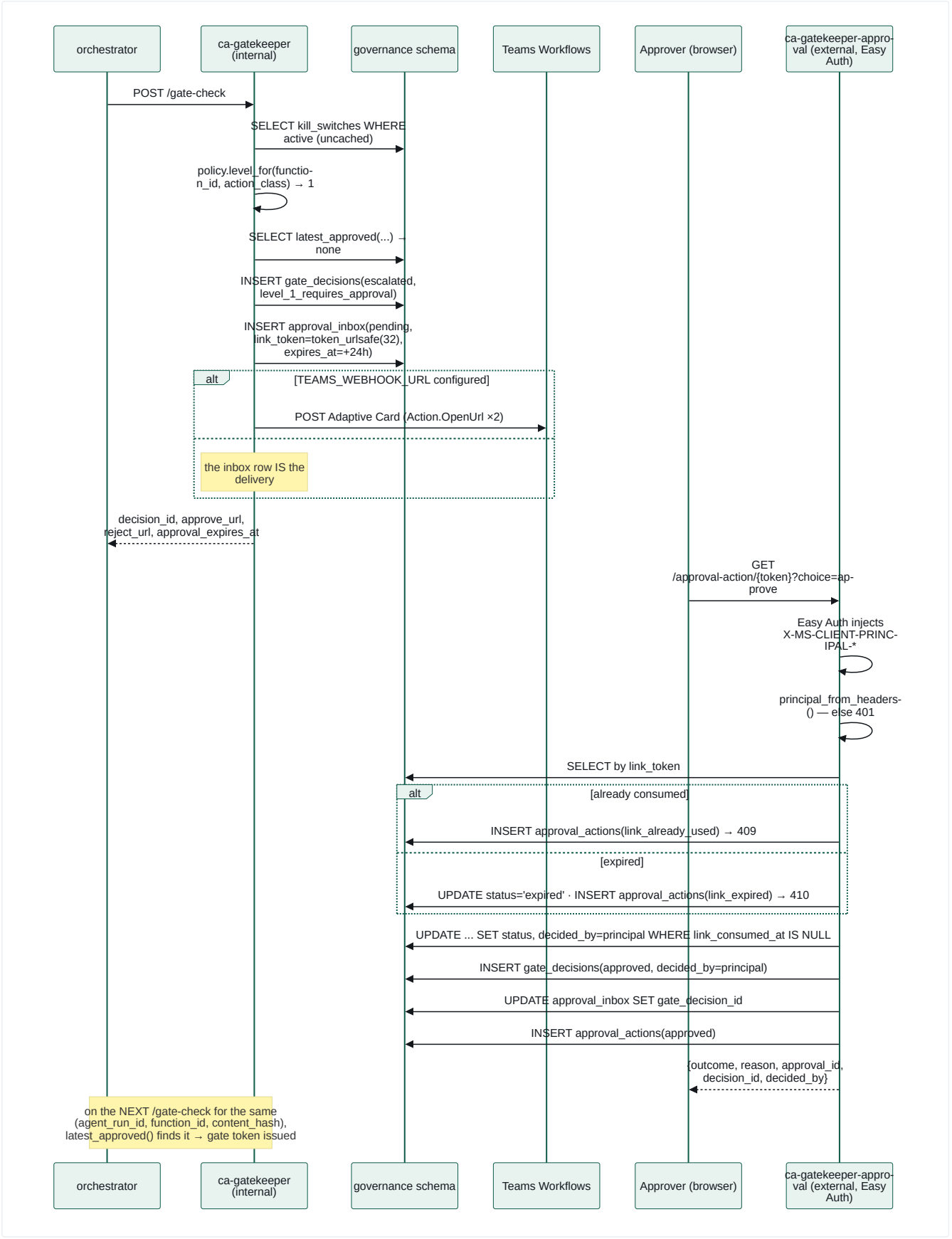
J3 — Nightly analytics ingestion

`caj-analytics-nightly-ingest`, cron `0 1 * * *` UTC = 03:00 SAST, running `python -m analytics_ingest.cli nightly --day <yesterday>`:

1. Ingest Buffer (live if `BUFFER_API_KEY`), GA4, Search Console, LinkedIn (fixtures)
2. For every row with a `utm_campaign`, `reconcile_utm()` against `utm_campaign_map`; unmatched/malformed → `utm_quarantine` with a reason
3. Four rollups: engagement-by-archetype, publishing reliability, **cost per accepted asset**, Vault utilisation
4. `export_fabric_day()` — assemble, validate against `analytics/contracts/fabric-nightly-export.schema.json`, upload to `analytics-fabric-export` blob
5. Microsoft Fabric picks it up via shortcut; Power BI reads the starter dataset

Note the division-by-zero discipline: every rollup **skips** rather than writes a zero — channels with `scheduled_count = 0`, archetypes with `SUM(impressions) = 0` (via `HAVING`), agents with 0 approved assets.

J4 — Human approval



Three properties an auditor would care about, all enforced in code:

1. **Possession ≠ identity.** The link carries only an opaque token. The recorded approver is the Easy-Auth principal *on this request*. There is no module-level state in `auth.py` — every helper takes headers as an argument, so an identity can never be cached between requests.

2. **Single use is atomic.** `UPDATE ... AND link_consumed_at IS NULL` — two concurrent clicks cannot both win.
3. **All four outcomes are audited**, including the two failure modes.

J5 — Operator monitors a run

Landing at `/` redirects to `/tasks`. From there: `/tasks/{task_ref}/trace` runs a KQL query against App Insights, but only after `task_ref` matches `^[A-Za-z0-9][A-Za-z0-9_-]{0,127}$` — an invalid ref is a **400**, not an empty render (RISK-001, injection defence). Any *other* App Insights failure degrades to the empty state with a `query_failed` flag rather than a 500.

The exact same KQL is published in `console/README.md` so an agent can bypass the console entirely and query App Insights directly.

J6 — Emergency stop

```
POST /kill-switch/toggle  {"active": true, "reason": "incident"}
```

Requires an authenticated principal (401 otherwise). The operator identity is read from Easy Auth headers and recorded. Form submissions additionally pass an Origin/Referer same-origin check and get a 303 redirect (POST-redirect-GET); JSON callers get the state back directly.

Propagation is sub-5s by construction, not by cache expiry: both `gatekeeper/app/kill_switch.py` and `publisher/app/kill_switch.py` issue a fresh SELECT on every single decision. The rationale is documented at length: *"a cache with any TTL above zero would make the 5s bound a function of cache expiry rather than of the operator's action."*

Current limitation: the console's Gatekeeper client is in `mock` mode, so this toggle currently writes to a fixture, not to `governance.kill_switches`.

J7 — Agent inspects run state

```
GET /runs/{task_ref}
→ { task_ref, stage_count, stages[{task_id, task_type, state, result_ref}],
   span_presence: present|absent|not_checked,
   approval_decision_status: {status, decided_by, decided_at} }
```

This endpoint exists because of a subtle truth: **the `request-approval` task's own state says nothing about whether a human approved.** It is COMPLETED the moment the gate-check responds. So `run_state.py` walks the lineage backwards, finds the `request-approval` stage, pulls `agent_run_id` / `function_id` / `content_hash` off its `result_ref`, and asks Gatekeeper `/approval-status` for the *real* decision. A Gatekeeper outage degrades to `{"status": "unknown"}` rather than failing the whole response.

J8 — Author or change a function definition

```
1. Copy contracts/function-definition/TEMPLATE/ → functions/NN-name/
2. Write prompt.md — MUST contain "Return a single JSON object"
3. Write skill.md (purpose / when to invoke / when NOT to invoke)
4. Write tools.yaml → validated against contracts/function-definition/tools.schema.json
5. Write schema.json with a /vN/ segment in $id
6. Write ≥5 golden eval tasks in evals/
7. python services/registry/validate_package.py --all
8. python services/registry/eval_harness.py --all
9. python services/registry/safety_suite.py --dir <package>
10. python services/registry/lint_rubrics.py --all
11. python services/registry/build_registry.py --out dist/ --sign
```

To make it *actually run*, three more things are required and are not automated: add the `task_type` to a loop YAML, add a handler to `DISPATCH_TABLE`, and add the package path to the orchestrator Dockerfile's staging step. That three-step manual gap is why 20 of 23 packages are inert.

J9 — Deploy

Push to `main` → `ci.yml` (7 jobs) → `*-image.yml` builds and pushes to the shared ACR via OIDC → `deploy-infra.yml` runs `az deployment group create` against `main.bicep` → **a human approves the `cmos-dev` GitHub Environment gate** → app-specific deploy workflows → `deploy-loop-e2e-smoke.yml` starts `caj-loop-e2e-smoke`, which injects a synthetic heartbeat and polls `/status` for the predicted task ids.

The smoke test's success criterion was itself amended (`F-SMOKE-GREEN-REDEFINITION`): **a legitimate `QA_BLOCKED` verdict counts as proof of life**, because reaching a real QA verdict means the whole chain worked. That is an unusually mature definition of "green".

J10 — Knowledge ingestion

The only ingestion path: `ingest_signals_handler` → mcp-web `fetch_url` over 4 allowlisted URLs → bodies truncated to 2,000 chars each → assembled into a prompt → sent to Claude Haiku → structured signals written to Vault.

The **redaction-fallback** deserves a mention as a journey in itself: real news prose routinely trips the `full-name-like` pattern (any two consecutive Title-Case words). Rather than failing, `_complete_ingest_with_redaction_fallback` drops one source at a time and retries until a request clears — degrading signal *completeness* instead of dead-lettering the task. And the firewall's ruling is always authoritative; the fallback never second-guesses it.

J11 — Retention expiry / erasure

Triggerable two ways, one implementation: `POST /retention-expiry-runs` or `caj-vault-retention-expiry` running `python -m vault.retention`. Sweeps expired rows, deletes the object, dedup-safely deletes the blob, writes one audit row per deletion, and **fails closed** if the blob delete errors — leaving the row eligible for retry with a `retention_deletion_failed` audit event rather than recording a deletion that didn't happen.

`legal_hold` never expires (year-9999 sentinel, so the NOT NULL constraint holds uniformly).

This is the closest thing to a POPIA erasure mechanism in the platform. There is no subject-initiated erasure request flow, no data-subject access request handler, and no export-my-data endpoint.

What does not exist — and why that matters

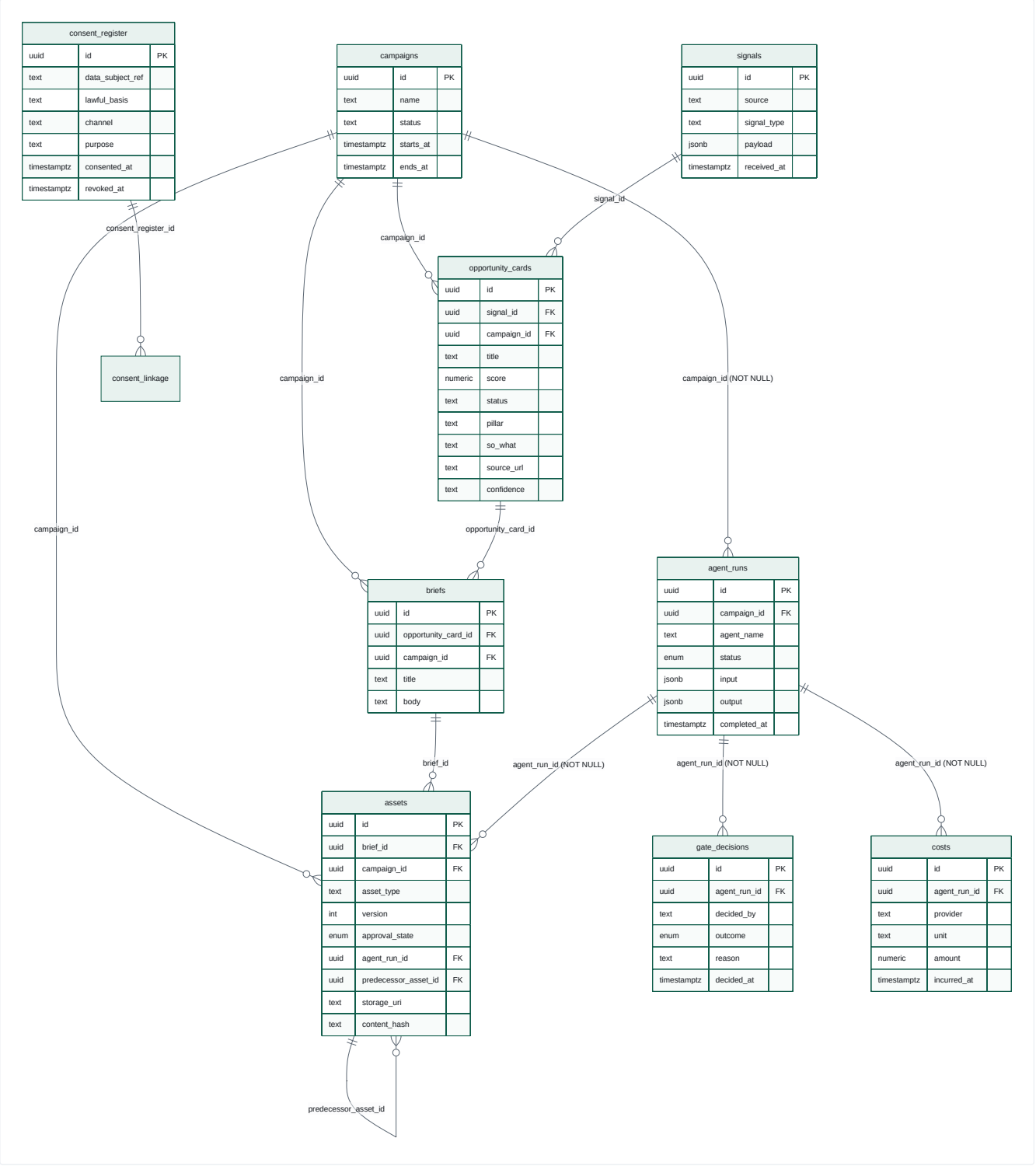
ABSENT FLOW	CONSEQUENCE
Registration / sign-up	Not a SaaS. Users are provisioned by an Entra admin (`docs/console-auth-runbook.md` , `scripts/bootstrap-console-auth.sh`)
Tenant / organisation model	**Single-tenant, single-company.** No `tenant_id` anywhere in any schema. Every multi-tenant assumption a buyer would make is false today
User-initiated project or campaign creation	Campaigns are created *by handlers* via `get_or_create_campaign(f"run-{campaign_id}")` . A human cannot create one
Conversational agent interaction	No chat, no threads, no session memory. Agents are batch functions in a DAG
Content editing / revision UI	Assets are versioned in the schema (`version` , `predecessor_asset_id`) but nothing writes a v2 or renders a diff
Role-based access control	Any authenticated tenant user reaches the kill switch. Documented accepted risk; mitigated only by an Entra "Assignment required" Portal setting
Notification preferences / digests	One webhook URL, globally
Onboarding / configuration wizard	All configuration is YAML in git, deployed by pipeline

04

Data Model

Every entity, from DDL. Five schemas in one Postgres 16 instance, owned by four different services.

1. Entity-relationship overview



2. The nine core Vault entities (`public` , frozen)

Source: `contracts/vault-schema/schema.sql` , hash-pinned in `contracts/.frozen-v1.sha256` .

2.1 `campaigns` — top of the roll-up chain

The aggregate root. Everything financial rolls up here. In practice campaigns are auto-created by handlers as `run-{envelope.campaign_id}` where `campaign_id = uuid5(heartbeat.event_id, f"campaign:{loop_id}")` — so **one campaign per loop per heartbeat**, not a marketing campaign in the business sense. **[INFERRED]** The entity is named for a future state it does not yet serve.

2.2 `signals` — raw inbound market signal

`payload jsonb` holds function 09's whole structured output. Immutable in practice (patchable but nothing patches it).

2.3 `opportunity_cards` — scored opportunities

Written by `score_signals_handler`, one row per signal in the day's batch, ranked highest-score-first. `score` is function 09's own `confidence` mapped to a number and multiplied by that pillar's weight, under the policy in `functions/_shared/scoring-policy.yaml`.

`pillar`, `so_what`, `source_url` and `confidence` were added after the v1 schema froze — additive, nullable, with the frozen baseline refreshed deliberately for that one file. Before them a card carried a headline and a number, which meant it could be listed but not *used*: a reader had no source to check the claim against, and the weekly planner had to re-derive every score from raw signal payloads because the field it selects on was not on the card. Cards written before that change carry NULL in all four and are skipped by readers rather than guessed at.

2.4 `briefs` — creative/marketing briefs

Written by `draft_brief_handler`, twice per run (a full brief and an "Executive Edition"). `opportunity_card_id` names the batch's **lead card** — the highest-scored signal, the one the brief leads with. A brief covers the whole batch, so no single id is the complete truth; the lead card is what a one-column FK can honestly record. A brief drafted with no scoring ancestor still leaves it NULL.

2.5 `agent_runs` — the central entity of the whole data model

Every model invocation is an `agent_run`. It is the join point for:

- cost attribution (`costs.agent_run_id`, NOT NULL)
- governance (`gate_decisions.agent_run_id`, NOT NULL)
- authorship/approval of artefacts (`assets.agent_run_id`, NOT NULL)
- budget enforcement (`budget.py` resolves `agent_run_id → agent_name`)
- the proof-circuit dry-run forcing (`agent_name == "loop-proof-circuit"`)

`status` is an enum: `pending | running | succeeded | failed | cancelled`. Handlers create it `running`, then update to `succeeded` / `failed` with the output payload.

If you want to understand this platform's data model in one sentence: `agent_runs` is the ledger of AI labour, and everything else hangs off it.

2.6 `assets` — versioned, approvable artefacts

Four things make this table unusually well-designed:

- `version integer` + `predecessor_asset_id` self-FK with a `CHECK (predecessor_asset_id IS DISTINCT FROM id)` — an asset chain is reconstructable
- `approval_state` enum: `draft | pending_review | approved | rejected | superseded`
- `content_hash` — the sha256 that binds into gate tokens
- `storage_uri` — points at the content-addressed blob

How to answer "which gate decision approved this asset version?" There is no direct FK. The schema documents the join convention inline:

```
SELECT gd.* FROM gate_decisions gd
WHERE gd.agent_run_id = <assets.agent_run_id>
ORDER BY gd.decided_at DESC LIMIT 1;
```

Because `gate_decisions` is append-only, the latest `decided_at` for that `agent_run_id` *is* the authoritative outcome. This is a deliberate design note in the DDL, not an accident.

2.7 `gate_decisions` — append-only governance ledger

Has no `updated_at` column, deliberately, "to discourage in-place mutation of a recorded decision." A re-decision is a new row.

Written by three distinct deciders, namespaced by convention:

DECIDED_BY	WRITTEN BY
`gatekeeper:policy`	Autonomy policy evaluation
`gatekeeper:kill-switch`	Kill switch block
`<name> (<oid>)`	A human via Easy Auth
`system:model-gateway:redaction-firewall`	PII block
`system:model-gateway:budget-gate`	Budget hard breach

This table is the platform's single most important audit artefact. It is the one place where "an AI wanted to do something and here is what was decided, by whom, and why" is recorded — for both human and machine deciders, in the same shape.

2.8 `costs` — three rows per completion

`unit` is free text, which is what lets one schema carry three signals without a migration: `usd`, `tokens`, `ms` (`services/model-gateway/metering.py`). The unbroken FK chain `costs → agent_runs → campaigns` is called out explicitly in the DDL as the per-campaign roll-up path.

2.9 `consent_register` — POPIA-shaped

Three design decisions are called out in the DDL comments and matter legally:

- `lawful_basis text NOT NULL` — **a real value, not a boolean.** POPIA s11 recognises consent as *one* of several lawful bases; a boolean would model only consent.
- `channel` + `purpose` columns — **not a single global opt-in flag.** Consent is per-channel, per-purpose.
- `revoked_at` distinct from `consented_at` — revocation is its own event, not a mutation of the grant.
- `UNIQUE (data_subject_ref, channel, purpose, consented_at)` — repeated grants over time are all preserved.

---

3. `vault_internal` — the governance sidecar (7 tables)

Exists because the public schema is frozen. Every table is keyed by `(object_table, object_id)` — a **polymorphic association**, uniformly applied to all 9 object types.

TABLE	PURPOSE	KEY CONSTRAINT
`object_taxonomy`	The 6 mandatory taxonomy fields for every object	`UNIQUE(object_table, object_id)`; `campaign_id` carries a real FK to `public.campaigns`
`consent_linkage`	Binds a client-derived object to the exact consent row that authorised it	`UNIQUE(object_table, object_id)`
`audit_log`	Every taxonomy rejection, consent rejection, retention deletion	indexed on event_type, object, correlation_id
`retention_policy`	`retention_class → expires_at`, with `deleted_at`	partial index `WHERE deleted_at IS NULL`
`retention_run`	Sweep bookkeeping	—
`access_log`	One row per object GET, keyed by `X-Caller-Service`	90-day self-purge
`utilisation_daily`	Daily rollup of access_log	`UNIQUE(day, object_table, caller_service)`

The taxonomy is the governance metamodel. Six fields, mandatory, immutable after create:

FIELD	DOMAIN	ENFORCED BY
`vertical`	free text	presence only
`function_id`	free text	presence only
`campaign`	uuid, FK to `campaigns`	uuid parse + FK
`evidence_grade`	`A | B | C | D | unverified`	closed set
`consent_status`	`granted | revoked | not_required | pending`	closed set
`retention_class`	`ephemeral_30d | standard_1y | extended_3y | legal_hold`	closed set

`evidence_grade` is the most interesting of the six: it means **every object in the system carries a claim about how well-evidenced it is.** That maps directly to `positioning.md` 's "proof over platitude" rule. The data model encodes the brand policy.

---

4. `governance` — the control plane (6 tables)

TABLE	OWNER	LIFECYCLE
`kill_switches`	Gatekeeper + Publisher (read)	mutable `active` flag; CHECK ensures `scope='global' ⇒ function_id IS NULL` and vice versa
`approval_inbox`	Gatekeeper	`pending → approved|rejected|expired` ; `link_consumed_at` is the single-use latch
`approval_actions`	Gatekeeper	append-only, 4 CHECK-constrained outcomes
`publish_attempts`	Publisher	append-only, `published|rejected` + reason
`jti_ledger`	Publisher	`jti` **is** the primary key — the PK is the enforcement mechanism
`schema_migrations`	migration job	—

Zero dollar signs in this migration file, deliberately — constrained domains are CHECK constraints over `text` , never `CREATE TYPE` , because `CREATE TYPE` needs a `DO $$ ... $$` block and Container Apps collapses `$$` to `$` in secret values.

---

5. Orchestrator tables (`public` , additive)

```
task_state(task_id PK, loop_id, task_type, state, retry_count,
           depends_on jsonb, vault_write_failed_count, result_ref jsonb,
           created_at, updated_at)

task_transitions(id PK, task_id FK, from_state, to_state,
                 reason CHECK IN (10 values), occurred_at)
```

Two defence-in-depth mechanisms in one place: `reason` is a closed `TransitionReason` enum in Python *and* a CHECK constraint in Postgres. The history of that constraint is instructive — migrations 0003 and 0004 both had to be rewritten to `ADD CONSTRAINT ... NOT VALID` because the migration script re-applies all four files on *every* deploy, so a later migration's wider vocabulary broke an earlier migration's re-validation.

The ten reasons: `created` , `dependency_satisfied` , `dispatched` , `completed` , `failed_attempt_1` , `failed_attempt_2` , `dead_lettered` , `dependency_dead_lettered` , `vault_write_failed` , `qa_blocked` .

Each of those last three encodes a distinct business meaning:

- `dead_lettered` — "we tried three times and gave up"
- `dependency_dead_lettered` — "we never tried; it was already impossible"
- `qa_blocked` — "a normal business verdict said no" (not a failure)

`result_ref` **is the platform's inter-task memory.** It is deliberately a *small structured pointer*, never content: `{vault_signal_id, brief_id, content_hash, agent_run_id, campaign_id, decision_id, ...}` . A downstream handler resolves what to do by reading its predecessor's `result_ref` off the lineage — never by re-deriving or guessing.

5b. Options inbox (`public` , additive v2 — the ratification model)

Three tables, not in the frozen file — additive v2 (`services/vault/migrations/0002_options_inbox_init.sql` , `0003`), each mirroring its own contract (`contracts/option-card.schema.json` , etc.) rather than a fourth copy of one.

TABLE	OWNER	LIFECYCLE
`option_cards`	option-card-emitting orchestrator handlers	a machine-generated menu of choices awaiting one human ratification; `expires_at` -bound
`approval_decisions`	Gatekeeper's `GET /decide`	**one row per card** (`UNIQUE(card_id)`) — single-use, not append-only, unlike `gate_decisions`
`standing_permissions`	a human ratifying a standing-permission card	durable "always allow this class" grant, with a `review_by` date

This is the newer, general-purpose sibling to `governance.approval_inbox` / `gate_decisions` — introduced for the Appendix D function set, not a replacement. The two mechanisms currently coexist.

6. `analytics` — the measurement schema (11 tables)

4 raw fact tables (`buffer_post_metrics` , `ga4_metrics` , `search_console_metrics` , `linkedin_metrics`), each `UNIQUE(source, day, natural_row_id)` ; `utm_campaign_map` + `utm_quarantine` ; `scheduled_posts` (the reliability denominator); and 4 `kpi_rollup_*` tables.

analytics-ingest never connects to the Vault's database directly — it reads Vault exclusively over REST (`C-VAULT-TABLES`). That is a real architectural boundary, not a convention.

6b. `mcp_ops` — tool-call logging (1 table)

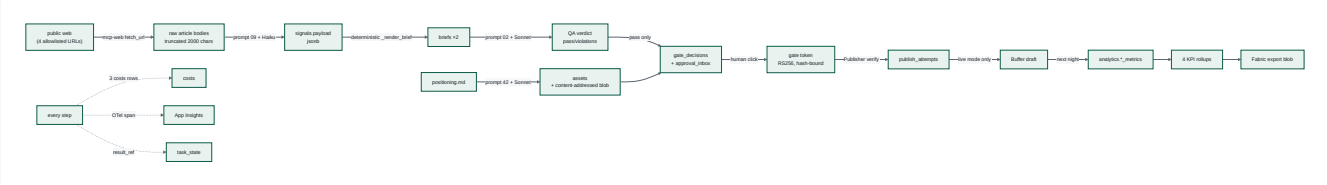
`mcp_ops.tool_calls` — one row per real call through any MCP server's shared logging wrapper: server, tool, caller identity, latency, outcome. Arguments are never stored, only a SHA-256 hash of their canonical-JSON form (POPIA-safe by construction). Owns no FK into `public` — tool-call logging is deliberately not campaign-scoped.

---

7. Ownership and lifecycle

ENTITY	CREATED BY	MUTATED BY	DELETED BY	TERMINAL STATES
`campaigns`	dispatch handlers (`get_or_create_campaign`)	Vault PATCH	retention sweep	—
`signals`	`ingest_signals_handler`	—	retention sweep	—
`opportunity_cards`	`score_signals_handler`	Vault PATCH	retention sweep	—
`briefs`	`draft_brief_handler`	—	retention sweep	—
`agent_runs`	every handler	handler on completion	retention sweep	succeeded / failed / cancelled
`assets`	`draft_content_handler`	Vault PATCH (approval_state)	retention sweep (+ blob)	approved / rejected / superseded
`gate_decisions`	Gatekeeper ×3 paths, model-gateway ×2 paths	**never**	never	append-only
`costs`	model-gateway metering	**never**	never	append-only
`consent_register`	manual / API	PATCH `revoked_at` only	never	revoked
`task_state`	`insert_task_batch`	`transition` , `set_result_ref`	never	completed / dead_lettered / failed
`approval_inbox`	`/gate-check` level 1–2	`consume_link` once	never	approved / rejected / expired
`jti_ledger`	Publisher	**never** (PK insert only)	never	consumed

8. How information flows through the system



Three information-flow invariants, each enforced structurally:

1. **Content is never on the wire between components.** Queues carry ids (`contracts/service-bus/spec.md`). `result_ref` carries ids (`0002_task_result_ref.sql` header). Spans carry ≤200-char enum-keyed values (`telemetry_lib/attributes.py`). Content lives in the Vault and is fetched by id.
2. **PII cannot reach a provider un-scanned.** The redaction firewall is step 3 of the gateway's own pipeline, before any adapter call, and it scans the `tools[]` passthrough too — because *"client-identifying data smuggled into a tool definition would bypass a messages-only scanner entirely."*
3. **Every rejection produces a durable audit row on an isolated connection.** `write_audit_isolated()` exists precisely because the rejection path raises and rolls back its own transaction — a shared connection would roll back the audit row along with it, "silently losing the very audit trail the rejection is supposed to produce."

9. Data-model gaps worth naming

GAP	IMPACT
No `tenant_id` / `organisation_id` anywhere	Single-tenant by construction. Multi-tenancy is a schema-wide change, not a feature
No FK `assets ↔ gate_decisions`	Resolvable only by the documented join convention; a direct FK would be safer
No `users` / `roles` / `permissions` tables	Identity is entirely delegated to Entra; there is no in-app authorisation model. `standing_permissions` (§5b) grants action-classes, not users — it doesn't close this gap
`publish_attempts.agent_run_id` has no FK	It is in a different schema from `agent_runs`; referential integrity is by convention
Publisher's Vault "publish record" is a stub	The publication event is recorded in `publish_attempts` but never in the Vault
No campaign→brief→asset lineage query API	Reconstructable by hand; not exposed

10. Table catalogue — operator function, user journey, writers, readers

Every table, why it exists in plain terms, and the read/write edges that justify it. Short by design — §§2–6b above give the full detail for anything that needs it. "Written by" is the only writer(s); "Read by" excludes the writer reading its own row back.

`public` — frozen core (9)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`campaigns`	Groups everything under one run, so cost rolls up per campaign	Vault API (`get_or_create_campaign` , called by every handler)	cost/asset roll-ups, console
`signals`	The record of what the market scan noticed	`ingest-signals` + the 11 scanner handlers	`score-signals` , `draft-brief` 's lineage walk
`opportunity_cards`	Scored, ranked signals — "what's worth acting on today"	`score-signals` , `dedupe-signal-cards`	Monday planning (evidence-led pillar pick), morning-brief rollup, console
`briefs`	The narrative artefact an operator actually reads	`draft-brief` , `draft-research-brief` , both brief-rollup handlers	QA handlers, drafting handlers, console, Teams brief card
`agent_runs`	One row per agent/tool call — the audit spine everything else hangs off	every handler (`create_agent_run` / `update_agent_run`)	budget check, cost join, gate-token issuance (approving identity), console trace
`assets`	Versioned content — drafts through to published copy	every drafting handler; QA retry loop (new version per regen)	QA handlers, Publisher (hash cross-check), console
`gate_decisions`	Append-only ledger of every allow/deny ruling	**two writers**: Gatekeeper (policy/approval) and Model Gateway (redaction/budget)	Publisher (approval lookup), console, audit
`costs`	Three rows per model call — what this cost and how long it took	Model Gateway `metering.py`	budget check, analytics cost-per-asset KPI, console cost ledger
`consent_register`	POPIA lawful-basis record per subject/channel/purpose	Vault API, from an external consent-collection process	Vault's own consent gate, on every create carrying `data_subject_ref`

`public` — orchestrator, additive (2)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`task_state`	The task graph's live state, and the `result_ref` bus one task hands the next	orchestrator only	orchestrator, console task list
`task_transitions`	Append-only "what happened to this task, in order"	orchestrator `state_machine.py`	console trace/detail, an operator debugging a stuck loop

`public` — options inbox / ratification model, additive v2 (3)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`option_cards`	A menu of choices put to a human for one ratification decision	option-card-emitting handlers (Appendix D functions)	Gatekeeper `/decide`, Teams render, console options inbox
`approval_decisions`	The one decision made on one card — single-use, never re-decided	Gatekeeper `GET /decide` (Teams link or console form)	whatever executes the chosen option, audit
`standing_permissions`	A durable "always allow this" grant, so the same class of action isn't asked twice	a human ratifying a standing-permission card	option-card-emitting handlers, checking before they ask

`governance` (6)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`kill_switches`	The emergency stop an operator can throw	Console kill-switch toggle	Gatekeeper + Publisher, uncached, on every decision
`approval_inbox`	Single-use, TTL-bound human approval link (the original gate-check flow)	Gatekeeper, on a level-1/2 escalation	the external approval-action app
`approval_actions`	Append-only record of every click — approved, rejected, expired, already-used	the approval-action app	audit, console
`publish_attempts`	Append-only, one row per Publisher call, including every refusal	Publisher	console, audit
`jti_ledger`	Stops a gate token being replayed — the PK enforces it	Publisher	Publisher only
`schema_migrations`	Migration bookkeeping	the governance migration job	—

`vault_internal` — the sidecar the frozen schema can't carry (7)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`object_taxonomy`	Vertical, evidence grade, consent status, retention class — fields the frozen file can't hold	Vault API, on every object create	Vault's own `fetch_object` join, the retention sweep
`consent_linkage`	Durably ties a client-derived object to the exact consent that permitted it	Vault's consent gate	audit, a POPIA data-subject request
`audit_log`	Rejection and deletion audit trail	Vault (consent rejection, retention deletion)	audit
`retention_policy`	This object's expiry date, by retention class	Vault API, on create	the retention sweep job
`retention_run`	One row per retention sweep — how many deleted, when	`vault/retention.py`	console, ops
`access_log`	One row per GET on an object — who read what, when	Vault's request middleware	the utilisation rollup
`utilisation_daily`	Daily rollup of `access_log` by object class	Vault `rollup.py`	`GET /utilisation/rollup`, the vault-utilisation KPI

`analytics` — the measurement schema (11)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`buffer_post_metrics`, `ga4_metrics`, `search_console_metrics`, `linkedin_metrics`	Raw per-day performance facts, one table per channel	analytics-ingest's nightly per-source ingest	the 4 KPI rollups
`utm_campaign_map`	Registers a `utm_campaign` against a real Vault campaign/asset, so a post can be attributed	`publish-newsletter` (registers on send)	`reconcile_utm`
`utm_quarantine`	Rows whose `utm_campaign` didn't match the map — never silently dropped	`reconcile_utm`	ops investigating an attribution gap
`scheduled_posts`	How many posts were scheduled per channel/day — the reliability KPI's denominator	orchestrator `record_scheduled_post()`	`rollup_publishing_reliability`
`kpi_rollup_engagement_by_archetype`, `_publishing_reliability`, `_cost_per_accepted_asset`, `_vault_utilisation`	The 4 nightly headline numbers an operator actually reports	analytics-ingest `rollup.py` (upsert)	Fabric export, Power BI, the month-end report

`mcp_ops` (1)

TABLE	EXISTS FOR	WRITTEN BY	READ BY
`tool_calls`	One row per real MCP tool call — never arguments, only their hash	every MCP server's shared logging wrapper	ops (rate/error investigation); no console reader exists yet

Reading the writer/reader columns as a journey, end to end: a signal (`signals`) becomes a scored opportunity (`opportunity_cards`), becomes a brief (`briefs`), becomes drafted content (`assets`) tied to whoever wrote it (`agent_runs`) and what it cost (`costs`); it clears review, and a human either clicks an `approval_inbox` link or ratifies an `option_cards` card (`approval_decisions` or `gate_decisions` records which); Publisher checks `jti_ledger` and `kill_switches` and appends to `publish_attempts` ; and only once it is live does `analytics.*` pick it up, attributed through `utm_campaign_map` , and feed the KPI a human reads next month. Every arrow in that sentence is a table above.

05

AI Architecture

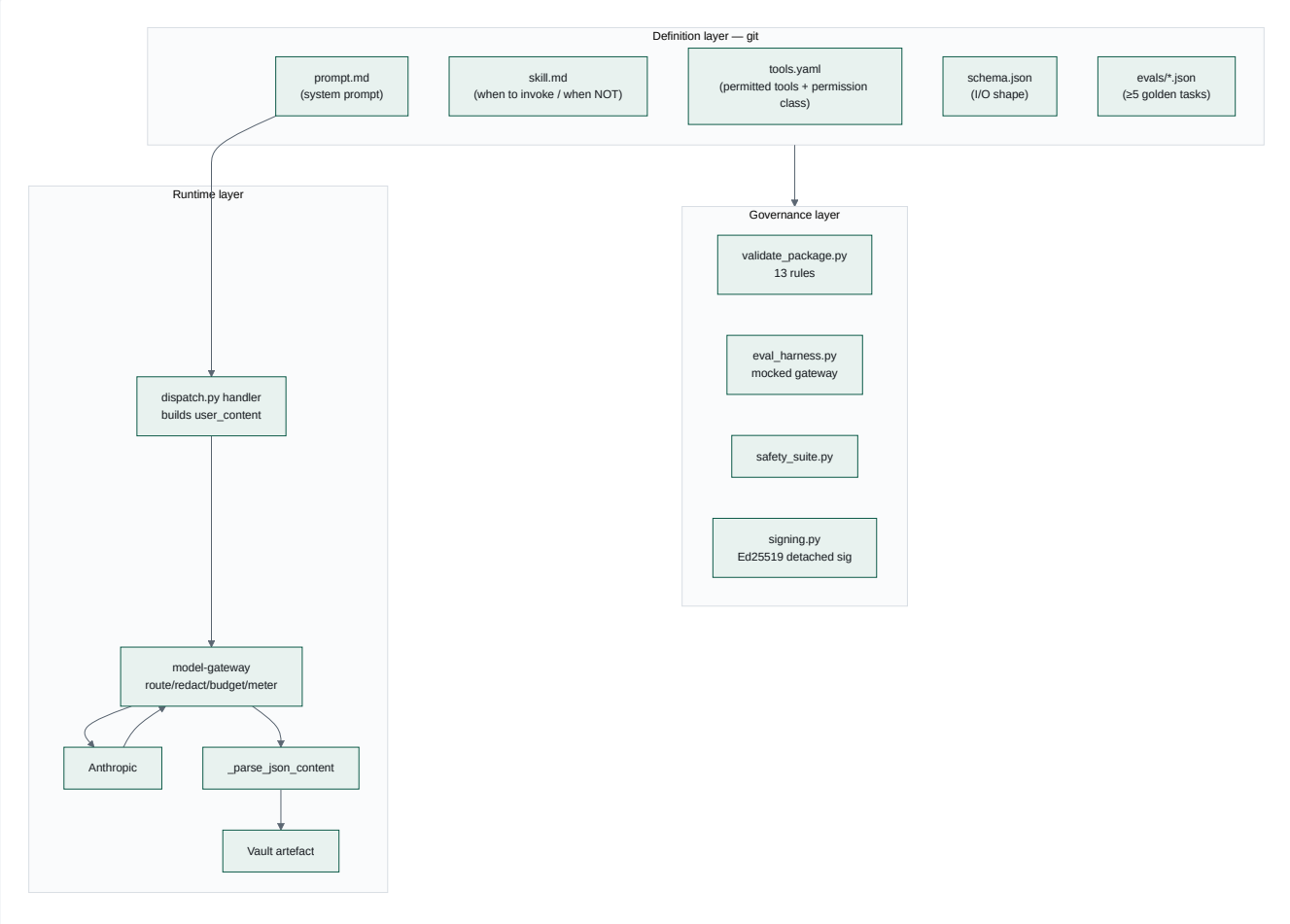
Every AI capability in the platform, and — just as importantly — every AI capability that is conventionally expected and is deliberately absent.

1. The shape of the AI system

This is **not** an agentic framework in the LangChain/AutoGPT sense. There is no planner, no ReAct loop, no tool-calling loop, no self-reflection, no multi-turn conversation, and no vector store. What exists instead:

> **A deterministic DAG of single-shot, schema-constrained, prompt-versioned LLM invocations, each routed through one governed chokepoint, each producing a validated JSON artefact that becomes the next node's input by reference.**

That is a deliberate and defensible architecture. It trades agent autonomy for *auditability, cost predictability and reproducibility*. Every decision in the codebase is consistent with that trade.



2. Agents — what actually runs

Strictly speaking there are **five runtime agent behaviours**, invoked from `DISPATCH_TABLE` (`services/orchestrator/orchestrator/dispatch.py`):

TASK_TYPE	FUNCTION PACKAGE	MODEL TIER	LLM CALL?	PRODUCES
`ingest-signals`	09 Market Intelligence Director	`claude-haiku`	✅	`signals` row + `agent_run`
`draft-brief`	`brief.compose` (no package)	—	❌ **deterministic**	2 × `briefs`
`qa-review`	02 Brand Steward QA	`claude-sonnet`	✅	verdict → COMPLETED or FAILED/`qa_blocked`
`draft-content`	42 LinkedIn Post Writer	`claude-sonnet`	✅	`assets` + blob
`request-approval`	— (`publish.social_post`)	—	❌	gate decision + approval card

Everything else — 20 function packages, ~30 loop task_types — resolves to `legacy_task_pass_through`, which transitions RUNNING → COMPLETED and does nothing. **The gap between the designed agent estate (23) and the running agent estate (3) is the platform's defining execution gap.**

Model tier assignment is meaningful, not arbitrary: extraction/triage work (ingest) runs on Haiku; judgement work (QA) and generative work (drafting) run on Sonnet; Opus is configured but currently unused by any handler.

3. Prompts

Prompts are **files in git**, one per function, with a rigid house style visible across all 23:

1. **Role statement** — "You are the Brand Steward. You do not write marketing copy — you judge it."
2. **Output contract** — a literal JSON block, and the exact phrase *"Return a single JSON object and nothing else"*. This phrase is now **mechanically enforced** by `validate_package.py`'s `prompt-missing-json-output-contract` rule.
3. **Numbered hard rules** — each one corresponding to a violation code.
4. **Method / structure** — how to reason, not just what to output.

The prompt-as-eval-oracle pattern

This is the most sophisticated idea in the AI layer, and it is easy to miss. From `services/registry/README.md`:

> *"The mocked gateway is driven by each package's own `prompt.md`. A package's `tool_check.py` derives its simulated output from the rules stated in its prompt. Delete a rule from a prompt and the tasks grading that rule fail."*

And there is a permanent regression fixture proving it: `services/registry/fixtures/regression/42-linkedin-post-writer-broken/` is a copy of function 42 with the roof-line rule removed, and it **must fail by task id**. Without that coupling, *"'evals passed' would only ever mean 'the code still runs'."*

Enterprise name for this: prompt regression testing with a behaviour-coupled oracle. Very few teams do this.

Prompt injection surface

The prompts are *static, developer-authored, checked into git*. Dynamic content only ever enters via the `user` role, as a JSON blob the handler constructs. That property is load-bearing — it is the entire justification for exempting `system`-role content from the redaction firewall (see §7).

The one real injection vector is `ingest-signals`: fetched news article bodies (2,000 chars each) go into the `user` message. **There is no prompt- injection defence on that path** beyond the redaction firewall (which looks for PII, not instructions). A malicious payload on `moneyweb.co.za` could attempt to steer function 09's output. Mitigations that *do* exist: the output is schema-constrained, the domain allowlist is tiny, and the downstream QA gate is a separate model call.

4. Tools

Tool access is declared per function in `tools.yaml`, validated against `contracts/function-definition/tools.schema.json`, with a coarse permission class the orchestrator is meant to enforce:

```
permissions: read-only | read-write | none
scope: <optional narrowing note>
```

Honest finding: the `permissions` field is declarative documentation, not an enforced runtime capability grant. `dispatch.py` does not read `tools.yaml` at all. The real enforcement is architectural — handlers only call the clients they were written to call, and mcp-web/mcp-buffer/mcp-canva enforce their own guardrails server-side (allowlist, draft-only, template-locked). That is *defence by construction*, which is arguably stronger, but it is not what the manifest implies.

The three real tool servers:

SERVER	TOOLS	SERVER-SIDE GUARDRAIL
mcp-web	`fetch_url`	host allowlist checked before any network call; sliding-window rate limit
mcp-buffer	`list_queue` , `get_post` , `create_draft`	`_CREATE_DRAFT_STATUS = "draft"` hardcoded; no status argument accepted; a test greps every tool name and description against `publish|share.?now|send.?now|go.?live`
mcp-canva	`create_design_from_template` , `bulk_create_from_csv` , `export_design`	`template_id` required on both creation tools — no free-form design generation exists

The guardrail philosophy is "make the dangerous thing unrepresentable." mcp-buffer cannot publish because no publish tool exists in the manifest — not because a flag is set to false.

5. Memory

LAYER	STORE	SCOPE	HOW IT IS READ
Inter-task	`task_state.result_ref` (jsonb)	one run	`resolve_lineage_result()` — BFS up `depends_on` , ≤6 hops, returns the first ancestor with a non-null `result_ref`
Artefact	Vault `public` + blobs	retention-bounded	by id, over REST
Strategic	`docs/positioning.md`	permanent	quoted **verbatim into prompt.md** at authoring time
Permission	`docs/permission-register.yaml`	permanent	read at runtime by `permission_check.py`
Engineering	`.compound/learnings/`	permanent	by humans/agents at design time

There is no vector database, no embedding model, no similarity search and no retrieval-augmented generation anywhere in this repository. Confirmed by absence across all 344 Python files.

`resolve_lineage_result` deserves attention because it is doing the job a memory system would do in a conventional agent framework:

> *"draft-brief's immediate predecessor is 'score', but the real content it needs lives 2 hops back at 'ingest'. qa-review's own two loop positions each have a result_ref-bearing IMMEDIATE predecessor, so this same walk resolves both in one hop for qa-review and two for draft-brief — one mechanism, no per-loop-position special casing."*

That is **content-addressed working memory over a DAG**, and it works without any semantic layer because the DAG is deterministic.

6. Knowledge

Knowledge enters through one door and one door only: mcp-web's `fetch_url` over four URLs across three domains (`functions/09-market-intelligence-director/fetch_sources.yaml`), read at runtime and mirrored into `MCP_WEB_ALLOWLIST` in `infra/main.bicep` , which currently matches exactly:

```
urls:
  - https://learn.microsoft.com/en-us/fabric/get-started/whats-new
  - https://www.moneyweb.co.za/feed/
  - https://www.moneyweb.co.za/news/tech/feed/
  - https://businesstech.co.za/news/feed/
```

The entire knowledge intake rests on one hand-set environment variable. `MCP_WEB_LIVE_MODE` is not declared anywhere in `infra/` ; without it, `fetch_url` ignores the URL and returns a synthetic placeholder for every source — and the loop still goes green, writing hallucinated signals to the Vault with real-looking `source_url` values. See `09-technical-debt.md` TD-31. This is the single most fragile point in the platform, because it fails silently and produces plausible output.

`web_search` is declared in function 09's `tools.yaml` but **not implemented** in mcp-web. The `fetch_sources.yaml` header documents the activation path: add search results to the same `urls` -shaped evidence set, no code change.

Knowledge quality is governed by three mechanisms:

- `evidence_grade` on every Vault object (`A|B|C|D|unverified`)
- function 09's hard rules: ≥3 signals, ≥2 distinct domains, every signal carries an `https://` `source_url` , `confidence: low` for thin evidence
- the "proof over platitude" rule enforced by function 02's `unsupported-claim` violation code

7. Decision making — the redaction firewall as a case study

This is where the platform's AI-safety thinking is most visible, and it is worth reading as a narrative, because the code preserves it.

Original design: scan every message role plus the `tools[]` payload against four patterns from a frozen contract (name-shape, email, SA phone, SA ID) plus exact-match fixtures.

Incident 1 (2026-08-03, deploy-loop-e2e-smoke #19). The `full-name-like` pattern is *any two consecutive Title-Case words*. Functions 02/09/42's own static `prompt.md` files trip it constantly — "Market Intelligence", "Brand Steward", "South African", "Microsoft Fabric". Since the scanner covered system role, **every LLM call the platform ever made was structurally guaranteed to be blocked.** Fix: narrow to non-system roles, justified because `system_prompt` is always `_read_prompt()`'s output — a static file read — and because system role is the universal API convention for developer instructions, never end-user content.

Incident 2 (round 15). Real news bodies trip the same pattern. The per-source drop-and-retry fallback always exhausted to zero. Fix: an explicit `content_class` request field mapping to a **reviewed, gateway-side allowlist** of pattern ids to exempt:

```
CONTENT_CLASS_PATTERN_EXEMPTIONS = {"public_source_content": frozenset({"full-name-like"})}
```

Note the design: the caller names a *content class*, not a pattern. It "can only ever widen this file's own reviewed mapping, never let a caller silently choose what to bypass."

Incident 3 (rounds 17–19). Two further call sites authorised, each as its own named ruling with its own reasoning, recorded in the module docstring — QA review of a rendered brief (same public news text, one hop later), and QA review of a drafted LinkedIn post (static positioning.md content). Round 18 deliberately left the second un-exempted pending real evidence; round 19 found the assumption didn't hold and fixed it.

What this demonstrates: a security control that was *too strict to function*, narrowed four times, with each narrowing scoped to one pattern and one named content class, each recorded with its justification and its authoriser, and never widened silently. Every block still writes a `gate_decisions` row precisely because *"pattern coverage is known to be incomplete — which is exactly why every block is written to gate_decisions as an audit row."*

Two subtle security properties in the same module:

- Matched-pattern ids are **opaque contract-side coordinates** (`fixture:client_names:0`), never the matched text. An earlier design used `f"fixture:{value}"` — which echoed the client's real name back to the caller in the 400 body *and wrote it permanently into the audit column*, "defeating the firewall through its own audit trail."
- Non-string content is `json.dumps`'d and scanned, never skipped: *"A scanner that silently continues past a shape it did not expect is a bypass, not a scanner."*

8. Orchestration and agent communication

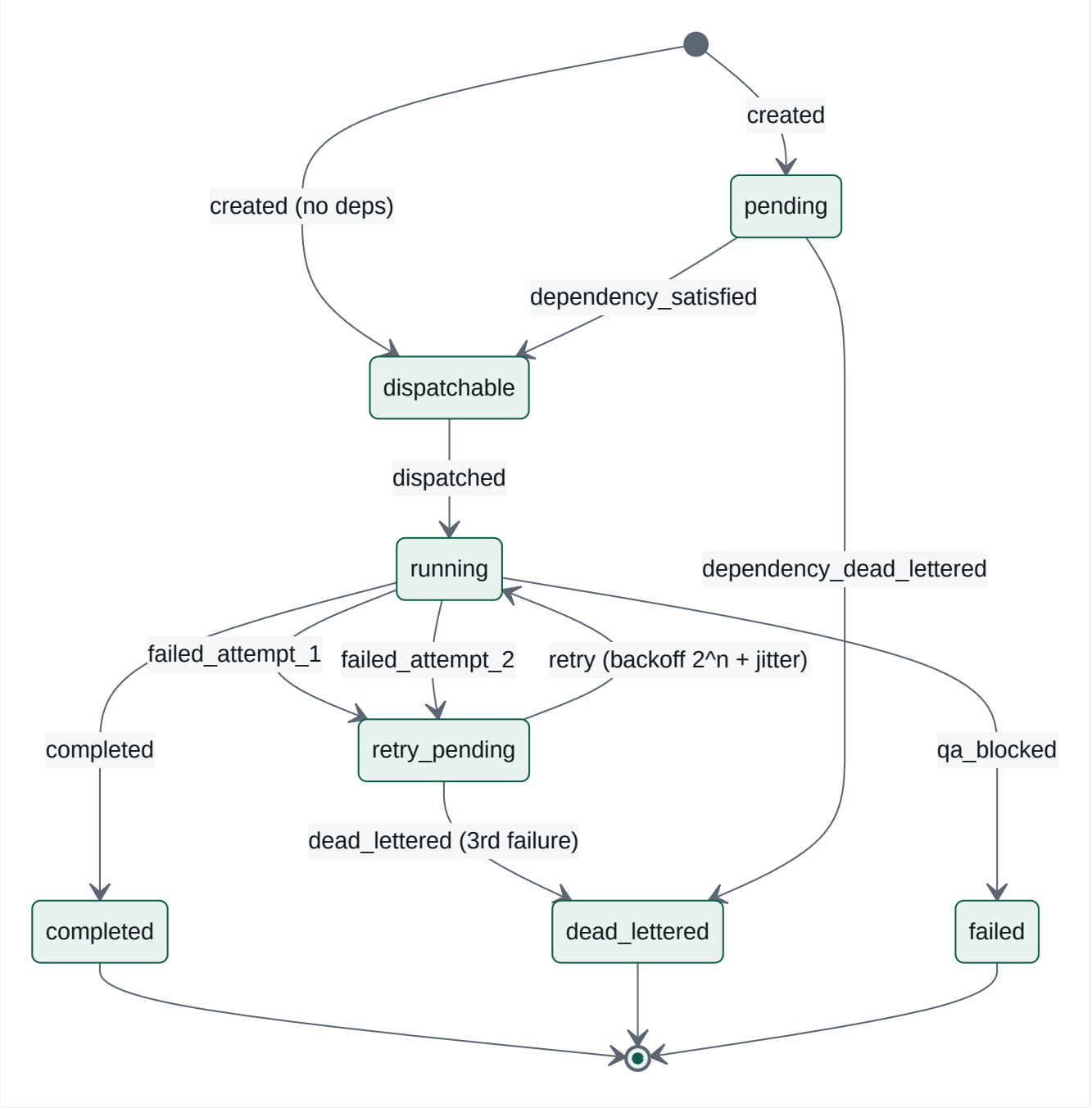
Agents do not talk to each other. There is no message passing, no shared scratchpad, no negotiation, no debate. Communication is exclusively:

```
handler A → set_result_ref(task_id, {small structured pointer})
        → db.advance_dependents(task_id)
handler B → resolve_lineage_result(task_id) → reads A's pointer
        → fetches the real content from the Vault by id
```

The orchestration topology is a **static, declarative, acyclic DAG in YAML**, validated by JSON Schema plus Kahn's algorithm at load time. Fan-out (11 parallel scanners), fan-in (dedupe joining all 11), and dual-gate joins (Friday depends on both Thursday verdicts) are all expressible.

Enterprise name: this is *choreography*, not orchestration-by-agent — and the deterministic `uuid5` decomposition makes it a *reproducible* workflow, which is rare in agent systems.

9. Failures, retries and cascade



Five distinct failure semantics, each with its own handling — this is unusually granular:

CONDITION	EXCEPTION	HANDLING
Task's turn hasn't come	`TaskNotReadyError`	Requeue the same envelope, bounded at 20 (tuned against **observed** ~14s production requeue cadence, not theory)
A dependency is permanently blocked	`DependencyDeadLetteredError`	Cascade dead-letter **immediately** — no requeue, no backoff, no 3-strike
Handler genuinely failed	any `Exception`	`_retry_or_dead_letter` : 3-strike state machine, retrying the *same handler directly*
Business verdict says no	—	`FAILED` / `qa_blocked` , `advance_dependents` **never called**
Process crashed mid-message	`delivery_count > 1`	`reconcile_redelivered_task` → `record_failure`

Two design decisions here are genuinely good and worth preserving:

1. `qa_blocked` **is not a failure.** It gets its own transition reason, its own DB CHECK value, and the smoke test explicitly counts it as proof-of-life. The system distinguishes *"the AI broke"* from *"the AI correctly said no."* Most platforms conflate these.

2. **Cascade fail-fast.** Discovered live in round 17: a `QA_BLOCKED` draft's dependent sat requeuing for ~15 minutes because only `DEAD_LETTERED` was recognised as permanent. `_PERMANENTLY_BLOCKED_STATES` now includes `FAILED`. The one-hop check is deliberately shallow, with a documented proof that wave-by-wave propagation makes a recursive walk unnecessary.

Known weakness, admitted in the code: `_retry_or_dead_letter`'s docstring states *"a handler that partially wrote to Vault before failing is not guaranteed idempotent on retry (e.g. a duplicate signal/agent_run row is possible)."* Flagged, accepted, not fixed.

10. Context building

Context is assembled **deterministically**, never retrieved:

```
system_prompt = _read_prompt("02-brand-steward-qa")     # static file
user_content  = json.dumps({"draft_text": ..., "client_references": [],
                            "channel": "internal-brief"})
```

Two things about that snippet are load-bearing:

- `channel` is not cosmetic. It selects which of function 02's rules apply — `"internal-brief"` exempts the draft from `missing-cta` and `url-utm` checks, per a documented ruling. **The prompt reads a runtime discriminator and changes its own rule set.** That is policy parameterisation inside a prompt.
- `client_references` is *always an empty list* in the current code, meaning the deterministic uncleared-client check (`permission_check.find_uncleared_references`) currently always passes trivially. The mechanism is real and tested; the data feeding it is not yet wired. **This is a live gap, not a design choice.**

11. Human approvals

Covered fully in `03-user-journeys.md` §J4. The AI-relevant point: the approval is bound to `content_hash`, not to an id. Approving *this asset* means approving *these exact bytes*. If one byte changes, the token's bound hash no longer matches the recomputed hash and publishing is refused with `content_hash_mismatch`.

This is the single strongest AI-safety property in the platform, because it closes the "approve a benign draft, publish a different one" attack entirely — including against the AI itself.

12. Learning opportunities — what the system does not yet do

The platform generates a large volume of high-quality training signal and **uses none of it**:

AVAILABLE SIGNAL	WHERE IT LANDS	CURRENTLY USED FOR
Every QA verdict + violation codes	`agent_runs.output`, `task_transitions`	nothing
Every human approve/reject + identity	`approval_actions`, `gate_decisions`	nothing
Every redaction block + pattern id	`gate_decisions`	nothing
Every budget breach	`gate_decisions`	nothing
`cost_per_accepted_asset` per agent	`kpi_rollup_cost_per_accepted_asset`	reporting only
Engagement by post archetype	`kpi_rollup_engagement_by_archetype`	reporting only
Every eval result	CI output	gating only

The four highest-value closed loops that the data model **already supports** and nobody has built:

1. **Approval-rate feedback → autonomy level.** A `(function_id, action_class)` pair with 50 consecutive human approvals and zero rejections is empirically safe to promote from level 1 to level 3. `autonomy.yaml` is already the single point of change.
2. **QA violation frequency → prompt improvement.** `violations[]` codes are already structured and countable per function.
3. **Engagement by archetype → content planning.** The rollup already exists; nothing reads it back into the weekly plan.
4. **`cost_per_accepted_asset` → model tier routing.** If Haiku's accepted- asset cost beats Sonnet's for a given function, `routing.yaml` is a one-line change — the file's own header already frames a model change as "one reviewed line."

[INFERRED] None of these require new data collection. They require a feedback service that reads what is already being written. That is the single highest-leverage AI investment available to this codebase.

13. AI architecture scorecard

DIMENSION	ASSESSMENT
Prompt versioning & testing	**Excellent** — git-versioned, 5+ golden evals each, prompt-coupled oracle, regression fixture
Cost governance	**Excellent** — per-function budgets, tier downgrade, 3-row metering, unit-economics KPI
Data protection	**Strong** — firewall + consent gate + audited exemptions; pattern coverage admittedly incomplete
Output governance	**Excellent** — hash-bound approval, single-use tokens, default-deny naming
Failure handling	**Strong** — 5 distinct semantics, cascade fail-fast, business-verdict separation
Observability	**Strong** — closed-enum spans, W3C propagation, structured JSON logs
Agent capability	**Weak** — 3 of 23 agents run; no tool-use loop; no multi-step reasoning
Memory	**Weak** — no semantic memory; DAG-lineage only
Learning	**Absent** — rich signal captured, zero feedback loops
Model portability	**Strong in design, single-provider in fact**

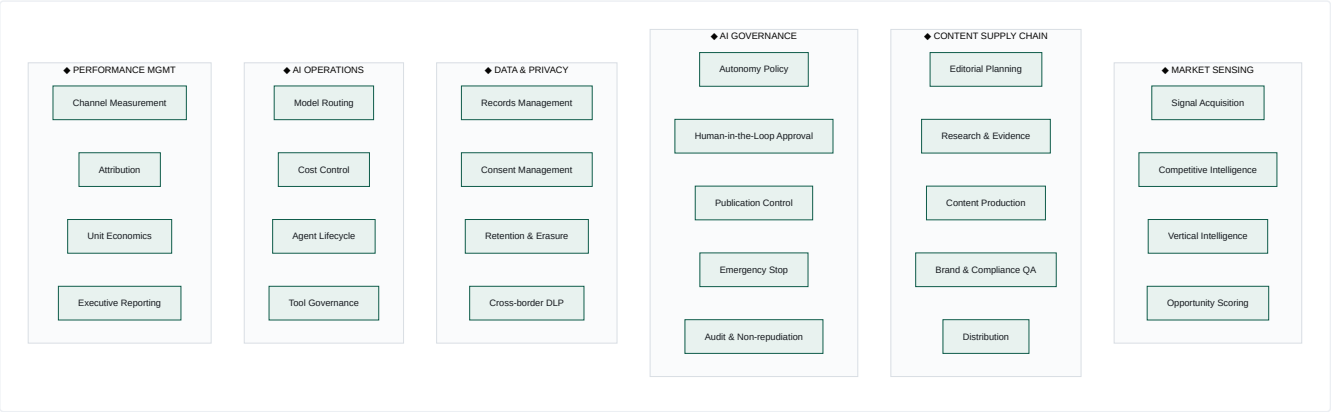
06

Business Architecture

*Mapping every implemented feature to business capability, department, process, objective, enterprise function, executive owner and value delivered. Executive owners are **[INFERRED]** — the codebase names no org chart. They are assigned by the standard functional accountability that a Series A/B company would apply.*

1. Business Capability Map

The classic three-tier capability map (Level 1 domains → Level 2 capabilities → Level 3 features), built from what the code actually does.



2. Capability → implementation → maturity

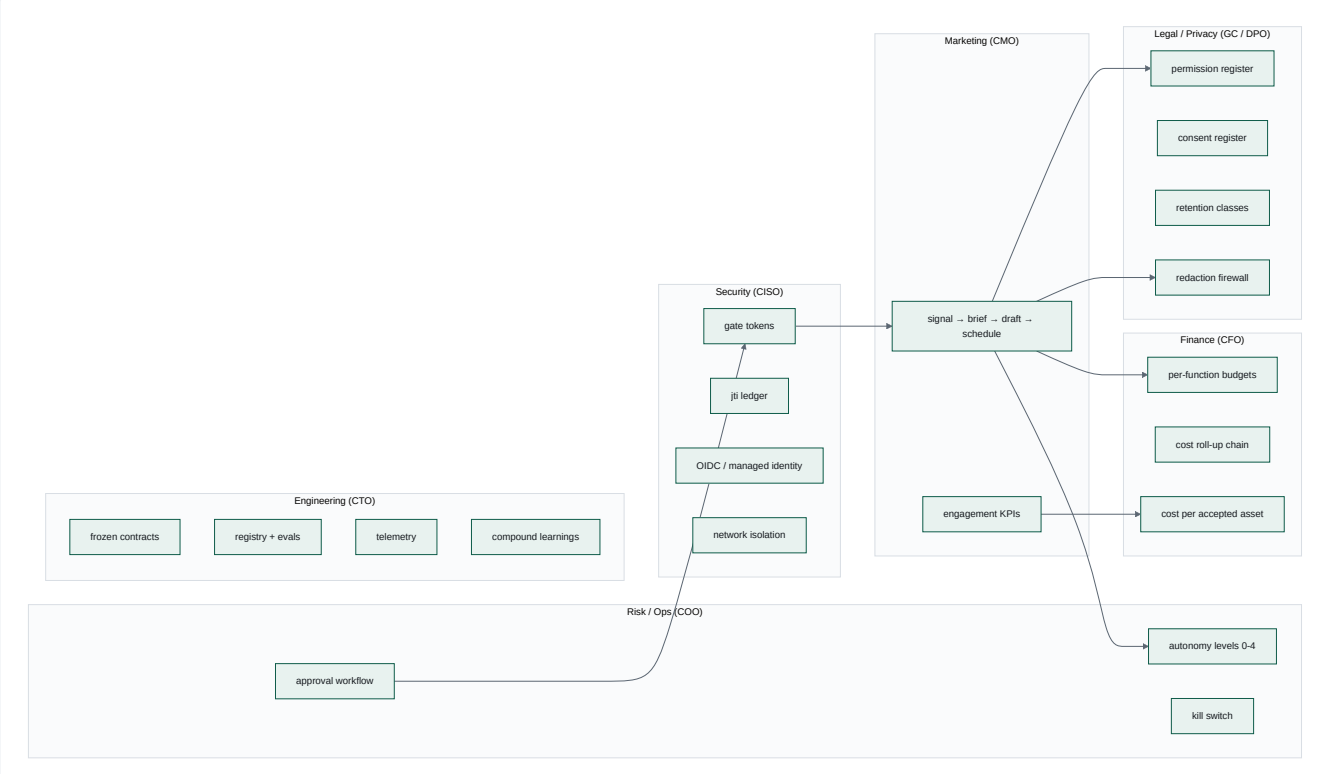
L2 CAPABILITY	IMPLEMENTED BY	MATURITY
Signal Acquisition	`ingest_signals_handler` + mcp-web + fn 09 + `fetch_sources.yaml`	**L4** live
Competitive Intelligence	fn 10/11/12/13/16 packages; 5 loop task_types	**L1** packages exist, task_types pass through
Vertical Intelligence	fn 18-01…18-06 + `_shared/vertical-intelligence-method.md`	**L1** same
Opportunity Scoring	`score_signals_handler` ; `functions/_shared/scoring-policy.yaml` ; `opportunity_cards`	**L3** live and read; the rule itself is still confidence-only
Editorial Planning	`weekly-content-loop.yaml` Monday node	**L1** DAG only
Research & Evidence	fn 41; `evidence_grade` taxonomy; fn 09 source rules	**L2**
Content Production	fn 39/42/43/45/46/47/52; `draft_content_handler`	**L2**, only fn 42 runs (**L4**)
Brand & Compliance QA	fn 02 + `permission_check.py` + `safety_suite.py` + `qa_review_handler`	**L4** live, terminal verdict
Distribution	Publisher + mcp-buffer + `schedule-social-buffer`	**L3**, dry-run only
Autonomy Policy	`autonomy.yaml` + `policy_loader.py` + `/gate-check`	**L4**
Human-in-the-Loop Approval	`approval_inbox` + `approval_action` + Teams card	**L4**
Publication Control	Publisher's 12-branch refusal matrix + gate tokens	**L3** (Vault record stubbed)
Emergency Stop	`kill_switches` + uncached reads in 2 services	**L3** (console client mocked)
Audit & Non-repudiation	`gate_decisions` , `approval_actions` , `publish_attempts` , `audit_log` , `task_transitions`	**L4**
Records Management	Vault 9 object types + 6-field taxonomy	**L4**
Consent Management	`consent_register` + `consent.py` gate + `consent_linkage`	**L3** (no subject-facing flow)
Retention & Erasure	`retention.py` + `caj-vault-retention-expiry`	**L3**
Cross-border DLP	`redaction.py` + frozen rules contract	**L4**
Model Routing	`routing.yaml` + provider registry	**L4**
Cost Control	`budgets.yaml` + `budget.py` + `metering.py`	**L4**
Agent Lifecycle	`services/registry` toolchain	**L2**, not consumed at runtime
Tool Governance	3 MCP servers + `tools.yaml` manifests	**L3**
Channel Measurement	`analytics-ingest` 4 sources	**L3**, 3 of 4 fixture-backed
Attribution	`utm.py` + `utm_campaign_map` + `utm_quarantine`	**L2**
Unit Economics	`kpi_rollup_cost_per_accepted_asset`	**L3**
Executive Reporting	Fabric export + Power BI starter dataset + console `/costs`	**L2**

3. Feature → business mapping (the full table)

FEATURE (CODE)	BUSINESS CAPABILITY	DEPARTMENT	BUSINESS PROCESS	BUSINESS OBJECTIVE	ENTERPRISE FUNCTION	EXEC OWNER [INFERRED]	VALUE DELIVERED
`daily-signal-loop.yaml`	Signal Acquisition	Marketing	Market monitoring	Never miss a relevant market move	Demand Generation	CMO	Replaces a daily manual scan; runs 06:00 unattended
fn 09 + `fetch_sources.yaml`	Signal Acquisition	Marketing	Source curation	Evidence-backed claims only	Brand	CMO	Every claim traceable to a URL; `confidence` never rounded up
`_render_brief` (deterministic)	Editorial Planning	Marketing	Daily brief production	Cheap, reproducible internal briefing	Internal Comms	CMO	Zero LLM cost; byte-identical for identical input
fn 42 + `draft_content_handler`	Content Production	Marketing	Social content creation	Scale thought leadership	Brand	CMO	Draft produced from positioning.md, not improvisation
fn 02 + `qa_review_handler`	Brand & Compliance QA	Marketing + Legal	Pre-publication review	Never publish an indefensible claim	Brand + Risk	CMO / GC	A blocking, machine-readable brand gate; no partial pass
`docs/permission-register.yaml`	Consent Management	Legal	Client reference clearance	Never name a client without written permission	Legal & Compliance	**General Counsel**	Default-deny; absence blocks identically to UNCLEARED
`autonomy.yaml` levels 0–4	Autonomy Policy	Risk / Ops	Delegation of authority	Explicit, reviewable AI mandate	Enterprise Risk	**COO / CRO**	The AI's "delegation of authority matrix" as code
`approval_inbox` + Teams card	Human-in-the-Loop	Marketing Ops	Approval workflow	Accountable human sign-off	Governance	COO	Single-use, expiring, identity-bound approval
Gate token (RS256, hash-bound)	Publication Control	Security	Authorisation	Approve *these bytes*, not "this thing"	InfoSec	**CISO**	Closes approve-A-publish-B entirely
`jti_ledger`	Publication Control	Security	Replay prevention	One authorisation = one action	InfoSec	CISO	Durable across replicas and restarts
`kill_switches`	Emergency Stop	Ops	Incident response	Stop everything in <5s	Business Continuity	COO	Uncached; global or per-function
`gate_decisions` (append-only)	Audit	Compliance	Evidence retention	Reconstruct any decision	Internal Audit	**CFO / GC**	One shape for human and machine deciders
`redaction.py` firewall	Cross-border DLP	Legal / Security	Data transfer control	No PI to a US inference provider	Privacy	**DPO / GC**	Blocks pre-transfer; every block audited
`consent_register` + gate	Consent Management	Legal	Lawful-basis management	POPIA s11 compliance posture	Privacy	DPO	Per-channel, per-purpose, revocable
`retention_class` + sweep	Retention & Erasure	Legal / IT	Records lifecycle	Defensible disposal	Records Mgmt	GC / CIO	4 classes incl. legal hold; fails closed
6-field taxonomy (immutable)	Records Management	Data Governance	Metadata management	Every object classified at birth	Data Governance	**CDO**	Search, retention and evidence all key off it
`budgets.yaml` + `budget.py`	Cost Control	Finance	AI spend control	Predictable AI opex	FP&A	**CFO**	Soft breach degrades service, never blocks work
3 `costs` rows per completion	Cost Control	Finance	Cost allocation	Attribute spend to a function	FP&A	CFO	Unbroken chain costs→agent_runs→campaigns
`kpi_rollup_cost_per_accepted_asset`	Unit Economics	Finance + Marketing	Performance mgmt	Know the cost of usable output	FP&A	CFO	The AI-labour productivity metric
`routing.yaml` tiers	Model Routing	Engineering	Vendor management	Avoid provider lock-in	IT Architecture	**CTO**	Provider swap = 1 module + 1 register call
`services/registry`	Agent Lifecycle	Engineering	Change management	Prompts are versioned software	Engineering	CTO	Signed, reproducible, eval-gated artefacts

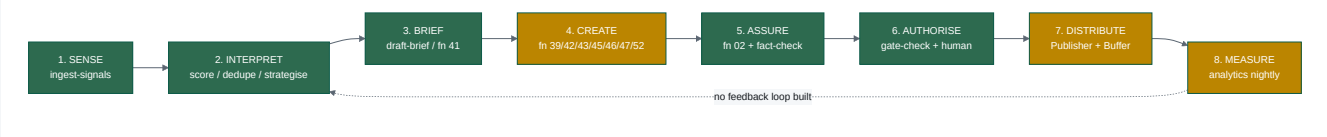
FEATURE (CODE)	BUSINESS CAPABILITY	DEPARTMENT	BUSINESS PROCESS	BUSINESS OBJECTIVE	ENTERPRISE FUNCTION	EXEC OWNER [INFERRED]	VALUE DELIVERED
mcp-buffer draft-only	Tool Governance	Engineering / Risk	Least privilege	Tools cannot exceed their mandate	InfoSec	CISO	Publish path does not exist to be misused
`telemetry-lib` closed enum	Observability	Engineering	Monitoring	Trace without leaking	IT Ops	CTO	PII structurally excluded from telemetry
`console/` 6 screens	Operational Oversight	Marketing Ops	Daily operations	One place to see what the AI did	Operations	COO	Dual HTML/JSON — human and agent parity
`analytics-ingest` nightly	Channel Measurement	Marketing	Performance reporting	Close the produce→perform loop	Analytics	CMO / CDO	4 sources → 4 KPIs → Fabric
Fabric export + Power BI	Executive Reporting	Finance / Exec	Management reporting	Board-ready numbers	Corporate Reporting	CFO	Lands in the company's own Fabric estate
OIDC-only CI/CD	Secrets Management	Engineering	Deployment	No standing credentials	InfoSec	CISO	Zero client secrets in 13 workflows
`.compound/learnings/`	Knowledge Management	Engineering	Continuous improvement	Don't repeat a mistake	Org Learning	CTO	79 classified, cross-referenced learnings

4. Departmental view — who this platform serves



The observation that matters commercially: a conventional marketing automation platform serves the CMO alone. This one has *material, code-level* deliverables for the **GC, the DPO, the CFO, the COO, the CISO and the CDO** — each of which is an independent budget holder and, in an enterprise sale, an independent blocker. See `08-product-positioning.md`.

5. Process map — the end-to-end value chain



Green = fully operational · Amber = partially operational · Red = modelled but not implemented.

The hole at step 2 is closed; the broken return arrow at step 8 remains. `score-signals` writes a ranked `opportunity_cards` row per signal, the brief leads with the best-evidenced ones, and the weekly content loop picks its pillar from those cards rather than from an ISO-week rotation. What is still missing is the return arrow: nothing feeds measured performance back into what gets planned. That one gap is now the difference between a *pipeline* and an *operating system* — see `07-operating-model.md`.

Step 2 is green in the sense that it runs, is read, and is governed by a reviewed policy file. It is not green in the sense of being *finished*: the score is function 09's own `confidence` weighted by pillar, and no selection cut is configured, so scoring today reorders the brief rather than filtering it. Both are one reviewed edit to `functions/_shared/scoring-policy.yaml` away.

6. Business rules encoded in code (the compliance register)

Every one of these is a business policy that exists as executable code, not as a document someone might read:

#	RULE	WHERE
BR-01	No client may be named publicly without written permission; **absence from the register is not permission**	`permission_check.py` (with a self-test proving absent ≡ UNCLEARED)
BR-02	Every claim carries a client, a number or an artefact	fn 02 `unsupported-claim` ; fn 42 hard rule 1
BR-03	South African English throughout	fn 02 `sa-english-spelling`
BR-04	Exactly one CTA per publishable asset; internal briefs exempt	fn 02 rule 4
BR-05	Every URL is a canvasintelligence.com link with 3 UTM params; internal briefs exempt	fn 02 rule 5
BR-06	No link shorteners (they break attribution and hide destinations)	fn 02 rule 2
BR-07	Posts close on the roof line "Your Data. Delivered."	fn 42 hard rule 3
BR-08	Exactly five messaging pillars, named verbatim	fn 09 rule 4, fn 42
BR-09	≥3 signals, ≥2 distinct domains, every signal source-attributed	fn 09 rules 1–3
BR-10	Thin evidence is never rounded up to medium confidence	fn 09 rule 5
BR-11	Manufacturing is proof-light — never `evidence_grade: strong`	fn 18-03
BR-12	Paid media spend is **blocked outright** (level 0)	`autonomy.yaml` `publish.paid_ad`
BR-13	Long-form owned content requires elevated approval (level 2)	`autonomy.yaml` `publish.blog_article`
BR-14	No `smoke.*` / `test.*` function may exist in the autonomy policy	`test_policy.py` invariant
BR-15	No `publish` -class entry may sit above level 2	`test_policy.py` invariant
BR-16	Buffer Free tier: ≤10 queued posts; the weekly plan caps at 8	`config.py` + `weekly-content-loop.yaml`
BR-17	Nothing publishes live by default	`PUBLISHER_DRY_RUN=true`
BR-18	A proof-circuit asset can never publish live, whatever the flag says	`vault_lookup.py` agent_name check
BR-19	Consent is per-channel, per-purpose, and revocable	`consent_register` schema
BR-20	Every object declares its retention class at creation, immutably	`object_taxonomy` + PATCH 422

This list is the strongest commercial artefact in the repository. Twenty business policies, each independently testable, each with a named violation code. That is what an auditor or an enterprise procurement team asks for and almost never gets.

07

Operating Model

What operating model has been built — including the one that was built unintentionally, which is the more interesting of the two.

Part A — The operating model you meant to build

A.1 The model, named

Using the standard McKinsey operating-model frame (People · Process · Technology · Governance · Data · Performance), what exists is:

> **A machine-executed marketing operating model with human authority retained at exactly one point — the publication gate — and with all delegation of that authority expressed as versioned policy.**

The conventional marketing operating model asks: *who does the work, and who approves it?* This one answers: **the machine does the work; the human approves only the irreversible step; and the boundary between those two is a YAML file under change control.**

A.2 Operating-model canvas, as implemented

DIMENSION	CONVENTIONAL MARKETING ORG	WHAT THE CODE IMPLEMENTS
Work allocation	People assigned to briefs	`loops/*.yaml` DAG; `uuid5` deterministic task ids
Capacity	Headcount	Model tier + `budgets.yaml` daily USD limit
Escalation	Manager judgement	`autonomy.yaml` levels 0–4, fail-closed at 0
Quality control	Peer review	fn 02 Brand Steward QA — blocking, machine-readable, no partial pass
Sign-off	Email / Slack approval	Single-use expiring link, identity from Entra, hash-bound to exact bytes
Audit	Recollection + inbox archaeology	`gate_decisions` , `approval_actions` , `publish_attempts` , `task_transitions` — all append-only
Emergency control	"Stop posting" in a group chat	`kill_switches` , uncached, <5s, global or per-function
Cost control	Retainer / budget line	Per-function daily USD limit, soft breach downgrades tier
Performance mgmt	Monthly report deck	Nightly rollups incl. `cost_per_accepted_asset`
Knowledge	Tribal, in people's heads	`positioning.md` quoted verbatim into prompts; 79 compound learnings

A.3 The autonomy ladder as a delegation-of-authority matrix

`services/gatekeeper/policy/autonomy.yaml` is, in enterprise-governance terms, a **Delegation of Authority (DoA) matrix** — normally a board-approved document living in a policy library. Here it is executable:

LEVEL	MEANING	CURRENTLY ASSIGNED TO
0 blocked always	No approval can unblock it	`publish.paid_ad` ; **and every unlisted pair** (default)
1 single approver	One human click	`publish.social_post`
2 elevated	*Currently identical to 1*; reserved for quorum	`publish.blog_article`
3 auto-approved, audited	No human; decision row written	`draft.social_post` , `draft.brief`
4 autonomous passthrough	No human; logged	`analyse.signal` , `analyse.campaign_performance`

Read as a business statement, this policy says: *"AI may read and analyse freely. AI may draft freely, with a record. AI may propose a social post but a human must click. AI may propose long-form but with more scrutiny. AI may never spend money on paid media. Anything not explicitly listed is forbidden."*

That is a complete, coherent, defensible AI mandate — and it fits on one screen. **Most enterprises spend six months in committee to produce a worse version of this document, and then cannot enforce it.**

A.4 Operating rhythm (cadence)

CADENCE	TRIGGER	WHAT RUNS
Daily 06:00 SAST	`dailySignalLoopTrigger`	Signal loop: ingest → brief → QA → approval card
Weekly Mon 07:00 SAST	`weeklyPlanningTrigger`	Content studio: plan → research → draft ×6 → dual QA → schedule
Nightly 03:00 SAST	`caj-analytics-nightly-ingest`	Ingest 4 sources → reconcile UTM → 4 KPIs → Fabric export
Monthly, last day	`monthEndReportingTrigger`	Fires a heartbeat — **no loop definition matches it**
On demand	`POST /retention-expiry-runs` or the CA Job	Retention sweep
Per deploy	`deploy-loop-e2e-smoke.yml`	Live end-to-end proof of life

Finding: the month-end trigger publishes a heartbeat that `handle_heartbeat_message` will log as `heartbeat_unknown_loop` and discard. The monthly reporting cadence exists in infrastructure and not in software.

Part B — The operating model you built unintentionally

This is the more important half of this document.

B.1 What actually happened

The repository was **built by an AI agent loop, running one unit of work per git worktree per branch**, and it accumulated its own engineering knowledge as it went. Evidence:

- `README.md` documents worktree-per-session: `git worktree add ../cmos-session-<id> -b session/<id> main`. Branch names are `session/{id}` (`s0-foundation`, `s1-gateway`, `s2-vault`, `s3-orchestrator`, `s4-governance`, `s6-registry`, `s7-console`, `s8`, `s9-analytics`, `s10-intelligence`, `s11-content`).
- `.compound/learnings/` — 79 numbered learnings across four classes, each with a status field carrying strengthening and recurrence history ("*strengthened again 2026-08-02 — 3rd recurrence on s9-analytics*").
- Commit messages and code comments carry finding codes and live incident references: `F-CASCADE-QA-BLOCKED (4 Aug 2026, heartbeat round 17)`, `F-INGEST-PUBLIC-SOURCE`, `deploy-loop-e2e-smoke #33`.
- Comments preserve *root-cause narratives*, not just fixes — see `migrations/0003_qa_blocked_reason.sql`, which explains that the previously attempted fix in 0004 "had zero effect for exactly this reason: 0004 was never reached."
- References to `.loop/spec.json`, `.loop/plan.json`, `.loop/review.json`, `.loop/lenses.json`, `.loop/domain.md` — a formal spec→plan→build→review loop with acceptance criteria (`AC-xx`), constraints (`C-x`), decisions (`DE-x`) and multi-lens review (risk-security, data-residency, performance).

B.2 The unintentional model, named

> **You have built a Compound Engineering Operating Model: a spec-driven, isolation-per-unit-of-work, multi-lens-reviewed, learning-accumulating software factory in which AI agents are the labour, and the organisational memory is a first-class versioned artefact.**

Five properties, each visible in the repository:

1. **Isolation per unit of work.** Worktree + branch per session. No shared working directory, no branch-switching races, `main` protected.
2. **Frozen interfaces as coordination protocol.** Parallel sessions coordinate through hash-pinned contracts and a documented *first-to-land- wins* rule (`mcp/README.md` records exactly this playing out for `tools.schema.json` — the MCP session's version was dropped in favour of the registry session's, and the manifests were reshaped to fit).
3. **Learning capture as a build artefact.** Not a retro doc — a classified, numbered, cross-referenced corpus that later specs cite as criteria (`LEARN-000`, "learnings-as-criteria").
4. **Multi-lens adversarial review.** `.loop/lenses.json` and `review.json` are referenced by finding codes throughout (`RS-01` risk-security, `PERF-2` performance, `DR-3/4/5` data-residency, `DE-x` decisions). Findings propagate into code as named comments.
5. **Live production as the test environment of record.** The smoke test runs against `cmos-dev` and its green criterion was *redefined* to accept a legitimate QA_BLOCKED verdict as proof-of-life (`F-SMOKE-GREEN-REDEFINITION`). Failures are root-caused from Log Analytics and the analysis is written into the code.

B.3 Why this is the more valuable asset

Read the two assets side by side:

	CANVAS MARKETING OS	THE COMPOUND ENGINEERING LOOP
What it is	An AI marketing platform for one company	A method for building governed AI systems
Addressable market	Canvas Intelligence + lookalike SA data firms	Every enterprise deploying AI agents
Replicability	High — competitors can build a marketing tool	Low — this took 79 learnings and ~20 live incidents to develop
Evidence of value	A working platform	A working platform, *built by the method*
Defensibility	Feature parity is achievable	The learning corpus is path-dependent

The compound learnings are not documentation about the product. They are **proof that the method works**, with the product as the exhibit. That inverts the usual pitch: instead of "buy our marketing platform", the stronger position is *"we built a fully-governed, POPIA-aware, cost-metered, human-gated AI production system with an agent loop — here it is running in production, and here are the 79 things we learned doing it."*

[INFERRED] This is the single most under-exploited asset in the repository.

Part C — Operating-model gap analysis

C.1 What exists (operational today)

COMPONENT	EVIDENCE
Scheduled autonomous execution	3 Logic Apps + 1 CA Job, live
Deterministic, reproducible decomposition	`uuid5`, golden-file tests
Application-level reliability engineering	retry/backoff/dead-letter/cascade state machine
Policy-as-code delegation of authority	`autonomy.yaml`, fail-closed
Identity-bound, hash-bound human approval	Easy Auth + RS256 + `jti` ledger
Complete append-only audit across 5 tables	`gate_decisions` et al.
Sub-5s emergency stop	uncached kill switch in 2 services
Per-function AI cost budgets with graceful degradation	soft breach → tier downgrade
Pre-transfer PII interdiction, every block audited	`redaction.py`
Default-deny client-naming clearance	`permission-register.yaml`, nothing CLEARED
Immutable governance taxonomy on every object	`object_taxonomy`, PATCH 422
Retention classes with fail-closed erasure	`retention.py`
Prompt-as-software lifecycle with regression evals	`services/registry`
PII-safe distributed tracing	closed-enum spans
Zero-standing-credential deployment	OIDC + managed identity, 13 workflows
Nightly measurement incl. AI unit economics	4 KPI rollups
Organisational learning capture	79 compound learnings

C.2 What is partially implemented

COMPONENT	STATE	WHAT'S MISSING
The agent estate	3 of 23 packages run	20 task_types fall through to a no-op pass-through
The weekly content studio	DAG valid, gates correct	Every stage is a pass-through; no content produced
Publication	Full verification chain built	`PUBLISHER_DRY_RUN=true`; Vault publish record is a stub
The console	Deployed behind Easy Auth	Gatekeeper client is `mock`; approval/kill-switch screens read fixtures
Measurement	Pipeline runs nightly	3 of 4 sources are fixtures; no Power BI workspace provisioned
Registry	Full toolchain, signed artefacts	Nothing verifies the signature at runtime; `registry.yml` covers 3 of 23 packages
Consent	Gate + linkage + revocation	No subject-facing request flow, no DSAR handler
Notifications	Teams cards built and tested	`TEAMS_WEBHOOK_URL` absent by default; inbox row is the fallback
Approval semantics	Levels 0–4 defined	Level 2 behaves identically to level 1
Opportunity scoring	`score-signals` writes a ranked card per signal; the brief and the weekly pillar both read them	The score is function 09's `confidence` weighted by pillar, and nothing else — no recency, no corroboration. No selection cut is configured, so it reorders rather than filters. Both are edits to `functions/_shared/scoring-policy.yaml`, not code

C.3 What is missing entirely

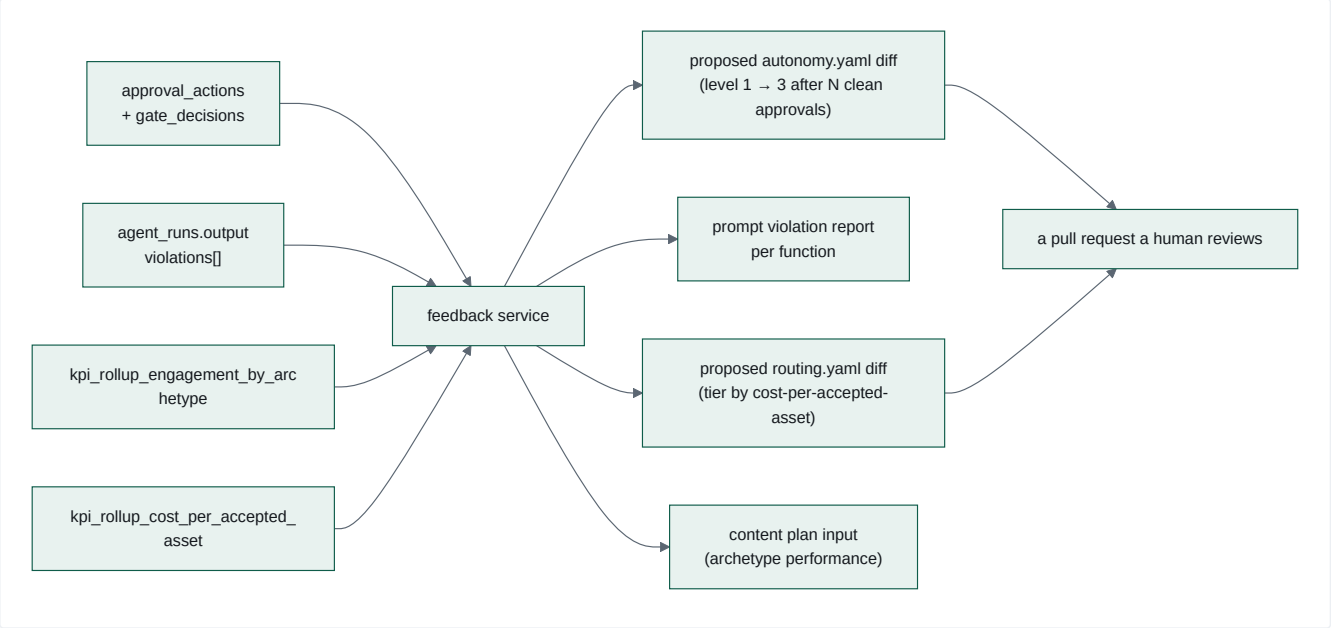
MISSING CAPABILITY	BUSINESS CONSEQUENCE
Any feedback loop	The system produces, governs and measures — but never *learns*. Approval rates, QA violation frequencies, engagement by archetype and cost-per-accepted-asset are all captured and none is read back. **This is the single largest gap between "pipeline" and "operating system."**
Multi-tenancy	No `tenant_id` in any of 5 schemas. The platform serves exactly one company. Every commercial model except "internal tool" or "managed service per instance" requires a schema-wide change.
Role-based authorisation	Any authenticated tenant user reaches the kill switch, the cost ledger and Vault search. Mitigated only by an Entra Portal setting a human must remember.
Vault API authentication	Zero authn/authz on the system of record. Network isolation is the only control, and it is documented as an unapproved accepted risk.
Month-end reporting	Logic App fires; no loop matches; heartbeat discarded.
Dead-letter alerting	`DeadLetterAlert` is published to the event queue; nothing consumes it. Silent failure.
Cost forecasting / spend caps beyond daily	Daily USD per function only. No monthly cap, no forecast, no anomaly detection.
Content editing / revision	`version` + `predecessor_asset_id` exist; nothing writes a v2. A rejected draft has no revision path.
Rollback / undo of a publication	No unpublish, no retraction workflow.
SLA / SLO definitions	No latency, availability or freshness targets anywhere.
Runbooks for the AI failure modes	Runbooks exist for deploy and auth; none for "the agent produced something wrong and it shipped."

Part D — Target operating model

The three moves that convert what exists into a genuine operating system:

D.1 Close the learning loop (highest leverage, lowest cost)

Build one service — call it `services/feedback` — that reads what is already being written and proposes changes to the policy files that already exist:



Every output is a **diff to a versioned policy file**, reviewed by a human, merged by CI. No new autonomy is granted implicitly. This is the "auto-tuning governance" pattern and it is directly enabled by decisions already made.

D.2 Activate the agent estate

The gap between 23 packages and 3 running handlers is closed by making dispatch **registry-driven** rather than table-driven:

```
DISPATCH_TABLE (hardcoded, 5 entries)
  → registry-resolved handler: task_type → function package
    → generic handler: read prompt.md, build user_content from schema.json,
      call gateway, validate output against schema.json, write artefact
```

Four of the five existing handlers already share that exact shape. Extracting a generic handler and letting `registry.json` name the function per task_type would activate 20 packages **without writing 20 handlers** — and would give the registry a runtime role, closing the "signature never verified" gap at the same time.

D.3 Decide the tenancy question

Multi-tenancy is not a feature; it is a schema decision that gets exponentially more expensive with every row written. The three viable answers, in order of increasing investment:

1. **Single-tenant, deployed per customer** (nearest to today) — package `main.bicep` as a deployable unit; each customer gets their own resource group. No code change. Highest ops cost, fastest to market.
2. **Schema-per-tenant** — one Postgres, one schema set per tenant, tenant resolved at connection time. Moderate change; retains the frozen-contract guarantee.
3. **Row-level tenancy** — `tenant_id` on all 27 tables + RLS. Cheapest to run, most expensive to build, and **breaks the frozen v1 contract**, which is precisely the kind of change the freeze exists to force a deliberate decision about.

This decision should be made before more production data accumulates.

08

Product Positioning

Derived only from what the code does. No market research, no competitor interviews — every positioning claim below traces to an implemented capability.

---

1. The positioning problem, stated plainly

The repository is named "Canvas **Marketing** OS". Its own `README.md` calls it an *"agent-native marketing operations platform"*. But the code distribution says something different:

Measured, not estimated (`wc -l` , non-test Python across `services/` , `console/` and `mcp/` ; handler region isolated by AST-style line classification):

MEASURE	LINES
Non-test Python across the whole platform	**21,182**
Executable Python containing **any** marketing domain logic — `dispatch.py` L443–1023	**395** (1.9%)
Marketing content shipped into the running orchestrator image (5 files, Dockerfile L102–106) + the permission register	**607**
Marketing artefacts authored but never executed (20 inert packages: prompts, evals, checkers, skills)	**~14,500**

Every line of marketing logic that runs in production lives in one file, `services/orchestrator/orchestrator/dispatch.py` , in five handler functions — and 37% of that file is comments. Nothing else in 21,182 lines of Python knows what marketing is.

The marketing capability that *runs* is: fetch four URLs, summarise into signals, render a brief, QA it, draft one LinkedIn post, request approval. That is a modest marketing product. The governance chain around it is an enterprise-grade one.

The product is mis-named relative to what was built. That is the central positioning finding.

2. If Gartner classified this product

Gartner would struggle to place it in one Magic Quadrant, and would most likely split it. Candidate categories, assessed against the code:

Primary fit — AI Agent Orchestration / AI Governance Platform

(An emergent category; Gartner's 2025–26 framing is "AI Agent Platforms" and "AI Trust, Risk and Security Management" — TRiSM.)

Evidence for: `autonomy.yaml` (policy-based agent authority), `gate_decisions` (decision audit), the redaction firewall (AI TRiSM's "content anomaly detection" primitive), per-function budgets (AI cost governance), the gate token (agent action authorisation), `services/registry` (model/prompt lifecycle), kill switches (AI incident response). **This is where the strongest, most complete capability lives, and it is domain-agnostic.**

Evidence against: no agent marketplace, no low-code builder, no multi-agent reasoning, no LLM-agnostic breadth in practice (one provider implemented).

Secondary fit — Content Marketing Platform / Marketing Resource Management

Evidence for: content lifecycle from brief to publication, brand compliance QA, approval workflow, digital asset storage with versioning, channel scheduling. Evidence against: no calendar UI, no DAM interface, no collaboration, no creative brief authoring by humans, no campaign planning, one channel.

Tertiary fit — Data Privacy Management / Consent & Preference Management

Evidence for: `consent_register` with lawful basis, per-channel/per-purpose grants and revocation; consent gating enforced at write time; retention classes with defensible disposal; cross-border transfer interdiction; full audit. Evidence against: no data-subject portal, no DSAR workflow, no data mapping / RoPA, no breach management.

Gartner would probably say

> *"A vertically-focused AI agent orchestration platform with unusually mature governance, applied to a narrow marketing use case. Evaluate the governance layer independently of the application layer."*

That sentence is also the product strategy.

3. Overlapping enterprise software

Mapped honestly — where the code genuinely overlaps, and where it does not:

VENDOR / CATEGORY	OVERLAPS ON	DOES NOT OVERLAP ON
Jasper / Writer / Typeface (enterprise AI content)	AI content generation, brand voice enforcement, approval workflow	They have brand-voice training, editors, DAM, integrations, teams. We have deeper *governance* and *cost control*
Sprinklr / Hootsuite / Sprout (social suites)	Scheduling, publishing approval, engagement analytics	They have listening, inbox, CRM, ads, dozens of channels. We have one channel via Buffer, dry-run
Adobe Workfront / Aprimo (MRM)	Brief→asset→approval lifecycle, DAM with versioning	They have resource planning, budgets, project mgmt, proofing UI
OneTrust / Securiti / TrustArc (privacy)	Consent register with lawful basis, retention schedules, audit, cross-border controls	They have DSAR automation, data discovery, RoPA, vendor risk, breach mgmt
Credo AI / Holistic AI / Fairly (AI governance)	Policy-as-code, model registry, decision audit	They focus on model risk/fairness assessment and regulatory mapping; we focus on *runtime* action authorisation
LangSmith / Langfuse / W&B Weave (LLMOps)	Prompt versioning, eval harness, tracing, cost tracking	They have playgrounds, dataset mgmt, human labelling UI, model comparison. **We have runtime enforcement they do not**
Temporal / Airflow / Dagster (orchestration)	DAG definition, retries, dead-letter, deterministic runs	They are general-purpose with vastly richer scheduling, UI, backfill. We have governance they do not
Azure AI Foundry / Content Safety	Content filtering before provider call	Theirs is model-based and richer; ours is regex + fixtures, but *ours writes an audit row and theirs is not tied to a consent register*

The honest summary: there is no single vendor whose product is this combination. Not because the combination is brilliant, but because most vendors pick one side. That is simultaneously the opportunity and the risk — category creation is expensive.

4. What makes this fundamentally different

Six differentiators, each traceable to code, ranked by defensibility:

D1 — Approval is bound to bytes, not to an object

`contracts/gate-token/spec.md` + `services/publisher/app/hashing.py`. The gate token carries `content_hash` in canonical JSON; the Publisher **independently recomputes** SHA-256 over the raw bytes and refuses on any mismatch. It explicitly does *not* trust `assets.content_hash` because that column is nullable and could be written by another code path.

No competitor in the AI content space does this. Everywhere else, "approved" is a status flag on a row — and a row can be updated after approval. Here, approving a draft and publishing a different one is **cryptographically impossible**.

D2 — The refusal is the product

Publisher records **one immutable row on every branch**, with twelve distinct reason codes. Gatekeeper writes exactly one `gate_decisions` row per `/gate-check` on every branch. The Vault writes an audit row on every rejection, on an isolated connection so it survives the rollback.

Most systems log successes and swallow failures. This one treats *"the AI tried to do X and was refused because Y"* as the primary business record. That is what an auditor, a regulator or an enterprise procurement team actually asks for.

D3 — Default deny, everywhere, provably

- Unlisted `(function_id, action_class)` → autonomy level 0 → blocked.
- Client name absent from the register → blocked **identically** to explicit UNCLEARED, with a self-test that asserts the two produce the same `allowed`, the same status semantics and the same violation code.
- `data_subject_ref` present with no matching consent → 403 + audit.
- `asset_id` supplied but Vault lookup fails → refuse to publish.
- Gate token algorithm not in the pinned allowlist → refuse before any signature work.

Fail-closed is claimed by everyone. Here it is *tested* at every layer.

D4 — Cost governance with graceful degradation

Soft breach **downgrades the model tier and keeps working**; hard breach **queues the request as an escalated gate decision and hands back that row's id**. Three cost rows per completion (usd/tokens/ms) with an unbroken FK chain to campaigns. Then a nightly KPI: **cost per accepted asset**.

Most AI platforms report token spend. This one reports the *unit cost of usable output* — a CFO metric, not an engineering metric.

D5 — The dangerous capability doesn't exist

mcp-buffer has no publish tool. Not a disabled tool — **no tool**. A test greps every tool name and description against `publish|share.?now|send.?now|go.?live`. mcp-canva requires `template_id` on both creation tools, so free-form design generation is unrepresentable. mcp-web checks the host allowlist *before* any network call.

"Make the dangerous thing unrepresentable" is a stronger guarantee than "make the dangerous thing configurable-off", and it is legible to a security reviewer in seconds.

D6 — Jurisdiction-specific by design

SA ID number and SA phone-number regexes in a frozen contract. POPIA s11 lawful-basis modelling (a *value*, not a boolean). POPIA s72 cross-border transfer analysis documented as explicitly unresolved rather than assumed away. `southafricanorth` region pinned as a literal, never `resourceGroup().location`, with a documented reason. South African English enforced as a brand rule with its own violation code.

This is a South African data-sovereignty AI platform. For a Johannesburg CFO evaluating US AI tooling, that is not a feature — it is the purchase criterion.

5. What CIOs would call this

> **"An AI action-governance layer."**

The CIO's question is *"how do I let AI agents do real work in my environment without losing control of what they touch?"* This platform's answer is a five-layer chain they can inspect: policy → approval → signed token → boundary verification → audit. Plus the things a CIO checks first and usually finds missing: no standing credentials (OIDC + managed identity throughout), private endpoints on every data resource, internal-only ingress on six of eight apps, a kill switch with a proven sub-5s bound, and an accepted-risk register that *names the risks it hasn't closed*.

`docs/accepted-risks.md` is, in a CIO conversation, worth more than any feature. It says: Service Bus has no private endpoint (here are three compensating controls and the production path); the Vault API has no authentication (this was a builder judgement call, not a user-approved acceptance — flagged so it is tracked rather than silently shipped); the registry is signed with a committed dev key (here is why Key Vault couldn't be used, and here is the algorithm correction needed); the console authenticates but does not authorise. **Very few vendors will show a CIO that document.**

6. What CMOs would call this

> **"A marketing production line that can't embarrass us."**

The CMO's fear about AI marketing is a specific, nameable one: the tool publishes something wrong, or names a client without permission, or makes a claim legal can't defend, and it happens at 3am with nobody watching.

This platform's answer, in CMO language:

- **It cannot name a client we haven't cleared.** Not "it's told not to" — the register defaults to deny and absence is not permission.
- **Every claim needs a client, a number or an artefact.** Superlatives with nothing attached are a named, blocking violation.
- **It writes in our English.** `productise`, not `productize`.
- **It closes on our roof line.** Every post ends "Your Data. Delivered."
- **Nothing goes out without a human clicking, and the click is tied to the exact words approved.**
- **We can stop it in five seconds.**
- **We know what each usable post costs.**

Weakness to be honest about: today the CMO gets *one draft LinkedIn post per day, in dry-run*. The governance is enterprise-grade; the output volume is not yet. The 20 unwired function packages are the answer, and they are mostly written.

7. What investors would call this

Three framings, in ascending order of valuation and of execution risk:

Framing A — "AI-native marketing ops for regulated mid-market"

Vertical SaaS. Comparables: Jasper, Writer, Typeface. Multiples 6–10× ARR. Requires: multi-tenancy, more channels, a real editorial UI, activating the 20 packages. **Risk: crowded, well-funded, and the moat is governance the market may not yet price.**

Framing B — "The control plane for enterprise AI agents"

Horizontal infrastructure. Comparables: Credo AI, Vanta-for-AI, LLMOps platforms. Multiples 15–25× ARR. Requires: extracting the governance layer from the marketing layer, an SDK, multi-provider support, agent-framework integrations (LangChain, CrewAI, MCP-native). **The code already supports this: `gatekeeper`, `model-gateway`, `publisher`, `vault`, `registry` and `telemetry-lib` contain zero marketing logic.** The seam is real and it is clean.

Framing C — "Compound engineering — an AI software factory with proof"

Method + platform. The 79 compound learnings, the worktree-per-session model, the spec→plan→build→multi-lens-review loop, and a live production system built by it. Comparables: none clean. This is the framing that gets a different kind of conversation — but it needs the method extracted from this one repository to be credible.

The investor question that actually matters

> *"You built a marketing platform where 1.9% of the executable code contains any marketing logic at all. Was that the plan, or is the governance the product?"*

The honest answer — and the strongest one — is: **the governance is the product; marketing is the proof it works on something real.** Framing B follows directly from that answer, and the code supports it today.

8. Positioning statement

Built to the same structure as `docs/positioning.md`'s own house style:

> **Canvas Marketing OS is the AI action-governance platform that lets autonomous agents do real marketing work — with every model call metered, every claim evidence-graded, every client name permission-checked, every publication cryptographically bound to the exact bytes a named human approved, and every refusal permanently recorded.**
>
> Short form: *Autonomy you can audit.*
>
> Elevator: *Enterprises want AI agents doing real work. Their CIOs, GCs and CFOs won't allow it, because nobody can prove what the agent was allowed to do, what it actually did, what it cost, or who said yes. We built the control plane that proves all four — policy as code, human approval bound to content hashes, per-function cost budgets, and an append-only decision ledger. It runs today, in production, on Azure, in South Africa, governing a live marketing pipeline.*

9. Positioning risks

RISK	WHY IT'S REAL
The name works against the value	"Marketing OS" invites comparison to Sprinklr; the governance layer then reads as over-engineering rather than as the point
Category creation is expensive	"AI action governance" is not a line item in anyone's budget yet
Single-tenant is not a SaaS	No `tenant_id` in any of 5 schemas; every commercial model except per-instance managed service needs a schema change
Governance without volume is unconvincing	A demo showing one dry-run LinkedIn post per day undersells a five-layer control chain
Regional strength is regional limit	SA-specific regexes and POPIA framing are a moat in Johannesburg and a rebuild in Frankfurt
Provider-agnostic in design, single-provider in fact	The extension point is real and unexercised; a buyer will ask

09

Technical Debt Register

*Prioritised by business impact, not by engineering annoyance. Every item cites the file. Severity: **S1** blocks revenue or creates liability · **S2** blocks scale or credibility · **S3** slows delivery · **S4** hygiene.*

> **Re-verified against `main` @ `53a2560` , 17 Aug 2026.** Resolved items are struck through and kept, with the evidence that closed them, rather than deleted — a register that only shows open debt hides how it was paid down.
>
> **Closed this pass:** TD-01 (the largest item in the register), TD-22, TD-30. **Re-measured and raised:** TD-17 (1,138 → **7,068 lines**, S3 → S2), TD-13 (no alerting exists anywhere in IaC, not just for dead-letters). **Added:** TD-34 (`post_archetype` has no writer), TD-35 (unapproved QA policy gating publication), TD-36 (Buffer queue cap at the daily cadence), TD-37 (`mcp-canva` deployed and credentialled with no caller).
>
> A withdrawn finding is recorded too, because the failure mode is cheap to repeat: an earlier pass read `weekly-planning-trigger.bicep` 's stale `TEMPORARY` marker as a defect and costed the daily cadence as ~7× overspend. Daily is the intended cadence. **A comment describing intent is not evidence of intent** — the marker has since been removed and the file states the ruling.

Priority 1 — Business-blocking

IDs are stable identifiers assigned in discovery order, not priority ranks.

TD-31 · mcp-web's live mode is undeclared config drift — the next infra deploy silently reverts all knowledge intake to a synthetic fixture · S1

Where: `infra/main.bicep` L960–978 (`mcpWebApp`), `infra/modules/mcp/container-app.bicep` (`env: concat(envVars, keyVaultSecretEnv)`), `mcp/mcp-web/app/tools.py::fetch_url` .

`fetch_url` returns a checked-in synthetic fixture unless `MCP_WEB_LIVE_MODE` is truthy. **`MCP_WEB_LIVE_MODE` appears nowhere in `infra/` .** `mcpWebApp` 's `envVars` array contains exactly one entry, `MCP_WEB_ALLOWLIST` .

It *is* set on the live app — `.compound/learnings/architecture/L-0074.md` records it directly: *" `ca-mcp-web` 's `MCP_WEB_LIVE_MODE` flag was separately set"* (2026-08-02), and the same learning confirms the live `MCP_WEB_ALLOWLIST` had "already diverged from the code's own example.com-inclusive default." So the deployed app fetches real content because a human set an environment variable by hand, outside infrastructure-as-code.

Why that is not survivable:

1. ARM replaces the container's `env` list declaratively. A deploy sets mcp-web's environment to exactly `MCP_WEB_ALLOWLIST` — dropping `MCP_WEB_LIVE_MODE` .
2. `mcpDeployToken` defaults to `utcNow()` , so **every** deployment forces a fresh revision; the container restarts on the Bicep-declared env.
3. **Nine workflows** reference `main.bicep` , and at least two run `az deployment group create` against the whole template — `deploy-infra.yml` and `deploy-governance.yml` (commit `e148d18` documents exactly this blast radius, where a governance deploy rotated the live Postgres admin password).

What breaks, and how quietly. In fixture mode `fetch_url` ignores the URL entirely and returns the same 1-sentence placeholder for all four sources:

```
SYNTHETIC-TEST-DATA: this is a synthetic fixture response body for mcp-web's
fetch_url tool. It contains no real personal or client data (POPIA s72
fixture-mode default, AC-7).
```

`ingest_signals_handler` would fetch four URLs, receive four identical placeholders, hand them to function 09 as "retrieved evidence", and write the resulting hallucinated signals to the Vault with `evidence_grade` and `source_url` attached. The daily loop goes **green**. There is no failure, no alert, and no dead-letter.

Nothing would catch it. The only automated check on mcp-web's mode is `caj-mcp-smoke` , which asserts `source == "fixture"` — it **passes** in the broken state and would fail in the correct one. L-0074's fix (`force_fixture_mode()` , a request-scoped override) makes the smoke test deliberately mode-independent, so it now tells you nothing about ambient configuration either way.

Fix (~1 hour): add `{ name: 'MCP_WEB_LIVE_MODE', value: 'true' }` to `mcpWebApp` 's `envVars` in `main.bicep` . Then add the standing guard: a check that `ingest-signals` rejects a fetched body matching `^SYNTHETIC-TEST-DATA` , so fixture content can never be laundered into a Vault signal with a real `source_url` .

The replacement mechanism is now verified against Microsoft's own documentation, not just against my reading of the template. ARM's incremental mode is incremental *per resource*, not *per property*: a resource present in the template is applied as a full replacement, and *"properties that aren't included in the template are reset to the default values."* Because `env` is computed as `concat(envVars, keyVaultSecretEnv)` , a redeploy sets it to exactly that list and drops

anything set by hand. See `19-live-verification-log.md` P3 for the citation and its caveats.

Secondary consequence worth its own review. The redaction firewall's `public_source_content` exemption (`redaction.py` INCIDENT 2, round 15) was authorised on the stated grounds that this content is *"real bodies from fetch_sources.yaml's public news domains."* That justification is only true while live mode is on. In fixture mode the exemption is still applied — to a placeholder — which is harmless in itself, but it means **the exemption's premise is a deployment-state assumption, not a code invariant.** Worth re-reading with that in mind.

TD-01 · 20 of 23 function packages never execute · ~~S1~~ · ✅ RESOLVED 17 Aug 2026

Where: `services/orchestrator/orchestrator/dispatch.py` `DISPATCH_TABLE` (5 entries) vs `functions/` (23 packages) vs `loops/*.yaml` (~30 task_types).

> **Resolved.** `DISPATCH_TABLE` now covers **39 task_types**, and every task in every shipped loop resolves to a real handler except the 7 in `nightly-analytics-ingest-loop.yaml` , which are documentary by design (see `01` §3.5). Daily-signal-loop went from 17 no-op tasks of 23 to **zero**.
>
> The fix landed **exactly as the "Fix" line below proposed** — a registry-driven factory. `SCANNER_TASKS` (task_type → function_id, profile, agent_name) is expanded by `_make_scanner_handler` into `SCANNER_HANDLERS` and merged in as a **dict spread**: `DISPATCH_TABLE = { **SCANNER_HANDLERS, ... }` .
>
> **Trap for whoever audits this next:** grepping `DISPATCH_TABLE` for `"task-type":` misses all eleven factory-registered scanners and reports eleven false no-ops. Resolve `SCANNER_TASKS` before concluding anything is unwired. This exact mistake was made while re-verifying this entry.

Every task_type not in `DISPATCH_TABLE` hits `legacy_task_pass_through` , which transitions RUNNING → COMPLETED and does nothing. The entire weekly content studio (15 task_types) and the entire S10 intelligence fan-out (11 task_types) are inert. The DAG runs green and produces nothing.

Impact: the platform's advertised capability is ~8× larger than its delivered capability. Any demo of the weekly loop shows a fully-green run with zero output. **Root cause:** wiring a package requires three uncoordinated manual steps — add to `DISPATCH_TABLE` , add the path to the orchestrator Dockerfile's staging step, add the task_type to a loop YAML. **Fix:** extract a generic registry-driven handler (four of the five existing handlers already share the shape: read prompt → build user_content → gateway → parse JSON → write artefact → set_result_ref → advance). ~2 weeks.

TD-02 · Publisher's Vault record is an in-memory stub · S1

Where: `services/publisher/app/vault_adapter.py` — `StubVaultRecordingAdapter` appends to a Python list.

The docstring is candid: *"What 'publishing' means here is: record the publication in the Vault (Postgres) through this adapter"* — and the adapter does not do that. `governance.publish_attempts` records the attempt, but the Vault, which is the system of record, never learns that anything was published.

Impact: the governance chain has a hole at its last link. `record_publish` returns a `record_id` that references nothing. Any audit that starts from the Vault cannot find the publication. **Fix:** implement a real Vault write (a `gate_decisions` row and/or an `assets` state transition to `approved`). ~2 days.

TD-03 · Vault API has zero authentication · S1

Where: `services/vault/vault/main.py` and every router — no auth dependency anywhere. Documented in `docs/accepted-risks.md` , and *explicitly flagged as a builder judgement call, not a budget-owner-approved risk*.

Anything inside `cae-cmos-dev` can read, write or delete every business object, every consent record and every cost row. `X-Caller-Service` is self-asserted and explicitly not a trust boundary.

Impact: a single compromised container app or a mis-scoped future workload owns the entire system of record. Also a hard blocker for any enterprise security review. **Fix:** the accepted-risks doc already specifies it — a Key-Vault-sourced bearer token validated by a FastAPI dependency, ideally managed-identity service-to-service auth. ~1 week including infra wiring and smoke-test updates.

TD-04 · Console authenticates but does not authorise · S1

Where: `infra/modules/console/console-app.bicep` — `allowedApplications` is empty and no app-role or group claim is required.

Any user who can obtain a token for the console's App Registration reaches the kill switch, the cost ledger and Vault search. The only mitigation is a Portal setting ("Assignment required = Yes") a human must remember to apply.

Impact: the emergency stop is available to every authenticated tenant user. In an org of any size this fails a security review immediately. **Fix:** app-role claim in the `authConfig` `validation` block **and** a matching check in `require_principal` — defence in depth, matching the RISK-003 pattern the codebase already uses for authentication. ~2 days.

TD-05 · Single-tenant by construction · S1 (commercial)

Where: all five schemas. No `tenant_id` , `org_id` or `workspace_id` column exists anywhere in 27 tables.

Impact: every commercial model except "internal tool" or "one deployment per customer" is blocked. And the cost of retrofitting grows with every row written. **Fix:** a decision, not a patch. Three options costed in `07-operating-model.md` §D.3. **Make this decision before more production data accumulates.**

Priority 2 — Scale and credibility

TD-06 · Model-gateway cache is process-local · S2

Where: `services/model-gateway/caching.py` — module-level dicts, with the scope note *"Multi-replica / cross-process cache consistency is explicitly out of scope."*

`ca-model-gateway` autoscales. Two replicas serving the same `task_ref` produce two upstream calls, two sets of `costs` rows, two charges. The idempotency guarantee the module exists to provide **does not hold in production**.

Fix: move to Redis, or to a Postgres advisory-lock + `completions` table keyed on `task_ref`. ~3 days.

TD-07 · Handler retries are not idempotent · S2

Where: `worker.py::_retry_or_dead_letter` — admitted in its own docstring: *"a handler that partially wrote to Vault before failing is not guaranteed idempotent on retry (e.g. a duplicate signal/agent_run row is possible)."*

A handler that creates an `agent_run`, calls the gateway, and then fails on the Vault write will, on retry, create a second `agent_run` and incur a second model charge.

Fix: derive a deterministic idempotency key from `task_id` for `agent_run` creation (the `uuid5` decomposition already gives a stable seed), or make handlers resumable by checking for an existing `agent_run` first. ~1 week.

TD-08 · Kill switch duplicated across two services · S2

Where: `services/gatekeeper/app/kill_switch.py` and `services/publisher/app/kill_switch.py` — byte-similar files, kept honest by `test_kill_switch_parity.py` which loads *both files by path* and asserts identical behaviour across the scope matrix.

The parity test is genuinely clever and is the right mitigation for today. But the same pattern repeats elsewhere: `AGENT_NAME_LOOP_PROOF` duplicated with a cross-service equality test; `CANONICAL_JSON_SEPARATORS` and `parse_resource_claim` duplicated between `gatekeeper/app/tokens.py` and `publisher/app/verifier.py` with a comment *"Must stay byte-identical."*

Impact: three critical security behaviours are maintained in two places each. The tests catch divergence — but only for the cases they enumerate. **Fix:** a shared `services/governance-lib` package, adopted exactly the way `telemetry-lib` already is. `verifier.py`'s "standalone by design" constraint is about not importing `app.*` across services — a neutral shared package satisfies it. ~1 week.

TD-09 · The registry has no runtime role · S2

Where: `services/registry/` builds a signed, reproducible manifest. `dispatch.py` reads `prompt.md` straight off disk via `functions_dir()`. Nothing ever calls `verify_signature.py` at runtime.

Impact: the entire supply-chain-integrity story is theatre. A modified `prompt.md` in the container image runs happily. `registry_version` is a span attribute nothing authoritative populates. **Fix:** verify the manifest signature at orchestrator startup and resolve prompts *through* it. This also fixes TD-01. ~1 week (combined).

TD-10 · Console reads mock data for governance screens · S2

Where: `console/app/clients/gatekeeper_mock.py` is the default; `GATEKEEPER_API_MODE=mock` in `console-app.bicep`.

The approval inbox and kill-switch screens — the two most operationally important — display fixtures. `console/README.md` documents this precisely and corrects an earlier "config-only cutover" claim: Gatekeeper exposes no REST route over `kill_switches` or `approval_inbox`.

Impact: an operator looking at the kill-switch screen is not looking at production state. **Under an incident, this is dangerous. Fix:** add `GET/POST /kill-switch`, `GET /kill-switch/audit/last`, `GET /approval-inbox` to Gatekeeper's internal app; flip the env var. ~3 days.

TD-11 · Three of four analytics sources are fixtures · S2

Where: `analytics_ingest/{ga4,search_console,linkedin}_client.py` return bundled JSON fixtures. Only Buffer goes live, and only if `BUFFER_API_KEY` resolves.

Impact: every KPI except the Buffer slice is synthetic. `cost per accepted asset` is real (it reads the Vault) but engagement and reliability are not. Reporting on fixture data is worse than not reporting. **Fix:** real API clients + credentials. Note learning L-0074's warning about "goes live automatically once credentials exist" designs — the live path must be independently verified, not assumed. ~1 week per source.

TD-12 · Postgres is a Burstable B1ms with 50 max connections · S2

Where: `infra/modules/postgres.bicep` — `Standard_B1ms`, 32GB, single zone, no read replica, no HA.

The connection budget is already tight and has already caused an incident: `vault/db.py` documents reducing the pool from 20 to 12 because 3 replicas × 20 = 60 exceeded `max_connections=50` (PERF-2). Five schemas, six services, and every Container Apps Job share this one server.

Impact: a genuine scaling wall, and a single point of failure with no HA. **Fix:** General Purpose tier + HA + PgBouncer before any real load. ~2 days of infra, plus cost.

TD-13 · Dead-letter alerts go nowhere · S2

Where: `dead_letter.py::emit_alert` publishes a `DeadLetterAlert`. `worker.py` receives it, logs `dead_letter_alert_received`, and moves on — its own comment says *"informational only today — nothing in the worker loop consumes DeadLetterAlert yet."*

Impact: a permanently failed task is silent. No paging, no email, no Teams card, no console surface. The one place it would be visible is `/status`, which nobody watches at 06:00. **Fix:** route to the existing Teams webhook path, and/or an Azure Monitor alert rule on the log event. ~2 days.

> **Re-verified 17 Aug 2026 — worse than recorded.** There are **no alert rules anywhere in `infra/`**: no `metricAlerts`, no `scheduledQueryRules`, no action groups. So the "and/or an Azure Monitor alert rule" half of the fix has no foundation to build on, and this is not an isolated gap — nothing pages on anything. It compounds with two others: a failed worker still returns `/health` 200 (the worker is a single `asyncio.Task`; if its startup raised, `worker_task = None` and the app serves happily), and a missing `TEAMS_WEBHOOK_URL` / `DATABASE_URL` logs at WARNING and continues. Together these make *doing nothing while looking healthy* the platform's most likely failure mode, with no automated detector. Treat as one piece of work: a `/readiness` endpoint distinct from `/health`, explicit expected-integration env vars, and alert rules declared in Bicep.

TD-34 · `post_archetype` has no writer — the headline KPI groups on an empty column · S2

Where: `services/publisher/app/buffer_client.py::create_draft` sends exactly `channel_id` and `text`. `analytics.post_archetype` is *read* by `_render_month_end_report` and `db.py`'s engagement query, and written by nothing.

Impact: `kpi_rollup_engagement_by_archetype` — the platform's headline performance number — groups on a column the system never fills. It is also the last of three attribution join keys; the other two (`analytics.scheduled_posts` via `record_scheduled_post()`, and `analytics.utm_campaign_map`) were closed in August, so this single gap is what still prevents published output being tied back to the decision that produced it.

Constraint on the fix: AC-09 is a real safety invariant — `create_draft` must never accept a status/mode/state argument, and `mcp-buffer` is pytest-guarded against any tool name or description matching `publish|share.?now|send.?now|go.?live`. Add `utm_campaign` and `post_archetype` as **optional opaque labels** that cannot transition a post's state; resolve both from the `asset_id` Vault lookup `publish.py` already performs. Also add them to `ASSUMED_METRIC_FIELDS` so `buffer_introspect.py` verifies them live — expect that to reveal whether Buffer's schema actually has an `archetype` field, which is better learned at deploy time than after a quarter of NULL groups. ~2 days.

TD-35 · An unapproved QA policy gates production publishing · S2

Where: `functions/48-fact-check-verdict/prompt.md` opens with *"FIRST DRAFT — 6 Aug 2026. Not yet reviewed or approved by Pieter as settled QA policy."*

That prompt is the Thursday fact-check gate: it decides whether content reaches Buffer and the newsletter. It has been in the production critical path since 6 August. An over-strict verdict blocks good content; an under-strict one lets a fabricated number reach a client's inbox.

Fix: a real review, then delete the banner and date the approval — or gate the Thursday fact-check tasks off until that happens. **Do not delete the banner without the review**; it is currently the only thing signalling the risk. ~half a day of the owner's time, not an engineering task.

TD-36 · Daily cadence will breach Buffer's queue cap when publishing goes live · S2

Where: `weekly-content-loop.yaml` requests 4 Buffer posts per cycle (`friday-schedule-social-buffer-*` × 4); `la-weekly-planning-trigger` fires daily (deliberately — see `01` §3.5). ~28 queued posts/week against `BUFFER_FREE_TIER_QUEUE_CAP = 10`, enforced by a live `list_queue` count.

Impact: masked today, because `PUBLISHER_DRY_RUN` defaults true and is set nowhere in infra, so nothing is queued. It fails *safe* when it bites — a `buffer_queue_cap_exceeded` refusal row, not a crash — but it will begin refusing silently around day 3 of live publishing. **Fix:** a decision, not code: a paid Buffer tier, fewer posts per cycle, or accept the cap as a throttle. Record the choice next to `BUFFER_FREE_TIER_QUEUE_CAP` so it is not rediscovered live.

TD-37 · `mcp-canva` is deployed, credentialled, and called by nothing · S2

Where: `infra/main.bicep` declares `idMcpCanva`, `mcpCanvaKvRole`, `mcpCanvaAcrRole` and `mcpCanvaApp`, wiring `CANVA_CLIENT_ID` and `CANVA_CLIENT_SECRET`; `deploy-mcp.yml` builds and ships it. No caller exists anywhere outside `mcp/` itself. Function 45 still emits a `canva_bulk_create_csv` manifest into every carousel asset that no handler consumes.

Impact: standing compute cost plus a live third-party credential held by a service with no consumer — surface that exists only to be attacked. Distinct from ordinary dead code, which costs nothing at runtime. **Fix:** decide. Either wire `bulk_create_from_csv` into the carousel handler so function 45's manifest is used, or remove the four Bicep modules, drop it from `deploy-mcp.yml` and **revoke the Canva credentials**. Leaving it running is the only option with cost and risk but no benefit. ~1 day either way.

Open question this depends on: are those Canva credentials still live? Not determinable from the repository, and it decides whether this is a tidy-up or a credential-revocation task.

TD-32 · The brand rules have never been run against the brand's real output · S2

Where: `functions/02-brand-steward-qa/prompt.md` L40–44 (`link-shortener`), the `url-utm` and `sa-english-spelling` rules in the same file, and function 42's roof line. Measured against 100 real published posts pulled from the live Buffer account — see `19-live-verification-log.md` V2.

FN 02 RULE	RESULT AGAINST REAL OUTPUT
`link-shortener` — bans `bit.ly` , `lnkd.in` , `tinyurl.com` , `ow.ly` , `buff.ly`	**86 of 100 would FAIL** (85 `bit.ly` , 1 `lnkd.in`)
`url-utm` — Canvas URLs need 3 UTM params	12 posts carry a Canvas link, 4 carry any `utm_` → 8 fail
`sa-english-spelling`	`center` ×3, `behavior` ×4 → fails
fn 42 roof line `Your Data. Delivered.`	all 6 real occurrences read `Your data. Delivered.`

Impact: `qa_review_handler` 's `pass: false` is *terminal* — it transitions the task to `FAILED` with reason `qa_blocked` and never calls `advance_dependents` . A rule set this far from actual practice means that the day the platform is put in the publishing path, the overwhelming majority of drafts in its own house style die at the QA gate with no route past it. There is no override, by design.

Note that `buff.ly` — Buffer's own shortener, the one that would appear as a tooling artefact — occurs **zero** times. `bit.ly` is a deliberate, systematic editorial choice that the codified policy names as a blocking failure.

Fix: run `functions/02-brand-steward-qa/safety_suite.py` over an export of real published posts as a one-off calibration pass, then reconcile — either the rules move or the practice does. That is a decision for the CMO, not for engineering. No new code. ~1 day, and it is the cheapest de-risking available before TD-01 activates the agents. The roof-line casing is a one-character fix in whichever of the two places is wrong.

Priority 3 — Delivery friction

TD-14 · Contract freeze forces architectural workarounds · S3

The frozen `vault-schema/schema.sql` produced `vault_internal` — a 7-table sidecar schema stitched back on with a `LEFT JOIN` on every read. The frozen `gate-token/schema.json` (with `additionalProperties: false`) forced `function_id` and `content_hash` into a canonical-JSON string packed inside the `resource` claim, which then required byte-equality re-canonicalisation checks in **two** services.

Both workarounds are well-executed and well-documented. Both are debt: an extra join on every Vault read, and a parsing/serialisation contract that must stay byte-identical across two independently-maintained files.

Fix: a v2 contract window that consolidates `vault_internal` and promotes `function_id` / `content_hash` to first-class claims. The forward plan is already written into `contracts/gate-token/spec.md` . ~2 weeks + migration.

TD-15 · Migration script re-applies all four files on every deploy · S3

Where: `infra/main.bicep` 's `orchestratorMigrationSql` joins 0001–0004 into one `psql -f` with `ON_ERROR_STOP=1` , run unconditionally on every `deploy-infra` .

This caused a real production outage: migration 0003's `DROP + ADD CONSTRAINT` re-validated the whole live table on every deploy, and once rows with `dependency_dead_lettered` existed (added by 0004), 0003's older 9-value list failed — aborting the script *before* 0004 could run and supersede it. The fix (`ADD CONSTRAINT ... NOT VALID`) is correct, but the underlying design — no migration ledger, everything re-applied every time — remains.

Fix: adopt the `governance.schema_migrations` pattern the governance schema already uses, and apply only unapplied versions. ~2 days.

TD-16 · Duplicated Azure client code across services · S3

`resolve_live_fqdn` via `az containerapp show` is implemented independently in `orchestrator/clients/azure_fqdn.py` , `publisher/app/buffer_client.py` and `registry/gateway_client.py` . Same for HTTP client construction, traceparent injection and contract-shape validation.

The rationale (avoiding cross-service coupling) is sound. The cost is three copies of a subtle behaviour, only one of which has full test coverage.

TD-17 · `dispatch.py` is 7,068 lines and growing · S3 → S2

> **Re-measured 17 Aug 2026: 7,068 lines, up from the 1,138 recorded below — 6× in the interval, and +148 in a single day.** Raised to S2. It is the highest-change-rate file in the repo and every incident touches it. The "~3 days" estimate below was sized against 1,138 lines and no longer holds. It now also carries scoring, the eleven-scanner factory, dedupe, brief rollups, month-end reporting and the QA retry loop.
>
> The split must be a **pure move with no behaviour change in the same PR**, preserving every dated incident comment verbatim — as the original entry already argued, those narratives are the institutional memory.

Five handlers, lineage resolution, permission-check dynamic loading, redaction fallback, proof-circuit tagging, brief rendering, and the not-ready/cascade gate all in one module. Roughly 40% of its lines are comments — which is genuinely valuable (the incident narratives are the institutional memory) — but the module now has at least six responsibilities.

Fix: split into `dispatch/handlers/*.py` + `dispatch/lineage.py` + `dispatch/gating.py`, preserving every comment. ~3 days.

TD-18 · Registry CI covers 3 of 23 packages · S3

`registry.yml` hardcodes the paths for 02/09/42. Documented in `docs/function-register-coverage.md`. 20 packages have golden evals that CI never runs.

TD-19 · MCP test suite is not in CI · S3

`mcp/README.md` documents this as a known operational gap: none of the 10 pytest markers are wired into `ci.yml`, which does not touch `/mcp` at all. Only the `mcp_conformance` subset runs, once per deploy, in `caj-mcp-smoke`.

TD-20 · Hardcoded prices, channel ids and cadence · S3

`metering.PRICE_PER_MTOK` (silently drifts from actual billing); `BUFFER_LINKEDIN_CHANNEL_ID` in `publisher/app/config.py` (despite the weekly loop YAML carrying three channel ids); `ACCESS_LOG_RETENTION = 90 days`; `NOT_READY_MAX_REQUEUES = 20`. All of these belong in policy YAML given the codebase's own strong policy-as-data convention everywhere else.

Priority 4 — Hygiene

#	ITEM	WHERE
~~TD-22~~	✅ **RESOLVED 17 Aug 2026.** `month-end-reporting-loop.yaml` and `report_month_end_handler` now exist; the heartbeat lands on a real loop	`services/orchestrator/loops/month-end-reporting-loop.yaml`
TD-23	`TaskEnvelope.priority` in the frozen contract, never read	`contracts/service-bus/task-envelope.schema.json`
TD-24	`web_search` declared in fn 09's tools.yaml, not implemented	`mcp/mcp-web/app/tools.py`
TD-25	Registry signed with a committed dev key; Ed25519 unusable in Key Vault — **verified 2026-08-06**, and at *every* tier including premium and Managed HSM, not only standard, so no SKU upgrade lifts it. Supported curves are P-256/P-256K/P-384/P-521 only. The `accepted-risks.md` option (a) ES256 switch is the right path. See `19` P4.	`services/registry/keys/`, learning L-0031
TD-26	`redaction.py` docstring says "9 hash-guarded frozen contract files"; there are 10	`contracts/.frozen-v1.sha256`
TD-27	Level 2 autonomy behaves identically to level 1	`gatekeeper/app/routers/gate_check.py`
TD-28	`client_references` is always `[]` at the qa-review call site, so the deterministic uncleared-client check always passes trivially	`dispatch.py::qa_review_handler`
TD-29	`functions/task-worker/` is a health-check placeholder — **re-verified 17 Aug 2026: 23 lines, one route, and confirmed neither deployed nor built** (no reference in `infra/` or any workflow). Deletion is risk-free; the only cost of keeping it is that it implies an Azure Functions tier that does not exist	`function_app.py`
~~TD-30~~	✅ **RESOLVED 17 Aug 2026.** Reads are paged over `limit` / `offset` (API-capped at 500), and the console flipped from the mock to the real Vault *only after* that was true — `VAULT_API_MODE: 'real'`. The ordering mattered: flipping first would have reported "no costs" for any day past the first page	`console/app/services.py`; `console-app.bicep`; `console/tests/test_vault_reads_are_paged.py`
TD-33	`signing.py`'s module docstring predates the L-0031 correction: it gives only the *networking* reason the `keyvault://` path fails, and promises *"moving to a production signing key is a configuration swap, never a code change"* — which `docs/accepted-risks.md` now establishes is false for the recommended ES256 path. Blocks nothing; misdirects whoever picks up TD-25. Three-line fix.	`services/registry/signing.py` L7–11, vs `docs/accepted-risks.md` "Algorithm correction"

Security concerns, consolidated

#	CONCERN	SEVERITY	STATE
SEC-1	Vault API: no authn/authz	**Critical**	Accepted risk, *not budget-owner approved*
SEC-2	Console: authenticated but not authorised	**High**	Accepted risk; Portal-only mitigation
SEC-3	Service Bus: public endpoint (Standard SKU)	Medium	Accepted; `disableLocalAuth` + TLS 1.2 + metadata-only envelopes
SEC-4	Registry: committed signing key	Medium	Accepted; loud runtime warning, env-var-first resolution, no `alg:none` path
SEC-5	Redaction: regex coverage structurally incomplete	Medium	Acknowledged in the contract itself; every block audited *because* coverage is incomplete
SEC-6	Prompt injection via fetched news bodies	Medium	**Unmitigated** — no defence beyond schema constraints and a tiny domain allowlist
SEC-7	`X-Caller-Service` self-asserted	Low	Explicitly documented as not a trust boundary
SEC-8	POPIA s72 cross-border transfer unresolved for App Insights	Medium	Documented as open for legal review
SEC-9	Postgres admin password rotated on every governance deploy, leaving Key Vault stale	Medium	Fixed (commit `8277d38`); the class of bug remains — shared `main.bicep` deploys have wide blast radius

What is genuinely strong: zero client secrets across 13 workflows; managed identity for every data-plane access; private endpoints on Postgres, Key Vault and Storage; internal-only ingress on six of eight apps; RS256 with explicit algorithm pinning and `alg:none` rejection; durable replay ledger; append-only audit everywhere; and audit rows written on isolated connections so they survive the rollback of the transaction that triggered them.

Testing gaps

400 test functions across 128 files — genuinely substantial. What is **not** covered:

GAP	RISK
No load or performance test anywhere	Unknown behaviour at any scale; the B1ms wall is untested
No chaos test (Postgres down, Service Bus down, Anthropic 500)	Degradation paths are argued in comments, not proven
No end-to-end test of the *full* chain including a live publish	Dry-run is the only tested publish path
No test that a loop's task_types all have handlers	**Still missing, and still the highest-value gap.** TD-01 was closed by hand, so nothing stops it regressing — a new loop task with a typo'd `task_type` silently becomes a no-op and its loop still runs green
MCP markers absent from CI	10 markers, ~11 modules, run only locally
Registry CI covers 3 of 23 packages	20 golden eval sets never run in CI
No mutation testing	Coverage numbers unvalidated

The most valuable missing test: an assertion that every `task_type` appearing in any `loops/*.yaml` either has a `DISPATCH_TABLE` entry or is on an explicit allowlist of intentional pass-throughs. That one test converts TD-01 from invisible to loud, and would have surfaced it immediately.

> **Still true after TD-01 was resolved — arguably more so.** The wiring was fixed by hand; no test prevents it regressing. The allowlist is now a concrete, short list: the 7 documentary task_types in `nightly-analytics-ingest-loop.yaml`. Two implementation notes for whoever writes it: resolve `**SCANNER_HANDLERS` (a dict spread) rather than pattern-matching `DISPATCH_TABLE`'s literal keys, or the test will report eleven false failures; and assert the allowlist is *exhaustive*, so adding a new pass-through requires an explicit, reviewed edit.

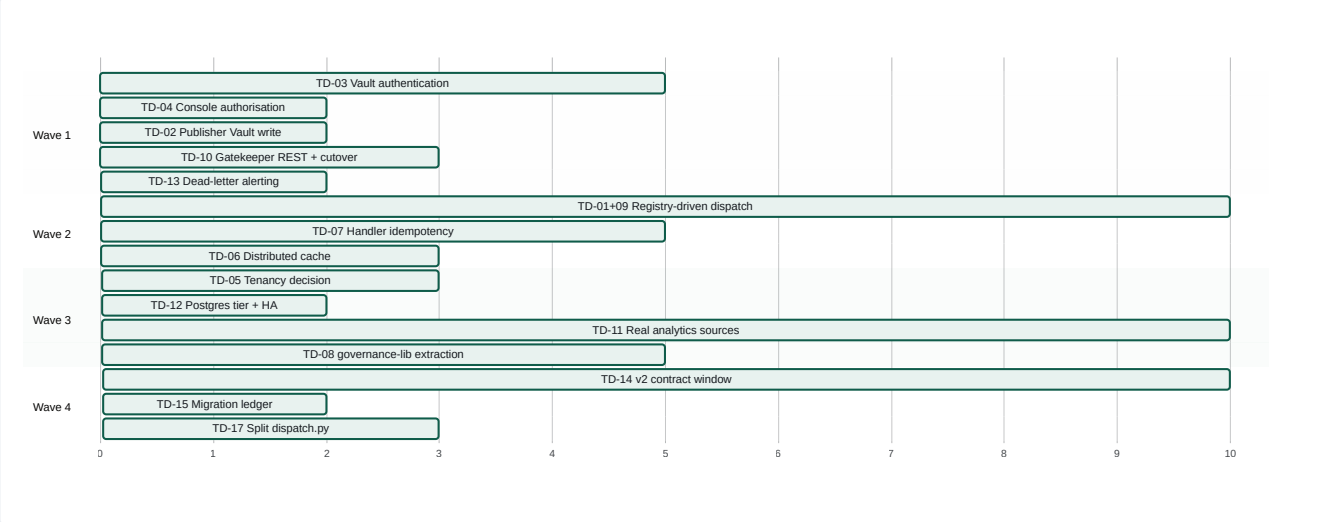
Documentation debt

Uniquely for a codebase of this size, documentation is a **strength**, not a gap. `docs/` holds 12 substantive documents; `.compound/` holds 79 learnings; module docstrings routinely run 40+ lines and carry root-cause narratives.

What is missing:

- **No single architecture overview existed before this document set.**
- No API reference generated from the OpenAPI specs (they exist; nothing renders them).
- No onboarding path — a new engineer must read `.compound/index.md` to understand why anything is the way it is.
- No incident runbook for AI-specific failures ("the agent published something wrong") — only deploy and auth runbooks.
- Several docs carry known-stale sections, flagged in-place (`credentials-runbook.md`'s secret names; `redaction.py`'s "9 files").

Remediation sequence (recommended)



Wave 1 — unblock (≈4 weeks) · Wave 2 — activate (≈6 weeks) · Wave 3 — scale (≈6 weeks) · Wave 4 — pay down (ongoing). Horizontal axis is working days.

Wave 1 is the one that matters. Five items, roughly four weeks, and they convert the platform from "impressive but unshippable to an enterprise" to "passes a security review". Everything else can wait.

10

Product Roadmap

Derived from the codebase's own gaps and its own latent capabilities. Every item is scored 1–5 on six axes. Nothing here is a generic "add more AI" suggestion — each item names the file it changes.

Scoring axes: **BV** business value · **EE** engineering effort (5 = largest) · **SD** strategic differentiation · **ER** enterprise readiness · **AL** AI leverage · **CI** customer impact.

Ranked backlog

#	ITEM	BV	EE	SD	ER	AL	CI	SCORE*
R1	Registry-driven generic dispatch (activate 20 packages)	5	3	4	3	5	5	**8.7**
R2	Close the learning loop (feedback service)	5	3	5	4	5	4	**8.7**
R3	Vault API authentication	4	2	1	5	1	2	**7.5**
R4	Gatekeeper governance REST API + console cutover	4	2	2	5	1	5	**8.5**
R5	Console role-based authorisation	3	1	1	5	1	2	**13.0**
R6	Real Publisher Vault write	4	1	2	5	1	2	**15.0**
R7	Dead-letter alerting	3	1	1	4	1	3	**13.0**
~~R8~~	~~Opportunity scoring (activate `opportunity_cards`)~~ — **shipped**, see below	5	3	4	2	5	5	**8.0**
R9	Multi-channel publishing (X, Facebook, email)	4	3	2	2	2	5	**5.3**
R10	Handler idempotency	3	3	1	4	1	2	**4.7**
R11	Distributed idempotency cache	3	2	1	4	2	2	**6.0**
R12	Multi-provider LLM support (OpenAI/Azure OpenAI/Gemini)	4	2	4	4	4	3	**9.5**
R13	Governance SDK extraction	5	4	5	5	4	3	**6.8**
R14	Multi-tenancy	5	5	2	5	1	5	**4.6**
R15	Editorial calendar & content plan UI	4	4	2	2	2	5	**3.8**
R16	Real analytics sources (GA4/GSC/LinkedIn)	4	3	1	3	2	4	**4.7**
R17	Prompt-injection defence on ingest	3	2	3	4	4	1	**7.5**
R18	v2 contract window	3	4	1	4	1	1	**2.5**
R19	Agent observability dashboard	3	2	2	3	3	4	**6.5**
R20	Compound-learning productisation	4	4	5	2	3	2	**4.0**

* Score = (BV + SD + ER + AL + CI) / EE

Horizon 1 — Next 90 days: *"make it shippable and make it produce"*

R6 · Real Publisher Vault write — 1 week

Replace `StubVaultRecordingAdapter` with an actual Vault call. Closes the last link in the governance chain. Highest score in the whole backlog because it is one file and it removes a hole in the platform's core claim.

R5 · Console role-based authorisation — 2 days

App-role claim in `console-app.bicep`'s `authConfig.validation` **plus** a matching check in `console/app/auth.py::require_principal` — defence in depth, exactly mirroring the RISK-003 pattern already used for authentication. Removes the "any tenant user can pull the emergency stop" finding.

R7 · Dead-letter alerting — 2 days

`worker.py` already receives and discriminates `DeadLetterAlert`. Route it to the existing `teams_notify` path and add an Azure Monitor alert rule on the log event. Two days to turn silent failure into a page.

R4 · Gatekeeper governance REST API + console cutover — 3 days

Add `GET/POST /kill-switch`, `GET /kill-switch/audit/last` and `GET /approval-inbox` to `ca-gatekeeper`'s internal app; flip `GATEKEEPER_API_MODE=real`. `console/README.md` already documents this as the precondition and the client already exists. Makes the two most operationally-critical screens show production state.

R3 · Vault API authentication — 1 week

Key-Vault-sourced bearer token validated by a FastAPI dependency, plus `secret-writer-job.bicep` provisioning, container env wiring, and updates to the smoke job and `test_contract_smoke.py`. The accepted-risks doc already specifies the design.

R1 · Registry-driven generic dispatch — 2–3 weeks

The single highest-impact change in the repository.

```
today:  DISPATCH_TABLE = {5 hardcoded handlers}
        prompt read via functions_dir()
        registry signature never verified

target: startup: verify registry.json signature → load manifest
        dispatch: task_type → manifest entry → generic_function_handler()
        generic_function_handler:
          prompt   = manifest.prompt (content-hash verified)
          input    = build from schema.json.input + lineage result_ref
          response = gateway.complete(tier from manifest)
          validate response against schema.json.output
          write artefact per manifest.artefact_type
          set_result_ref; advance_dependents
```

Four of the five existing handlers already have this exact shape. This one change activates 20 function packages, gives the registry a runtime role (closing TD-09), and makes adding a new agent a **pure content change** — no Python, no Dockerfile edit, no redeploy of the orchestrator.

Add the guard test at the same time: every `task_type` in any `loops/*.yaml` must have a manifest entry or be on an explicit intentional-pass-through allowlist.

Horizon 1 outcome: enterprise security review passes; the weekly content studio produces real content; the console shows real state; failures page someone.

Horizon 2 — 3 to 9 months: *"make it learn and make it sell"*

R2 · Close the learning loop — 3–4 weeks

The platform's largest conceptual gap. A `services/feedback` service that reads what is already written and emits **pull requests against policy files**:

READS	PROPOSES
`approval_actions` + `gate_decisions`	`autonomy.yaml` diff — promote a `(function_id, action_class)` from level 1 to 3 after N consecutive clean approvals
`agent_runs.output.violations[]`	Per-function QA violation report → prompt improvement backlog
`kpi_rollup_cost_per_accepted_asset`	`routing.yaml` diff — retier a function whose cheaper tier has equal acceptance
`kpi_rollup_engagement_by_archetype`	Content-plan input for the Monday planning node

Critical design constraint: every output is a *diff to a versioned file that a human merges*. No autonomy is ever granted implicitly. This preserves the platform's core property — that the AI's mandate is always an auditable artefact — while making the mandate self-tuning.

This is the feature that turns a pipeline into an operating system, and no competitor in the marketing-AI space has it.

R12 · Multi-provider LLM support — 2 weeks

The extension point is already built (`providers/base.py` is a one-method Protocol; `registry.py` is a name→class map; `routing.yaml` names the provider as a string; no vendor name appears in `config.py`, `completion.py` or `routing.py`). Adding Azure OpenAI is one module plus one `register()` plus a YAML edit.

Two payoffs: (a) **Azure OpenAI in `southafricanorth` removes the POPIA s72 cross-border question entirely** for functions that need it — which is a sales unlock, not an engineering nicety; (b) it proves the portability claim a buyer will test.

~~R8 · Opportunity scoring~~ — SHIPPED

`score-signals` writes a ranked `opportunity_cards` row per signal; the brief leads with the best-evidenced ones; the weekly content loop picks its pillar from those cards instead of an ISO-week rotation. `06-business- architecture.md` §5's red node is green.

Two pieces of the original scope are deliberately **configuration rather than code**, and both are unset:

- **"feed only the top-N into `draft-brief`"** exists as `selection.top_n` / `selection.minimum_score` in `functions/_shared/scoring-policy.yaml`, with no value set — so scoring currently *reorders* the brief rather than filtering it. Setting one is a reviewed YAML edit; the brief then states in its own body how many signals were held back.
- **The scoring rule itself** is function 09's `confidence` weighted by pillar. Recency decay and cross-source corroboration are absent because neither can be computed honestly from what a signal carries today (every signal in a batch shares one `received_at`; function 09 emits one item per story, not one per source). Adding either means changing what a scan *records* first — that is the real remaining work, and it is a scan change, not a scoring change.

R9 · Multi-channel publishing — 3 weeks

`weekly-content-loop.yaml` already carries three Buffer channel ids; `publisher/app/config.py` hardcodes only LinkedIn. Add the other two, plus an email path for the newsletter (`publish-newsletter` currently reuses `publish.blog_article` because *"no email-specific gate-check identifier exists"*).

R16 · Real analytics sources — 3 weeks

GA4, Search Console and LinkedIn clients. Heed learning L-0074: a "goes-live-when-credentials-exist" design must be *independently verified live*, not assumed.

R17 · Prompt-injection defence — 2 weeks

The only unmitigated attack surface. Fetched news bodies enter the `user` role with no instruction-injection detection. Mitigations available: strict output-schema validation (partially present), a delimiter/quoting convention in the prompt, and a cheap Haiku pre-classifier over fetched content.

R11 · Distributed idempotency cache — 3 days

Redis or a `completions` table keyed on `task_ref`. Makes the double-spend guarantee actually hold across replicas.

R19 · Agent observability dashboard — 2 weeks

The data exists (App Insights spans with 5 mandatory attributes, structured JSON logs, `task_transitions`, `costs`). Nothing visualises agent behaviour over time: success rate per function, cost trend, QA violation frequency, approval latency, dead-letter rate.

Horizon 3 — 9 to 24 months: *"decide what company this is"*

Horizon 3 is a **fork**, not a list. The three branches are mutually compatible in code but not in go-to-market focus.

Branch A — Deepen the vertical (marketing product)

R15 editorial calendar UI · richer content types (video briefs, ad copy, landing pages) · CRM/MAP integration (HubSpot, Dynamics) · a real DAM interface over the content-addressed blob store · A/B testing wired to the engagement rollup. **Comparable:** Jasper, Writer, Typeface. **Multiple:** 6–10× ARR. **Risk:** crowded, well-funded, and the governance moat may not be priced.

Branch B — Extract the horizontal (R13 · governance SDK) ★

The seam is already clean: `gatekeeper`, `model-gateway`, `publisher`, `vault`, `registry` and `telemetry-lib` contain **zero marketing logic**.

```
canvas-agent-governance/
  policy/        autonomy levels, fail-closed defaults
  gateway/       routing, budgets, redaction, metering
  approval/      inbox, single-use links, identity binding
  authorisation/ signed action tokens, hash binding, replay ledger
  vault/         taxonomy, consent, retention, audit
  registry/      prompt/agent versioning, signing, evals
  telemetry/     closed-enum PII-safe spans
```

Ship as: a Python SDK + a deployable control plane + MCP-native integration so any agent framework (LangChain, CrewAI, Claude Agent SDK, AutoGen) can route its tool calls through the gate.

The pitch: *"Your agents already work. Now prove what they were allowed to do, what they did, what it cost, and who said yes."* **Comparable:** Credo AI, LLMOps platforms, "Vanta for AI agents". **Multiple:** 15–25× ARR. **This is the branch the code most supports.**

Branch C — Productise the method (R20)

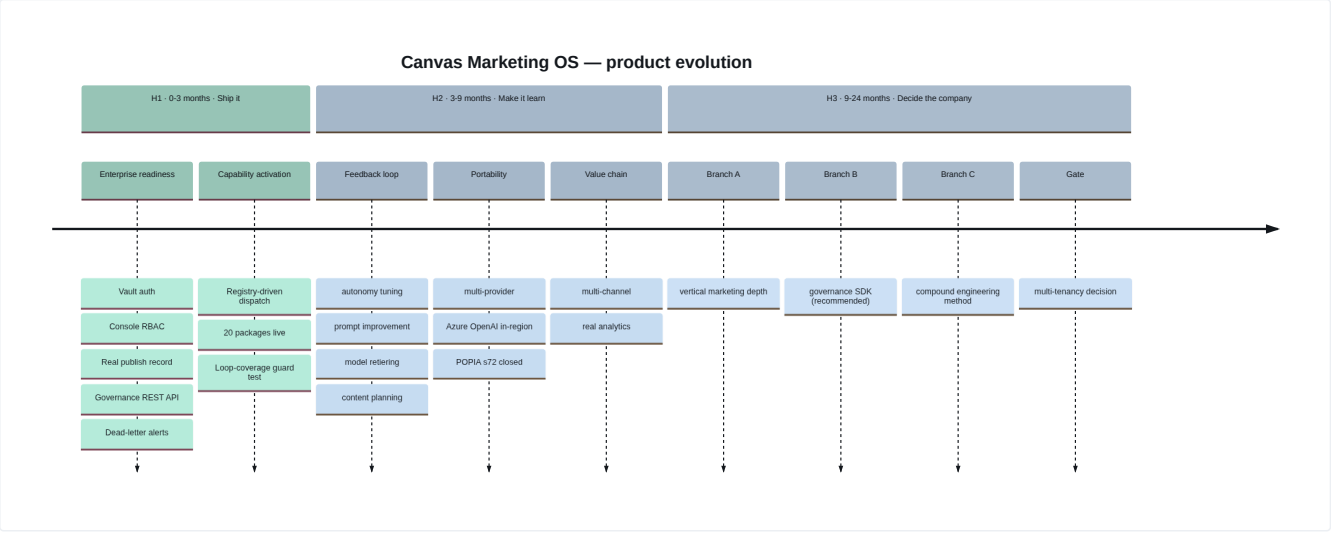
The 79 compound learnings, the worktree-per-session model, the spec→plan→build→multi-lens-review loop, and a live production system built by it. Sell the *method* — as a consulting practice, a licensed framework, or an agent-loop product — with Canvas Marketing OS as the reference implementation.

Comparable: none clean. **Risk:** requires extracting the method from this one repository to be credible, and the method's evidence is currently inseparable from the product.

R14 · Multi-tenancy — the gate on all three branches

Branch A and Branch B both require it. Branch C does not. Three options are costed in `07-operating-model.md` §D.3. **Make this decision in Horizon 2, not Horizon 3** — the retrofit cost grows with every row written.

The three-slide roadmap



What NOT to build

Equally important, and each backed by something the code already establishes:

DON'T BUILD	WHY
A conversational agent UI	The whole architecture is deterministic, reproducible, auditable DAGs. Chat destroys reproducibility and every governance property that depends on it
Autonomous publishing without human approval	The approval gate *is* the product. Removing it removes the differentiator
A vector store / RAG layer, by default	Nothing currently needs semantic retrieval. Prompts are static; context is DAG-lineage-resolved. Add it only when a use case demands it, not for completeness
More function packages	20 already exist and don't run. Activation beats creation until R1 ships
A custom LLM / fine-tuning	`routing.yaml` deliberately picks *conservative established snapshots over the newest generation*. Model choice is a data change; owning a model is a company change
Your own agent framework	MCP is already adopted. The differentiation is governance, not orchestration primitives
A general-purpose workflow engine	Temporal and Dagster exist and are better at it. The loop DAG is deliberately minimal and that minimality is what makes it verifiable

11

API Catalogue

Every HTTP surface in the platform. Auth column reflects what the code actually enforces, not what it should enforce.

1. Surface summary

SERVICE	APP	INGRESS	AUTH ENFORCED	ROUTES
Orchestrator	`ca-orchestrator`	internal	**none**	3
Model Gateway	`ca-model-gateway`	internal	**none**	2
Gatekeeper (internal)	`ca-gatekeeper`	internal	**none**	4
Gatekeeper (approval)	`ca-gatekeeper-approval`	**external**	Entra Easy Auth (Return401)	2
Publisher	`ca-publisher`	internal	**none**	3
Vault	`ca-vault`	internal	**none**	40
Console	`ca-console`	**external**	Entra Easy Auth + code backstop	9
mcp-web / mcp-buffer / mcp-canva	internal	**none**	2 each	

Total: ~69 routes. Two are reachable from the internet. Both are Entra-protected.

2. Orchestrator — `services/orchestrator/main.py`

METHOD	PATH	PURPOSE	RESPONSE
GET	`/health`	Static liveness. **No DB access** — must return 2xx before the DB is reachable (AC-017)	`{"status":"ok"}`
GET	`/status`	Every in-flight task with full state history. Synchronous DB read, **no cache**. Degrades to `[]` if the DB is unreachable	`[{task_id, task_type, state, retry_count, transitions[...]}]`
GET	`/runs/{task_ref}`	Agent-native run state: every stage via `depends_on` lineage, span presence, and the **real** Gatekeeper approval status	`{task_ref, stage_count, stages[], span_presence, approval_decision_status}`

`GET /runs/{task_ref}` returns **503** (not 500) if the lookup fails, and **404** for an unknown `task_ref`. `span_presence` is one of `present | absent | not_checked` — never an error.

3. Model Gateway — frozen contract `contracts/model-gateway/openapi.yaml`

METHOD	PATH	PURPOSE
POST	`/v1/completions`	Provider-agnostic completion with routing, redaction, budget, caching, metering
GET	`/v1/health`	Liveness

Request (`CompletionRequest`)

```
{
  "model": "claude-sonnet",             // required — LOGICAL id from routing.yaml, never a provider model id
  "messages": [                         // required, minItems 1
    {"role": "system|user|assistant|tool", "content": "string"}
  ],
  "agent_run_id": "uuid",               // required — FK to Vault agent_runs.id, drives cost attribution AND budget
  "max_tokens": 1024,                   // default 1024
  "temperature": 0.7,                   // default 0.7
  "tools": [ {...} ],                   // free-form passthrough — IS redaction-scanned
  // --- additive fields, not in the frozen v1 schema (which sets no additionalProperties:false) ---
  "task_ref": "string",                 // idempotency key: one compute() per task_ref
  "deliberate": false,                  // reasoning hint, feature-flagged → 400 NOT_IMPLEMENTED while disabled
  "content_class": "public_source_content" // maps to a reviewed pattern-exemption allowlist
}
```

Responses

CODE	BODY	WHEN
200	`CompletionResponse` + additive `routing_tier` , `budget_state` , `cache_hit` , `cost_id`	success
400	`INVALID_REQUEST`	schema violation — **message never contains a submitted value**
400	`UNKNOWN_MODEL`	model not in `routing.yaml` — message built from config-side facts only
400	`NOT_IMPLEMENTED`	`deliberate: true` while the flag is off
400	`REDACTION_BLOCKED` + `routing_tier` , `redaction_outcome`	firewall hit; `gate_decisions` row written; provider never called
429	`BUDGET_EXHAUSTED` + `queued_task_ref`	hard breach; the value is the escalated `gate_decisions.id`
500	`PROVIDER_ERROR`	upstream non-2xx
500	`INTERNAL_ERROR`	anything else — **fixed literal message, never** `str(exc)`

Three data-leak defences worth naming, each with its own finding code: `DR-3` — schema violations report the JSON path and the failed keyword, never the instance. `DR-4` — matched-pattern ids are contract-side coordinates (`fixture:client_names:0`), never the matched text. `DR-5` — the unknown-model message is built from `routing.yaml` 's own contents, never the submitted `model` string, because routing runs before the firewall and writes no audit row.

---

4. Gatekeeper — internal (`ca-gatekeeper`)

METHOD	PATH	PURPOSE
POST	`/gate-check`	Evaluate autonomy policy, write exactly one `gate_decisions` row, optionally raise an approval, optionally issue a gate token
GET	`/decisions/{decision_id}`	Agent-native read-back of a decision (AC-17)
GET	`/approval-status`	The **real** pending/approved/rejected/expired state, from `approval_inbox`
GET	`/healthz`	Liveness

POST /gate-check

```
// request
{ "agent_run_id": "uuid",       // required, must parse as uuid → else 400
  "function_id": "publish.social_post",
  "action_class": "publish",
  "content_hash": "sha256 hex",
  "preview_title": "[LOOP-PROOF] publish.social_post (publish)",
  "preview_reference": "loop-proof://<task_id>",
  "evidence_summary": "...",
  "subject": "optional JWT sub, defaults to agent_run_id" }

// response
{ "decision_id": "uuid", "agent_run_id": "uuid",
  "outcome": "approved|rejected|escalated",
  "reason": "level_0_blocked|level_1_requires_approval|level_2_requires_approval|
            level_1_approved_by_human|level_2_approved_by_human|
            level_3_auto_approved|level_4_autonomous_passthrough|
            kill_switch_active:<scope>[:fn]",
  "level": 0-4, "function_id": "...", "action_class": "...",
  "gate_token": "<JWT, only when outcome=approved>",
  "approval_route": "teams|inbox",
  "approval_id": "uuid", "approve_url": "...", "reject_url": "...",
  "approval_expires_at": "iso8601" }
```

Decision order is a security control: (1) kill switch — uncached live SELECT, before anything else; (2) level 0; (3) levels 1–2 → look for a prior approval matching `(agent_run_id, function_id, content_hash)`, else escalate, create the link and dispatch the card; (4) levels 3–4 → auto-approve. A gate token is issued **only** on an `approved` outcome.

GET /approval-status?agent_run_id=&function_id=&content_hash=

Exists because a `gate_decisions` row's `outcome` is `escalated` the instant the approval is raised and **never itself becomes approved** — a new row is only inserted on the *next* `/gate-check`. This endpoint reads `approval_inbox`, which *is* mutated in place by the click. Returns `{"status": "not_found"}` rather than 404 when no inbox row exists.

---

5. Gatekeeper — approval (`ca-gatekeeper-approval`, EXTERNAL)

METHOD	PATH	AUTH
GET	`/approval-action/{link_token}?choice=approve|reject`	**Entra Easy Auth, Return401**
GET	`/healthz`	none

STATUS	OUTCOME	AUDIT ROW
200	`approved` / `rejected`	`approval_actions` + new `gate_decisions` row
401	no authenticated principal	none (rejected before token lookup)
404	unknown token	none — *"disclosing more would turn this into a token oracle"*
409	`link_already_used`	`approval_actions`
410	`link_expired`	`approval_actions` + `approval_inbox.status='expired'`

---

6. Publisher — `ca-publisher`

METHOD	PATH	PURPOSE
POST	`/publish`	Verify a gate token and publish, or refuse with a recorded reason
GET	`/publish-attempts/{id}`	Agent-native read-back of any attempt, including every refusal
GET	`/healthz`	Liveness

```
// request
{ "agent_run_id": "uuid",
  "function_id": "publish.social_post",
  "asset_bytes_b64": "<base64 of the EXACT bytes>",  // hash recomputed from these
  "gate_token": "<JWT>",
  "asset_id": "uuid" }                          // optional; if present, lookup failure FAILS CLOSED
```

403 on every refusal, with the full `PublishResponse` in `detail`. See `02-module-catalogue.md` §M4 for the complete 12-branch refusal matrix and the security rationale for its ordering.

---

7. Vault — `contracts/vault-api.yaml` (OpenAPI 3.1, 1,623 lines)

The generic object surface — 9 types × 4 verbs

PATH SEGMENT	OBJECT TYPE	PATCH	DELETE
`/signals`	signals	✅	✅
`/campaigns`	campaigns	✅	✅
`/opportunity-cards`	opportunity_cards	✅	✅
`/briefs`	briefs	✅	✅
`/agent-runs`	agent_runs	✅	✅
`/assets`	assets	✅	✅
`/gate-decisions`	gate_decisions	❌ **append-only**	❌
`/costs`	costs	❌ **append-only**	❌
`/consent-register`	consent_register	✅ *revoke only*	❌

`POST /{type}` · `GET /{type}?limit&offset` · `GET /{type}/{id}` · `PATCH /{type}/{id}` · `DELETE /{type}/{id}`

Every create requires all six taxonomy fields:

```
{ "vertical": "mobility", "function_id": "signal-ingestion-v1",
  "campaign": "uuid",                       // FK to campaigns
  "evidence_grade": "A|B|C|D|unverified",
  "consent_status": "granted|revoked|not_required|pending",
  "retention_class": "ephemeral_30d|standard_1y|extended_3y|legal_hold",
  // + client-derived fields, which trigger the consent gate:
  "data_subject_ref": "...", "consent_channel": "...", "consent_purpose": "..." }
```

CODE	MEANING
201	created
403	`consent_required` — client-derived write with no active consent (+ audit row)
422	`taxonomy_field_missing` / `taxonomy_field_invalid` / `taxonomy_field_immutable` / `fk_violation` / `field_missing` (+ audit row)
404	`not_found`

Error bodies are `{"error": {"message", "code", "field?"}}`. Note that `main.py` installs a custom `HTTPException` handler because FastAPI's default re-wraps `exc.detail` under a further `"detail"` key — which meant *every* taxonomy/consent/not-found rejection this service ever returned had no top-level `error` key at all, contradicting the contract.

`/assets` is special-cased: `content_base64` on create (→ content-addressed blob, `content_hash` + `storage_uri` filled server-side) and on read.

Governance and reporting routes

METHOD	PATH	PURPOSE
GET	`/consent/check?data_subject_ref=&channel=&purpose=`	`{consented, consent_register_id}`
POST	`/retention-expiry-runs`	Runs the sweep inline, returns the run row (202)
GET	`/retention-expiry-runs` · `/{id}`	Run history
POST	`/utilisation/rollup`	Recompute a day's rollup
GET	`/utilisation/rollup?from=&to=&object_class=`	`[{date, object_class, caller_service, read_count}]`
GET	`/health`	Static — **no DB round-trip**

`X-Caller-Service` on any object GET feeds `access_log` → `utilisation_daily`. It is **informational only and explicitly not a trust boundary**.

---

8. Console — `ca-console` (EXTERNAL)

All routes require an Easy-Auth principal (401 otherwise), enforced both by the ingress and by a `require_principal` `Depends`. Every GET honours `Accept: application/json` and returns the same data the HTML renders.

METHOD	PATH	QUERY PARAMS
GET	`/`	→ 307 `/tasks`
GET	`/tasks`	—
GET	`/tasks/{task_ref}/trace`	`task_ref` must match `^[A-Za-z0-9][A-Za-z0-9_-]{0,127}$` else **400**
GET	`/approvals`	—
GET	`/vault-search`	`object_type`, `vertical`, `function_id`, `campaign`, `evidence_grade`, `consent_status`, `retention_class`
GET	`/costs`	`group_by=function|day`, `date`
GET	`/kill-switch`	—
POST	`/kill-switch/toggle`	JSON or form-urlencoded; form path adds an Origin/Referer same-origin check and returns 303
GET	`/health`	unauthenticated

---

9. MCP servers — one protocol, three servers

`POST /mcp` (JSON-RPC 2.0) + `GET /health`.

METHOD	PARAMS	RETURNS
`initialize`	—	protocol version + server info
`tools/list`	—	tool definitions **including the authoritative** `inputSchema` (which lives in `app/main.py`'s `TOOLS` list, never in `tools.yaml`)
`tools/call`	`{name, arguments}`	`{content[], structuredContent{}}`

SERVER	TOOLS	BEHAVIOUR
mcp-web	`fetch_url(url)`	Allowlist checked **before** any network call; sliding-window rate limit; fixture unless `MCP_WEB_LIVE_MODE`
mcp-buffer	`list_queue(channel_id)`, `get_post(post_id)`, `create_draft(channel_id, text)`	`create_draft` takes **exactly two arguments**; status hardcoded server-side
mcp-canva	`create_design_from_template(template_id, ...)`, `bulk_create_from_csv(template_id, ...)`, `export_design(design_id, format)`	`template_id` required on both creation tools

Every call is best-effort logged to `mcp_ops.tool_calls`; a Postgres outage never fails a tool call.

---

10. Cross-cutting API conventions

CONVENTION	APPLIED WHERE
`/health` or `/healthz` is dependency-free	every service — must return 2xx before the DB is reachable
W3C `traceparent` injected outbound, joined inbound	all orchestrator clients + every service's `telemetry_wiring.py`
`X-Correlation-Id` returned	Vault (one JSON log line per request)
Error bodies never echo submitted values	model-gateway (DR-3/4/5), routing, redaction
Errors carry a machine-readable `code`	model-gateway, Vault, Publisher, Gatekeeper
Never hardcode a service FQDN	every client — `az containerapp show` or an env override (L-0025)
Agent-native read-back for every write	`/decisions/{id}` , `/publish-attempts/{id}` , `/approval-status` , `/runs/{task_ref}`

---

11. API gaps

GAP	CONSEQUENCE
No authentication on 5 of 8 services	Network isolation is the only control
No REST over `kill_switches` / `approval_inbox`	The console cannot show production governance state (TD-10)
No server-side filtering on Vault list endpoints	The console fetches everything and filters in Python
No pagination metadata	`limit` / `offset` with no total count
No bulk endpoints	1,000 objects = 1,000 round trips
No webhooks / subscriptions	Consumers must poll
No API versioning outside model-gateway	Only `/v1/completions` is versioned in its path
No rate limiting except mcp-web	Any internal caller can saturate any service
OpenAPI specs exist but are never rendered	No developer portal, no generated client

12

Integration Catalogue

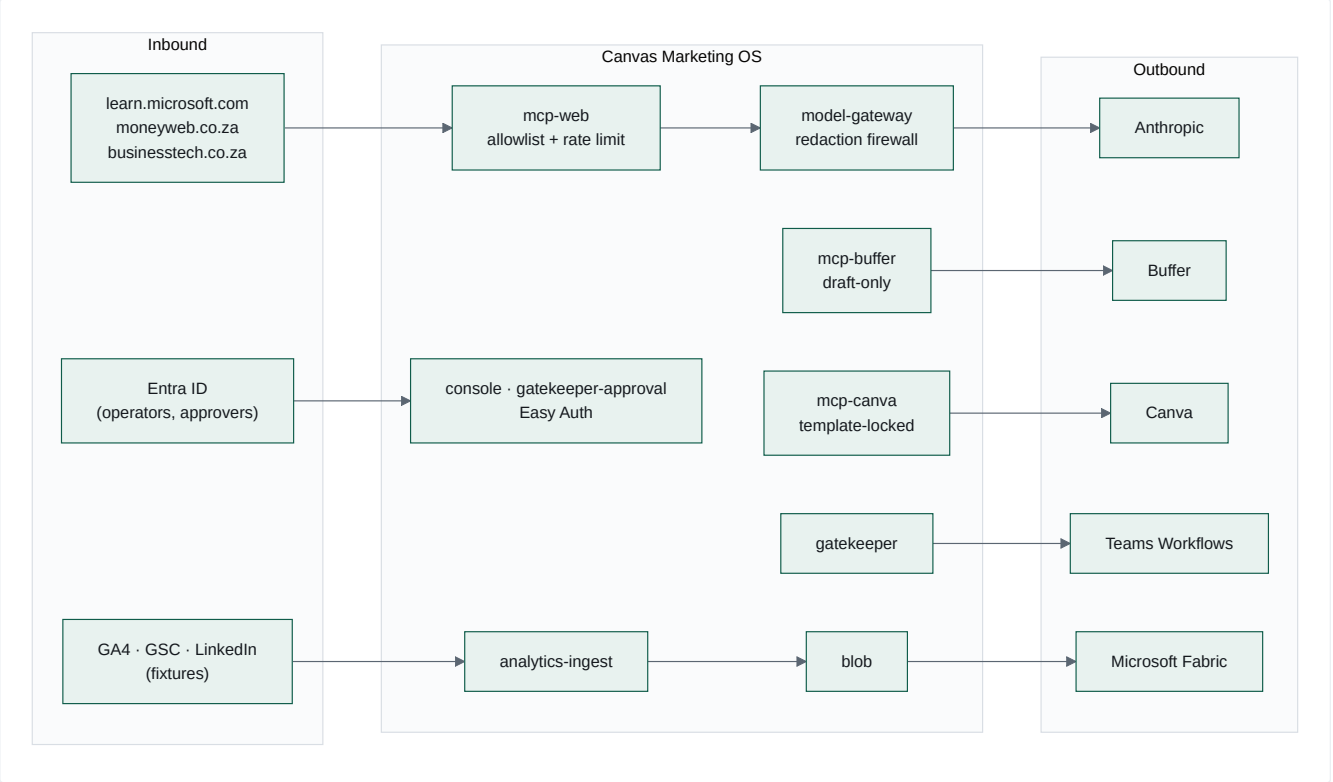
Every external system the platform touches, how it authenticates, what data crosses the boundary, and what happens when it fails.

1. Integration inventory

#	SYSTEM	DIRECTION	VIA	AUTH	MODE	MATURITY
I1	**Anthropic Messages API**	outbound	`providers/anthropic.py` (httpx, no SDK)	`x-api-key` from Key Vault → Container Apps secretRef	**live**	L4
I2	**Buffer**	outbound	mcp-buffer → GraphQL	`BUFFER_API_KEY` from Key Vault	**dual** — live if key resolves	L3
I3	**Canva**	outbound	mcp-canva → REST	OAuth2 + PKCE, client id/secret from Key Vault	**fixture** — no refresh token exists	L2
I4	**Public web** (3 domains)	inbound	mcp-web `fetch_url`	none (public)	dual, allowlisted	L4
I5	**Microsoft Teams**	outbound	Power Automate Workflows HTTP trigger	webhook URL from Key Vault	**flag-gated off**	L3
I6	**Microsoft Entra ID**	inbound	Container Apps Easy Auth	OIDC + Federated Identity Credential	**live**	L4
I7	**Azure Key Vault**	outbound	native Container Apps secretRef + SDK fallback	managed identity	live	L4
I8	**Azure Blob Storage**	both	`azure-storage-blob`	managed identity	live	L4
I9	**Azure Service Bus**	both	`azure-servicebus`	managed identity (`disableLocalAuth`)	live	L4
I10	**Application Insights / Log Analytics**	outbound	OpenTelemetry + `azure-monitor-query`	connection string / managed identity	live	L4
I11	**Microsoft Fabric**	outbound	blob shortcut	(Fabric-side)	**export written; shortcut not provisioned**	L2
I12	**Power BI**	outbound	starter dataset definition	—	**definition only**	L1
I13	**GitHub Actions → Azure**	outbound	OIDC federated identity	no client secret	live	L4
I14	**Azure Container Registry**	both	managed identity `AcrPull` ; OIDC push	admin user **disabled**	live	L4
I15	**GA4**	inbound	`ga4_client.py`	—	**fixture only**	L1
I16	**Google Search Console**	inbound	`search_console_client.py`	—	**fixture only**	L1
I17	**LinkedIn API**	inbound	`linkedin_client.py`	—	**fixture only**	L1
I18	**Fireflies**	—	referenced by fn 26's skill.md	—	**not implemented**	L0

Live external integrations: three (Anthropic, the public web, and Entra). Everything else is Azure-internal, fixture-backed, or flag-gated off.

2. Integration topology



Every outbound integration passes through a chokepoint that enforces a guardrail. There is no direct vendor SDK call from any handler.

---

3. I1 · Anthropic — the only live AI provider

Adapter: `services/model-gateway/providers/anthropic.py`, ~160 lines, raw httpx. The rationale is stated: httpx is already a dependency, exactly one endpoint is called, an SDK "would not earn its keep."

Endpoints: `POST /v1/messages`, `GET /v1/models` (startup validation). **Version header:** `anthropic-version: 2023-06-01`. **Timeout:** 60s.

Models (`policy/routing.yaml`):

LOGICAL ID	TIER	PROVIDER MODEL
`claude-opus`	opus	`claude-opus-4-8`
`claude-sonnet`	sonnet	`claude-sonnet-4-6`
`claude-haiku`	haiku	`claude-haiku-4-5-20251001`

The file's header is unusually rigorous and worth quoting: every original id (`claude-opus-4-20250514`, `claude-sonnet-4-20250514`, `claude-3-5-haiku-20241022`) was **absent** from a live `/v1/models` call — retired, not merely stale. The current ids were proven live on 2026-07-30 against the real account key. And the choice is **deliberately conservative**: established 4.x snapshots rather than the newest 5-generation, because the 5-generation "has not yet been evaluated against this gateway's routing tiers/budgets."

The `_split_system_prompt` fix (`F-GATEWAY-SYSTEM-ROLE`, round 16) is instructive: every caller builds `[{role:"system"},{role:"user"}]` — the provider-agnostic shape the frozen contract expects. Anthropic **rejects** `role: "system"` inside `messages` with an HTTP 400. The adapter is the one place responsible for translating, exactly as its docstring promises. It was discovered live because every unit test used a stub `Provider`, so **nothing had ever asserted the real HTTP body shape end to end.**

Data crossing the boundary: system prompts (static, from git), user content (structured JSON built by handlers — brief bodies, draft text, fetched news article bodies). **Everything is redaction-scanned first except the system role**, for the reasons documented at length in `redaction.py`.

Failure: `raise_for_status()` → `httpx.HTTPStatusError` → a `500 PROVIDER_ERROR` with the upstream status logged server-side. No retry at the gateway layer; the orchestrator's own state machine handles it.

Cross-border note: this is a **US-hosted inference provider**. The redaction firewall exists precisely because of that, and `contracts/model-gateway/redaction-rules.yaml` 's own header is explicit that it *"is not itself a transfer-lawfulness mechanism and does not establish a ground for sending personal information to a US-hosted inference provider. Treat it as one control among several, never as the control."*

---

4. I2 · Buffer — social scheduling

Two independent clients, deliberately (the services share no library): `mcp/mcp-buffer/app/dispatch.py` (GraphQL) and `services/publisher/app/buffer_client.py` (MCP-over-HTTP to mcp-buffer).

Three tools only. `create_draft` sends exactly `{channel_id, text}` — "the ONLY 2 arguments ever sent — no status/mode/state (AC-09)". mcp-buffer hardcodes `_CREATE_DRAFT_STATUS = "draft"` server-side and accepts no override.

Channels (`publisher/app/config.py` , all three mapped so a known transposition error in the source GOAL text can never recur):

```
LinkedIn 68e73facca3a4e6b746d17b4   ← the only one Publisher uses
Facebook 68e74731ca3a4e6b746d2469
X        68e745c6ca3a4e6b746d22b2
org      68e5f2187fe9a5263a3509ab
```

Free-tier cap: 10 queued posts — verified against the live account on 2026-08-06 (`19-live-verification-log.md` B2; the code flags it as an unverifiable assumption, which it no longer is). Enforced as a **live** **`list_queue`** **count check** before every create, not as a static assumption — because "other actors/tools could also add to the same queue between runs". The weekly loop independently caps itself at 8 for headroom.

Failure: `BufferClientError` → the publish is **refused** with `buffer_queue_cap_exceeded` and a `publish_attempts` row. Fails closed.

---

5. I3 · Canva — design generation

Three tools, two of which are **template-locked** (`template_id` required in the runtime `inputSchema`). There is no blank/free-form design creation path.

Currently **fixture-only**: `canva-refresh-token` does not exist as a Key Vault secret. `mcp/mcp-canva/scripts/oauth_consent.py` runs a local OAuth2 + PKCE flow (redirect `http://127.0.0.1:8484/oauth/redirect`) and *prints* the refresh token for an operator to load into Key Vault through the existing gated in-VNet path. **The script never touches Key Vault itself** — a deliberate separation of the consent flow from the secret-loading flow.

Function 45 (carousel writer) produces a Canva Bulk Create CSV manifest, so the intended flow is: agent writes slides + CSV → `bulk_create_from_csv` autofills a brand template → `export_design` . That flow is designed and unexercised.

---

6. I4 · Public web — the knowledge intake

One tool, **`fetch_url`** **. Four URLs. Three domains.**

```
# functions/09-market-intelligence-director/fetch_sources.yaml
version: 1
topic: "Microsoft Fabric adoption and multi-entity finance consolidation in South African enterprises"
horizon_days: 30
urls:
  - https://learn.microsoft.com/en-us/fabric/get-started/whats-new
  - https://www.moneyweb.co.za/feed/
  - https://www.moneyweb.co.za/news/tech/feed/
  - https://businesstech.co.za/news/feed/
```

Read at runtime by `ingest_signals_handler` — never hardcoded in Python. Each body is truncated to 2,000 characters and handed to function 09's prompt as retrieval evidence.

The domain list is mirrored into `MCP_WEB_ALLOWLIST` in `infra/main.bicep` , and the two **currently agree exactly** (`learn.microsoft.com` , `www.moneyweb.co.za` , `businesstech.co.za`) — verified against `infra/main.bicep` L971–972. Nothing enforces that agreement, so it remains a drift risk rather than a guarantee. The allowlist is checked **before any network call is made** — not after a response, not by URL rewriting.

Live mode is undeclared config drift — see **`09-technical-debt.md`** **TD-31.** `MCP_WEB_LIVE_MODE` gates fixture-vs-live and appears nowhere in `infra/` . It was set by hand on the live Container App (evidence: learning L-0074, 2026-08-02). Because ARM replaces the `env` list declaratively and `mcpDeployToken` defaults to `utcNow()` , the next `deploy-infra` or `deploy-governance` run reverts mcp-web to fixture mode — where `fetch_url` ignores the URL and returns the same synthetic placeholder for all four sources, with the loop still reporting success.

Failure: one bad source never sinks the scan — `ingest_signals_handler` logs `fetch_url_failed` and continues. Only if *every* source fails does it raise `DispatchError`.

Second failure mode, more subtle: real news prose trips the `full-name-like` redaction pattern (any two consecutive Title-Case words). `_complete_ingest_with_redaction_fallback` drops one source at a time and retries — degrading signal completeness rather than dead-lettering the task — and never second-guesses the firewall's ruling.

Unmitigated risk: article bodies (2,000 chars each) reach the `user` role with **no prompt-injection defence**. See `09-technical-debt.md` SEC-6.

---

7. I5 · Microsoft Teams — approvals and notifications

Classic O365 connector webhooks (MessageCard + `potentialAction`) were retired in the **May 2026 connector shutdown** (learning L-0033), so `teams-webhook-url` must be a **Power Automate Workflows** HTTP-trigger URL and the body must be an **Adaptive Card v1.4**.

Approve/Reject are `Action.OpenUrl` deep links only. The reasoning is explicit and correct: submit-style actions would need a registered Bot Framework bot, and — *"more importantly — a submit-style postback would make 'who clicked' a claim of the card payload rather than an authenticated identity."* The OpenUrl links point at the Entra-protected approval app, where identity comes from Easy Auth headers.

Currently off. `TEAMS_WEBHOOK_URL` is absent in `cmos-dev`; the `approval_inbox` row *is* the delivery mechanism, and that is the primary end-to-end-tested path. `gatekeeper-app.bicep` was recently wired to accept the secret (commit `e148d18`).

---

8. I6 · Entra ID — the only identity provider

Two apps behind Container Apps built-in authentication:

APP	UNAUTHENTICATEDCLIENTACTION	IDENTITY USED FOR
`ca-console`	(Easy Auth)	operator identity on kill-switch toggles
`ca-gatekeeper-approval`	`Return401`	**the recorded approver on every approval decision**

Headers injected by the platform and parsed by `auth.py` in both apps: `X-MS-CLIENT-PRINCIPAL-ID` · `-NAME` · `-IDP` · `X-MS-CLIENT-PRINCIPAL` (base64 claims blob, used as a fallback).

Fully secretless via a Federated Identity Credential (learning L-0013, `scripts/bootstrap-console-auth.sh`, `docs/console-auth-runbook.md`) — no client secret exists for either app registration.

Gap: authentication only. `allowedApplications` is empty and no app-role or group claim is required, so any authenticated tenant user reaches every console screen. Mitigated only by a Portal-set "Assignment required = Yes" that a human must remember (`docs/accepted-risks.md`).

---

9. I11/I12 · Microsoft Fabric and Power BI

`analytics_ingest.fabric_export.export_fabric_day()` assembles one JSON object per day, **validates it against `analytics/contracts/fabric-nightly-export.schema.json`** (four arrays, each mirroring a `kpi_rollup_*` table exactly), and uploads it to the `analytics-fabric-export` blob container.

`analytics/powerbi/analytics-dataset.json` is a starter dataset definition — four tables, typed columns, each naming its `source_array`. Its own description says: *"This file is a starter definition only — no live Power BI workspace/dataset is provisioned or refreshed by this build."*

The last mile is not built. The Fabric shortcut is not provisioned; no Power BI workspace exists; no refresh schedule. The export lands in a blob container and stops.

[INFERRED] This is intentional and appropriate — Canvas Intelligence's own business *is* Microsoft Fabric implementation. The platform produces a contract-validated payload and hands off to the practice that owns Fabric.

---

10. Credentials and secret management

Key Vault secrets (`kv-cmos-dev-*`, `publicNetworkAccess: Disabled`):

SECRET	CONSUMER	STATE
`vault-db-connection-string`	Vault	live, re-synced by `caj-vault-secret-writer`
`buffer-api-key`	mcp-buffer	present
`canva-client-id` / `canva-client-secret`	mcp-canva	present
`canva-refresh-token`	mcp-canva	**does not exist**
`teams-webhook-url`	Gatekeeper	**absent**
`gate-token-signing-key`	Gatekeeper (sign) / Publisher (verify only)	live, RSA
`ANTHROPIC_API_KEY`	model-gateway	via secretRef

Two credential-flow patterns coexist, and the distinction is deliberate:

- **Native Container Apps secretRef** — the platform resolves the Key Vault reference and projects it into the process env. This is what keeps the vault's `publicNetworkAccess: Disabled` viable: *"the gateway never calls the vault data plane itself."*
- **SDK lookup with `DefaultAzureCredential`** — used by the Vault service's DB-URL fallback and by `mcp_common.credentials`.

`docs/credentials-runbook.md` is **known stale** on Buffer/Canva secret names; `mcp/README.md` records the correct ones and says so.

---

11. Failure-mode summary

INTEGRATION	FAILURE	BEHAVIOUR	FAILS
Anthropic	non-2xx	500 `PROVIDER_ERROR`, orchestrator retries 3× then dead-letters	closed
Anthropic	redaction block	400, `gate_decisions` row, ingest drops a source and retries	closed
Anthropic	metering write fails	**completion still returned**, `cost_id` omitted, ERROR logged	**open, deliberately**
Buffer	unreachable / cap	publish refused, `publish_attempts` row	closed
Public web	one source down	logged, continue with the rest	open
Public web	all sources down	`DispatchError` → retry → dead-letter	closed
Teams	webhook absent	inbox row is the delivery	open
Key Vault	unreachable	service cannot start / cannot resolve DB	closed
Postgres	unreachable	`/health` still 200; `/status` returns `[]`; MCP logging skipped	mixed, by design
App Insights	unreachable	span presence → `not_checked`; console trace → empty state + flag	open
Vault	lookup fails with `asset_id` supplied	publish **refused** (`vault_lookup_failed`)	closed
Gatekeeper	unreachable from `run_state`	`{"status":"unknown"}` rather than a 500	open

The one deliberate fail-open worth understanding: a metering write failure does **not** fail the completion. The reasoning is written into the code — *"the expensive, valuable work already succeeded; a transient Postgres blip must never turn a real, already-paid-for completion into a 500 for the caller — that would be strictly worse than a missed costs row."* Logged at ERROR, `cost_id` simply omitted (contract-safe, since it is not in the response's `required` list).

---

12. Integration gaps

MISSING	IMPACT
CRM / marketing automation (HubSpot, Dynamics, Salesforce)	No lead or pipeline linkage — marketing output cannot be tied to revenue
Email service provider	`publish-newsletter` reuses `publish.blog_article` because *"no email-specific gate-check identifier exists"*
Fireflies	Referenced by fn 26's skill.md; no client, no tool, no MCP server
Website / CMS	No path to publish web copy despite `positioning.md` naming "fix the shop window" as priority #1
Semrush / SEO tooling	`vault-api.yaml` 's own example cites `source: "semrush"` — no integration exists
Slack	Teams only
Calendar	No scheduling awareness in the content plan

13

AI Agent Catalogue

All 23 function-definition packages. "Wired" means the `task_type` has a real entry in `DISPATCH_TABLE`; "referenced" means a loop YAML names it but it falls through to `legacy_task_pass_through` and does nothing.

Status summary

STATUS	COUNT	PACKAGES
Wired and running	3	02, 09, 42
Referenced by a loop, not wired	20	10, 11, 12, 13, 16, 18-01…18-06, 25, 26, 39, 41, 43, 45, 46, 47, 52
No package (deterministic code)	2	`brief.compose`, `request-approval`

All 23 packages pass `validate_package.py`, carry ≥5 golden eval tasks, and pass `eval_harness.py` against the mocked gateway. The gap is purely runtime wiring — see `09-technical-debt.md` TD-01.

A · Governance agent

Function 02 — Brand Steward QA ★ WIRED

Role	"You are the Brand Steward. You do not write marketing copy — you judge it."
Model	`claude-sonnet`
Invoked from	`qa-review` (daily loop, twice: brief-QA and proof-circuit content-QA)
Output	`{pass: bool, violations: [code], notes: string}`
Authority	**Blocking. `pass:false` → task FAILED / reason `qa_blocked`, `advance_dependents` never called.**

Six checks, in order, each with its own violation code:

#	CODE	RULE
1	`uncleared-client-reference`	Default deny. Absence from the register blocks identically to explicit UNCLEARED
2	`link-shortener`	bit.ly, lnkd.in, tinyurl, ow.ly, buff.ly — hide destinations and break UTM attribution
3	`sa-english-spelling`	US variants: productize, behavior, organization, optimize, analyze, center, color
4	`missing-cta`	Exactly one CTA. **Exempt when `channel == "internal-brief"`**
5	`url-utm`	Full `https://www.canvasintelligence.com/...` with 3 UTM params. **Same exemption**
6	`unsupported-claim`	Superlatives with nothing attached. "the leading platform" fails; "one of the platforms behind 40+ business units consolidated across 14+ ERP systems" passes

Design notes worth preserving:

- *"Never rewrite the draft"* — a QA function that edits its own input cannot be trusted to judge it. `tools.yaml` is 100% read-only for this reason.
- *"No partial pass."* `pass` is true only when `violations` is empty.
- *"Never resolve an uncleared client name by removing it yourself — that is the writer's decision to make with the verdict in hand."*
- The `channel` parameter changes which rules apply. `dispatch.py` passes `"internal-brief"` for brief-QA and `"linkedin"` for content-QA. **A prompt that reads a runtime discriminator and adjusts its own rule set** is policy parameterisation, not just templating.
- Function 02's `permission_check.py` is **loaded by reference** into the orchestrator via `importlib` (a digit-prefixed directory can't be dotted-imported — learning L-0039) so the deterministic uncleared-client check is the *same code*, never a fork. Its verdict is **merged with**, not substituted for, the LLM's own.

Live gap: `client_references` is always `[]` at the qa-review call site, so the deterministic check currently always passes trivially (TD-28).

B · Intelligence agents

Function 09 — Market Intelligence Director ★ WIRED

Model	`claude-haiku` (extraction tier)
Invoked from	`ingest-signals` (daily loop, first node)
Tools	`web_search` (declared, **not implemented**), `fetch_url` (live), `vault_signal_lookup` (declared)

Output: `{topic, horizon_days, summary, signals[{headline, so_what, source_url, pillar, confidence}]}`

Eight hard rules, and each one is an epistemics rule rather than a formatting rule:

1. 3–8 signals
2. every signal carries an `https://` `source_url` — *"a signal you cannot attribute to a retrievable source is not a signal — drop it"*
3. ≥2 distinct domains — *"three headlines from one vendor blog is one signal, not three"*
4. exactly one pillar per signal, from the five verbatim names
5. `confidence: low` for a single unverified source, a vendor's claim about itself, or anything outside the horizon — *"do not round thin evidence up to medium"*
6. the summary repeats the horizon number so a reader knows the window
7. **never name a client** — *"describe organisations generically ('a listed logistics group')"*
8. South African English

Method: *"Work source-first, not conclusion-first. Retrieve, then attribute, then interpret."*

This is the best-written prompt in the repository. Rules 2, 3 and 5 are anti-hallucination controls expressed as editorial standards.

Functions 10 · 11 · 12 · 13 · 16 — Competitive intelligence — REFERENCED

FN	NAME	LOOP TASK_TYPE
10	Competitor Discovery Scanner	`competitor-discovery-scan`
11	Competitor Change Monitor	`competitor-change-monitor`
12	Competitive Positioning Analyst	`competitive-positioning-analysis`
13	Competitor Content Performance Scout	`competitor-content-performance-scout`
16	Microsoft Fabric Ecosystem Scout	`fabric-ecosystem-scout`

All five share an eval shape: card-count-and-shape, source-attribution, domain-diversity, no-client-naming, tagging-in-set. Functions 11 and 13 add a sixth: **no-named-individual** — a stricter privacy rule than the others, appropriate for monitoring competitors' people.

Functions 18-01 … 18-06 — Vertical intelligence — REFERENCED

Six independent packages, **never one monolith**, sharing `functions/_shared/vertical-intelligence-method.md`:

FN	VERTICAL	PROOF BASIS (POSITIONING.MD §4)
18-01	Logistics & Fleet/Telematics	Imperial, Hestony/Powerfleet
18-02	Mining & Industrial	ArcelorMittal, Weir, Rotork
18-03	Manufacturing	**none — deliberately proof-light**
18-04	Construction	Construction sector
18-05	FMCG & Beverage	Delta
18-06	Financial Services	a Fabric client

18-03 is the most interesting package in the repository. `positioning.md` §4 names five vertical proof areas and manufacturing is not one of them, so 18-03's prompt has a "Proof-light default" section, its evals default `evidence_grade` to `light` and never `strong`, and it carries a sixth eval task (`task-06-proof-light-default.json`) enforcing exactly that.

The agent is configured to be less confident where the company has less evidence. That is epistemic humility encoded as a testable behaviour, and it is rare.

Function 25 — Competitive Response Strategist — REFERENCED

Consumes the deduped cards from all 11 scanners and produces a severity-ranked response plan, naming playbook templates ("RIB BI+ move", "BuildSmart-native-BI move"). Six evals including a seeded golden (`task-06-rib-bi-buildsmart-seeded-golden.json`).

C · Content agents

Function 42 — LinkedIn Post Writer ★ WIRED

Model	`claude-sonnet`
Invoked from	`draft-content` (S8 proof circuit only, permanently dry-run)
Output	`{post, pillar, cta_url}`

Seven hard rules: proof over platitude (with an explicit list of the superlatives function 02 will reject, and a worked pass/fail example), client names gated, roof line "Your Data. Delivered." on its own line, exactly one CTA with three UTM params and no shortener, SA English, 90–220 words with ≤3 hashtags, name at least one pillar verbatim.

The prompt embeds `positioning.md` **'s messaging house as a table** — five pillars, five messages, five lead proofs — and the CFO's voice-of-customer language verbatim from the pre-meeting survey ("different number for the same question", "More than 3 days", "No more Excel accounting"). The instruction is explicit: *"Mirror their own words — do not paraphrase them into consultant language."*

The prompt-02-awareness pattern: function 42's prompt tells the model what function 02 will reject and why. **The generator is aware of the critic.** That is a two-agent design expressed entirely in prompt content.

`F-PROMPT-OUTPUT-CONTRACT` **(round 18c)** — this prompt originally had no "return a single JSON object" instruction at all, unlike its siblings 02 and 09. Every production `draft-content` call failed to parse. The repo-wide sweep found 6 of 23 packages affected, and the fix added a **validator rule** so a 7th can never slip through.

Function 41 — Research Brief Writer — REFERENCED

The Tuesday node every Wednesday drafting function works from. Evals include `cfo-quote-included` and `missing-source-citation` .

Function 39 — Insight-to-Story Editor — REFERENCED

Turns a raw insight into a narrative closing on the roof line.

Function 43 — Executive Ghostwriter — REFERENCED

First-person opinion in a named executive's voice. **Its second eval is** `task-02-fabricated-opinion-blocked.json` — the agent must refuse to invent an opinion the executive did not actually state. Deliberately excluded from the Friday auto-schedule step.

Function 45 — Carousel Post Writer — REFERENCED

Multi-slide carousel **plus its Canva Bulk Create CSV manifest**. Eval 02 validates the CSV manifest shape.

Function 46 — Newsletter Writer — REFERENCED

Long-form owned-channel digest. Publishing reuses `publish.blog_article` because no email-specific autonomy identifier exists.

Function 47 — Case Study Writer — REFERENCED

Situation/approach/result structure. Names a client **only if CLEARED** — nothing is. Ships its own `permission_check.py` . **Intentionally excluded from the Friday auto-schedule**, published on a human-driven cadence once a client is cleared.

Function 52 — Content Repurposer — REFERENCED

One long-form asset → 2–3 shorter derivatives. Eval 05 asserts the output count matches `target_formats` .

Function 26 — Client Advocacy Harvester — REFERENCED

Turns an approved Fireflies transcript excerpt into a testimonial intake record, gated on a **local consent-register fixture** *and* the permission register. Its six evals include `revoked-consent-blocks` and `absent-from-register-blocks-identically`. Ships its own `permission_check.py`. The Fireflies integration itself does not exist.

D · Deterministic (non-LLM) handlers

`brief.compose` — `draft_brief_handler`

No package, no LLM call, no cost. `_render_brief()` turns function 09's structured output into two markdown documents (full + "Executive Edition", top-3 signals). Cites sources **by domain only**, never the bare URL — *"a raw external citation link is neither a Canvas CTA link nor one that should carry Canvas's own utm_ parameters"*, so function 02 never judges an internal citation against customer-facing link rules.

A deterministic handler between two LLM handlers is a good pattern. It is free, reproducible byte-for-byte, and cannot hallucinate.

`request-approval` — `request_approval_handler`

No LLM. Calls `/gate-check` once and **completes as soon as it responds** — never polls, never waits for the human. Stores `agent_run_id`, `function_id` and `content_hash` in its `result_ref` so `run_state.py` can look up the real decision later.

E · Agent design patterns worth naming

PATTERN	WHERE	ENTERPRISE NAME
Generator aware of critic	fn 42's prompt lists fn 02's rejection rules	Adversarial prompt co-design
Critic cannot edit	fn 02 `tools.yaml` is read-only	Separation of duties
Deterministic step between LLM steps	`_render_brief`	Hybrid symbolic-neural pipeline
Prompt drives its own eval oracle	`tool_check.py` derives output from `prompt.md`	Behaviour-coupled regression testing
Broken-copy regression fixture	`fixtures/regression/42-...-broken/`	Mutation testing for prompts
Confidence calibrated to evidence	fn 18-03 proof-light default	Epistemic humility as a testable property
Runtime discriminator changes rule set	fn 02's `channel` parameter	Policy parameterisation
Shared method document	`_shared/vertical-intelligence-method.md`	Prompt DRY
Code reused by reference, never forked	`permission_check.py` via importlib	Single source of truth
Skill file states when NOT to invoke	every `skill.md`	Negative capability specification

The `skill.md` convention deserves special mention. Every package documents **"When NOT to invoke"**. Function 02's is exemplary:

> *"To write or rewrite copy (it returns verdicts only, by design — a QA function that edits its own input cannot be trusted to judge it). To source market claims (function 09).* ***As an advisory 'second opinion' that a caller may override: a `pass: false` verdict is blocking."***

Most agent catalogues document capabilities. Documenting *anti*-capabilities and *authority* is what makes an agent safe to compose.

F · Agent estate gaps

GAP	IMPACT
20 of 23 agents never execute	The advertised estate is 8× the delivered estate
No agent evaluates another's output at runtime	Function 02 does — but only for the 3 wired agents
No agent has multi-turn / tool-use capability	Every call is single-shot
`web_search` **declared but not implemented**	Function 09's intelligence is limited to 4 static URLs
No agent reads its own history	No agent sees prior runs, prior verdicts, or prior performance
No agent-to-agent negotiation or debate	Communication is `result_ref` pointers only
`registry_version` **never authoritatively populated**	Cannot answer "which prompt version produced this asset?"
No cost-per-agent budget differentiation	`budgets.yaml.agents` is `{}` ; every function shares the $20/day default

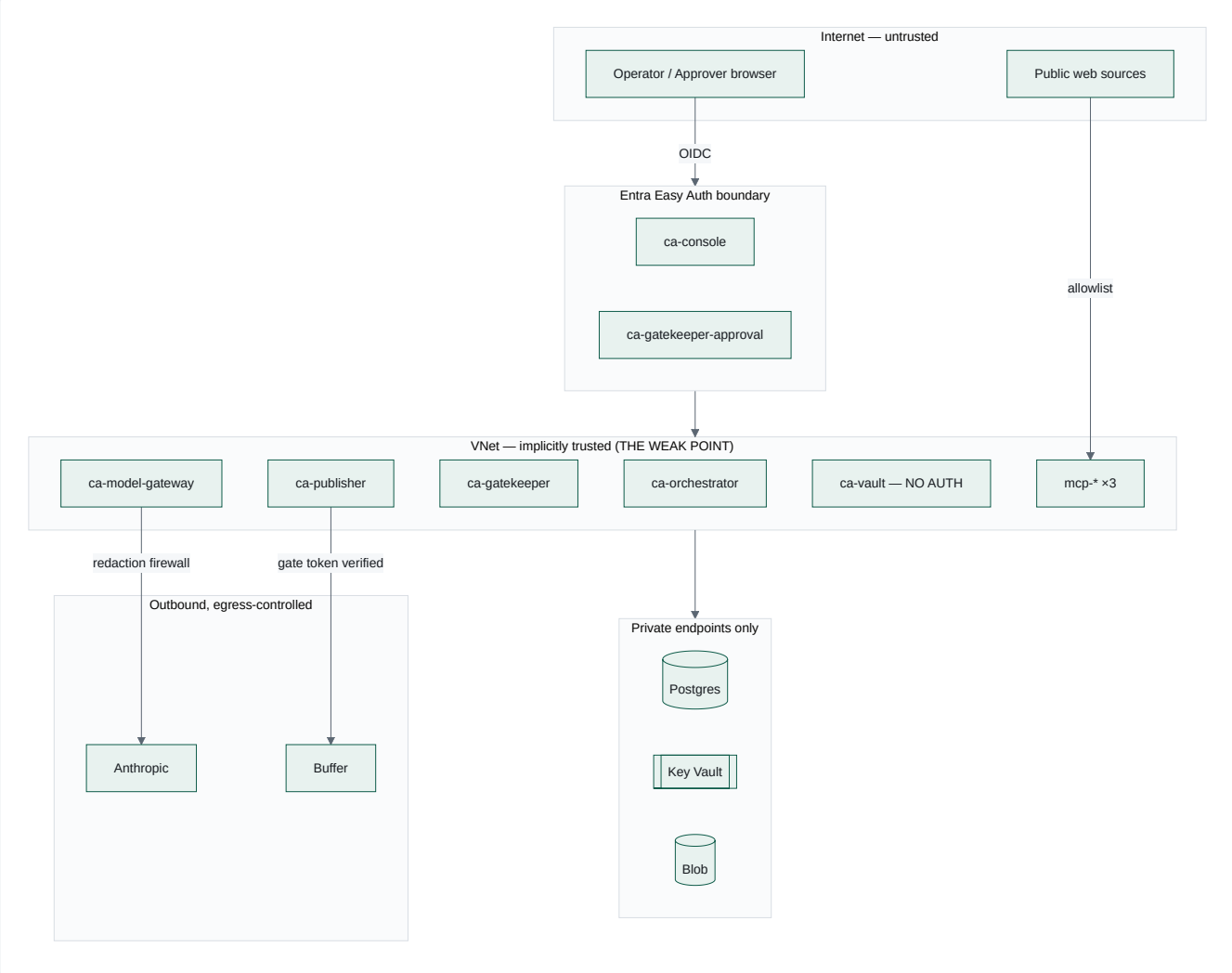
14

Security Model & Permission Model

*What the code enforces. Where a control is documented but not enforced, it is marked **NOT ENFORCED**.*

Part 1 — Security Model

1.1 Trust boundaries



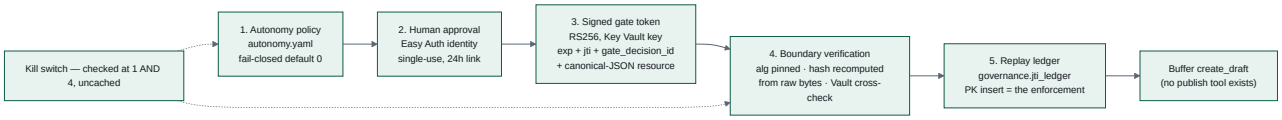
The security model is fundamentally perimeter-based. Once inside the VNet, there is no service-to-service authentication at all — five of eight services enforce zero authentication. This is documented, not accidental (`docs/accepted-risks.md`), but it is the single largest structural weakness.

1.2 Authentication inventory

BOUNDARY	MECHANISM	STRENGTH
Human → console	Entra Easy Auth + FIC (no client secret)	**Strong**
Human → approval app	Entra Easy Auth, `Return401`	**Strong**
Service → Postgres	admin username/password from Key Vault	Moderate — a single shared admin account for 5 schemas
Service → Key Vault	managed identity	**Strong**
Service → Blob	managed identity (no account key, no SAS)	**Strong**
Service → Service Bus	managed identity, `disableLocalAuth: true`	**Strong**
Service → Anthropic	API key from Key Vault via secretRef	Standard
Service → ACR	managed identity `AcrPull`, admin user disabled	**Strong**
CI → Azure	OIDC federated identity, **zero client secrets across 13 workflows**	**Strong**
Service → Service	**none**	**Absent**

1.3 The authorisation chain for an outbound action

Five independent layers. An attacker must defeat **all five**.



What each layer defeats:

LAYER	DEFEATS
1	An agent attempting an action outside its mandate; an unlisted function silently gaining authority
2	Autonomous publication; unattributable approval; link forwarding (identity ≠ possession); stale approvals (24h)
3	Forged authorisation (asymmetric, issuer-held private key); unbounded validity (`exp`)
4	`alg:none` ; algorithm confusion; **approve-A-publish-B** (hash recomputed from the bytes, never trusted); a corrupted `assets.content_hash` column
5	Replay across replicas and restarts (durable, PK-enforced, atomic)
Kill switch	Everything, within 5 seconds, globally or per function

1.4 The gate token in detail

```
Header  {"alg":"RS256","typ":"JWT"}
Claims  {iss, sub, aud, iat, exp, jti, gate_decision_id,
         resource: '{"content_hash":"<hex>","function_id":"<id>"}'}
```

Four properties, each enforced:

(a) Algorithm pinning happens first. `_pin_algorithm()` inspects the header *before any signature work*, rejecting `alg: none` and algorithm confusion (HS256 signed with the RSA public PEM as an HMAC secret) with the distinct reason `invalid_alg` .

(b) The `resource` claim is canonical JSON, and canonicalisation is a security property, not formatting. The frozen v1 schema sets `additionalProperties: false` , so `function_id` and `content_hash` cannot be top-level claims. They are packed as `json.dumps(obj, sort_keys=True, separators=(",",":"))` . The verifier **re-serialises the parsed claim and requires byte equality** — because without that, two different serialisations of the same object would compare unequal, or a semantically different one could compare equal, in a hash-binding comparison.

(c) The approver is never a claim. It is resolved server-side via `gate_decision_id → gate_decisions.decided_by` , *"so a token never carries a human identity it could leak."*

(d) Publisher holds only the public key. Its managed identity has verify/get on Key Vault and cannot sign (AC-20).

Contract-level note: `contracts/gate-token/spec.md` allows RS256, ES256, EdDSA and PS256. Only RS256 is issued and accepted, because **Azure Key Vault standard tier cannot create, sign or verify Ed25519 keys at any SKU** (learning L-0031).

1.5 Data protection

CONTROL	MECHANISM	STRENGTH
Egress DLP	Redaction firewall pre-provider: SA ID (13-digit), SA phone (+27/0XX), email, name-shape + exact-match fixtures	Moderate — regex-based, **coverage known incomplete**, which is *why* every block is audited
Queue redaction	Metadata-only envelopes, ids not content	**Strong** — structural
Telemetry redaction	Closed enum keys + 200-char rejection	**Strong** — structural
Consent gating	403 on any `data_subject_ref` write with no active matching consent	**Strong**
Encryption at rest	Azure defaults (Postgres, Blob, Key Vault)	Standard
Encryption in transit	TLS 1.2 minimum on Service Bus; HTTPS throughout	Standard
Retention	4 classes, sweep fails closed on blob-delete error	**Strong**
Content addressing	SHA-256 blob names, dedup, reference-counted deletion	**Strong**
Error-message hygiene	DR-3/4/5 — no submitted value ever echoed in a 400	**Strong, unusually thorough**

The DR-4 fix deserves repeating because it is the kind of bug that only appears in careful reviews: matched-pattern ids were originally `f"fixture:{value}"`, where a fixture value **is** a real client name. That id went into the caller-facing 400 body *and* into the permanent `gate_decisions.reason` column — *"defeating the firewall through its own audit trail."* Ids are now contract-side coordinates (`fixture:client_names:0`).

1.6 Network security

RESOURCE	PUBLIC ACCESS	REACHABLE FROM
Postgres	**Disabled**	private endpoint in `snet-pe`
Key Vault	**Disabled**	private endpoint
Storage	private endpoint	VNet
Container Registry	public endpoint, admin user disabled	managed-identity pull only
Service Bus	**Public (Standard SKU)**	Entra-authenticated callers only
Container Apps	6 internal, 2 external	—

The Service Bus exposure is an explicitly accepted, budget-owner-approved risk with **three compensating controls**: `disableLocalAuth: true` (SAS disabled entirely — Entra only), `minimumTlsVersion: 1.2`, and metadata-only envelopes. The reasoning is sound: even a successful read exposes only ids.

1.7 Audit and non-repudiation

Six append-only tables. Nothing in the codebase issues an `UPDATE` or `DELETE` against any of them.

TABLE	RECORDS	IMMUTABILITY MECHANISM
`gate_decisions`	Every authorisation decision, human and machine	**No `updated_at` column, deliberately**
`approval_actions`	Every approval-link click, 4 outcomes	CHECK-constrained outcome
`publish_attempts`	Every publish attempt, 12 reasons	CHECK-constrained outcome
`vault_internal.audit_log`	Every taxonomy/consent rejection, every retention deletion	Written on an **isolated connection** so it survives the rollback
`task_transitions`	Every state change, 10 CHECK-constrained reasons	append-only by construction
`jti_ledger`	Every consumed token	PK insert only

`write_audit_isolated()` **is the subtlest and most important of these.** The rejection paths run inside a transaction that they then abort by raising. On a shared connection, the audit row would roll back with everything else — *"silently losing the very audit trail the rejection is supposed to produce."* A separate pooled connection makes the audit survive.

1.8 Threat model — what an attacker can and cannot do

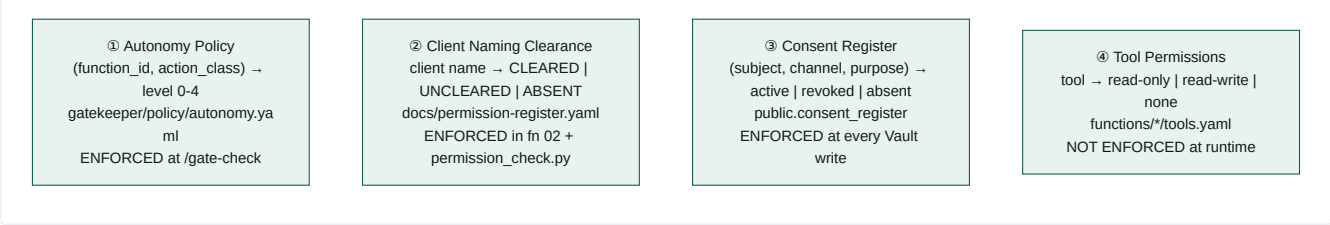
ATTACK	OUTCOME
Steal an approval link	**Blocked** — 401 without an Easy Auth principal
Replay a used approval link	**Blocked** — atomic conditional UPDATE; 409 + audit
Use an expired link	**Blocked** — 410 + audit
Forge a gate token	**Blocked** — RS256, issuer-held private key
`alg: none` / algorithm confusion	**Blocked** — pinned before any signature work
Replay a gate token	**Blocked** — durable PK-enforced ledger
Approve draft A, publish draft B	**Blocked** — hash recomputed from the raw bytes
Publish after the kill switch is pulled	**Blocked** — re-checked before jti consumption
Exfiltrate PII via a prompt	**Mitigated** — firewall; regex coverage incomplete
Exfiltrate PII via a tool definition	**Blocked** — `tools[]` is serialised and scanned
Exfiltrate PII via an error message	**Blocked** — DR-3/4/5
Exfiltrate PII via telemetry	**Blocked** — closed enum + 200-char limit
Grant an agent new authority by naming a function	**Blocked** — fail-closed default level 0
Bypass approval via a smoke/test function id	**Blocked** — none exists; a test forbids `smoke.*` / `test.*`
Name an uncleared client	**Blocked** — default-deny register with a self-test
Reach any internal service from inside the VNet	**NOT BLOCKED** — no service-to-service auth
Read/write/delete any Vault object from inside the VNet	**NOT BLOCKED** — zero auth on the system of record
Pull the kill switch as any authenticated tenant user	**NOT BLOCKED** — authentication without authorisation
Inject instructions via a fetched news article	**NOT BLOCKED** — no prompt-injection defence

The four unblocked rows are the security roadmap. Three are one week of work each (`10-product-roadmap.md` R3, R5, R17).

Part 2 — Permission Model

2.1 The four independent permission systems

This platform has **four separate permission models that do not share a vocabulary or an enforcement point**. Understanding that is essential.



2.2 ① Autonomy policy — the delegation-of-authority matrix

```
version: 1
default_level: 0          # fail closed
entries:
  - {function_id: publish.social_post,          action_class: publish,  level: 1}
  - {function_id: publish.paid_ad,              action_class: publish,  level: 0}
  - {function_id: publish.blog_article,         action_class: publish,  level: 2}
  - {function_id: draft.social_post,            action_class: draft,    level: 3}
  - {function_id: draft.brief,                  action_class: draft,    level: 3}
  - {function_id: analyse.signal,               action_class: analyse,  level: 4}
  - {function_id: analyse.campaign_performance,  action_class: analyse,  level: 4}
```

Seven entries. Every unlisted pair resolves to level 0 and is blocked.

LEVEL	HUMAN?	TOKEN ISSUED?	AUDIT ROW?
0 blocked always	—	❌	✅ `level_0_blocked`
1 single approver	✅ one click	after approval	✅ escalated, then approved
2 elevated	✅ one click *(quorum reserved, not built)*	after approval	✅ distinct reason string
3 auto-approved, audited	❌	✅	✅ `level_3_auto_approved`
4 autonomous passthrough	❌	✅	✅ `level_4_autonomous_passthrough`

Load-time validation is strict and fails the service, not the request: duplicate `(function_id, action_class)` pairs, unknown keys, non-integer or out-of-range levels, and a non-mapping top level are all `PolicyError` at startup — *"the service must never start with a half-understood policy and then fail open on the first request."*

Two invariants enforced by `tests/test_policy.py`: no `publish`-class entry above level 2, and no `function_id` matching `smoke.*` / `test.*`.

The policy is **cached deliberately** (unlike kill switches) because it is a build-time artefact shipped in the bundle and only changes on redeploy.

2.3 ② Client naming clearance — default deny

`docs/permission-register.yaml`, six clients, **all UNCLEARED**: Imperial, Rotork, Weir, ArcelorMittal SA, SGB Cape, Delta.

```
default_policy: deny
absent_name_policy: deny     # absent == UNCLEARED == blocked
graduates_to: vault.consent_register
```

Four properties, all tested:

- Only the exact string `CLEARED` permits naming. A typo, an empty value, or a future `PENDING` all block.
- **Absence blocks identically to explicit UNCLEARED** — same `allowed=False`, same violation code. `_self_test()` asserts exactly this equality.
- Alias resolution and case-insensitivity ("imperial logistics" → Imperial).
- **A missing register file returns an empty index, not an allow-all** — *"a missing register cannot mean 'allow everything'."*

The file documents its own graduation path to the Vault `consent_register` table, with the requirement that *"the default-deny semantics must survive that move unchanged."*

2.4 ③ Consent register — POPIA-shaped, enforced at write time

Enforced in `services/vault/vault/routers/objects.py::handle_consent_gate`:

```
if object_table == "consent_register":  return None   # a grant is not a consumption
if "data_subject_ref" not in payload:   return None   # not client-derived
if not consent_channel or not consent_purpose:  raise 422
consent_id = find_active_consent(subject, channel, purpose)   # revoked_at IS NULL
if consent_id is None:  audit + raise 403 consent_required
link_consent(object, consent_id)                      # durable linkage
```

The self-exclusion is important and explained: gating `consent_register` on itself *"would make it structurally impossible to ever record a subject's first consent."*

Note the two deliberately distinct naming schemes, called out in the code so they are never conflated: the generic cross-cutting `consent_channel` / `consent_purpose` payload keys used to gate *other* object types, versus `consent_register`'s own `channel` / `purpose` **columns**.

2.5 ④ Tool permissions — declared, NOT ENFORCED

```
tools:
  - name: permission_register_lookup
    permissions: read-only
    scope: docs/permission-register.yaml
```

`contracts/function-definition/tools.schema.json` calls `permissions` *"the coarse permission class the orchestrator enforces."* **The orchestrator does not read `tools.yaml` at all.**

Real enforcement is architectural: handlers only construct the clients they were written to construct, and each MCP server enforces its own guardrail server-side. That is defence by construction — arguably stronger — but the manifest implies a capability system that does not exist.

2.6 Human roles — as implemented

ROLE	IDENTITY	CAN DO	CANNOT DO
Approver	Entra principal on the approval-action request	Approve/reject one specific action bound to one content hash	Anything else — the app mounts exactly one functional route
Operator	Entra principal on a console request	Read all 6 screens; toggle the kill switch	Approve, create, edit, delete — the console has one mutating route
Engineer	GitHub + OIDC	Change policy, prompts, code; deploy	Bypass the `cmos-dev` GitHub Environment human gate
Directory admin	Entra admin	App registration, FIC, assignment	—

No role hierarchy, no groups, no per-object ACLs, no delegation. Identity is entirely delegated to Entra; there is no in-app authorisation model at all — no `users`, `roles` or `permissions` tables.

2.7 Machine identities

IDENTITY	GRANTS
`id-orchestrator`	Service Bus Sender + Receiver, ACR pull, App Insights
`id-vault`	Key Vault Secrets User, Storage Blob Data Contributor, ACR pull
`id-gateway` (user-assigned)	Key Vault Secrets User, ACR pull
`id-gatekeeper`	Key Vault **Crypto User (sign)**, ACR pull
`id-publisher`	Key Vault **Crypto User (verify/get only — cannot sign)**
`id-console`	App Insights reader, ACR pull
`id-mcp-*` ×3	Key Vault Secrets User (scoped), ACR pull
Logic Apps ×3	Service Bus **Data Sender only**
GitHub OIDC	Contributor + User Access Administrator on the subscription

The gatekeeper/publisher split is the important one: **the service that signs cannot verify-only, and the service that verifies cannot sign.** That is correct key separation.

The GitHub OIDC identity holding **User Access Administrator** is the widest grant in the system — necessary because Bicep declares role assignments inline, but it means a compromised workflow can grant itself anything.

2.8 Permission model gaps

GAP	SEVERITY
No service-to-service authentication	Critical
No authorisation on the console — any authenticated tenant user reaches the kill switch	High
Tool permissions declared but not enforced	Medium — the manifest implies a capability system that isn't there
No role model — no users/roles/permissions tables, no groups, no delegation	Medium
Level 2 ≡ level 1 — no quorum, no second approver	Medium
No approval delegation or timeout escalation	Medium
Single shared Postgres admin account across 5 schemas	Medium — no per-service DB roles
GitHub OIDC holds User Access Administrator	Medium
No per-function budget differentiation — `budgets.yaml.agents` is `{}`	Low
No IP allowlisting on external apps	Low

15

Deployment Architecture & Configuration Guide

Part 1 — Deployment Architecture

1.1 Target environment

One Azure resource group: **cmos-dev**, region **southafricanorth** (pinned as a literal string in app-insights.bicep, never resourceGroup().location, so the data-residency guarantee cannot silently drift when another module's default changes).

1.2 Resource inventory

RESOURCE	NAME / SKU	NOTES
VNet	vnet-cmos-dev 10.20.0.0/16	snet-cae-infra /23 (delegated to Microsoft.App/environments), snet-pe /27
Container Apps Env	cae-cmos-dev	VNet-integrated, linked to log-cmos-dev
Postgres	psql-cmos-dev, PG16, Standard_B1ms Burstable, 32GB	publicNetworkAccess: Disabled, private endpoint, AZ 1, 7-day backup, **no HA**
Service Bus	sb-cmos-dev-<unique>, **Standard**	disableLocalAuth: true, TLS 1.2, queues task + event
Key Vault	kv-cmos-dev-<unique>	publicNetworkAccess: Disabled, private endpoint, RBAC mode
Storage	st...	containers vault-assets, analytics-fabric-export
ACR	acrcmosshared<unique>, **Basic**	adminUserEnabled: false
App Insights	workspace-based, southafricanorth	linked to log-cmos-dev
Private DNS zones	postgres / vault / blob	linked to the VNet
Container Apps	10	see below
Container Apps Jobs	17	see below
Logic Apps	3	Consumption, each with its own identity + SB Data Sender role

1.3 Container Apps

APP	INGRESS	REPLICAS	PURPOSE
ca-orchestrator	internal	1–3	FastAPI + background worker loop
ca-model-gateway	internal	—	completions
ca-gatekeeper	internal	—	gate-check, decisions, approval-status
ca-gatekeeper-approval	**external + Easy Auth**	—	the single external governance route
ca-publisher	internal	—	publish verification
ca-vault	internal	1–3	system of record
ca-console	**external + Easy Auth**	—	operator UI
mcp-web / mcp-buffer / mcp-canva	internal	—	tool plane

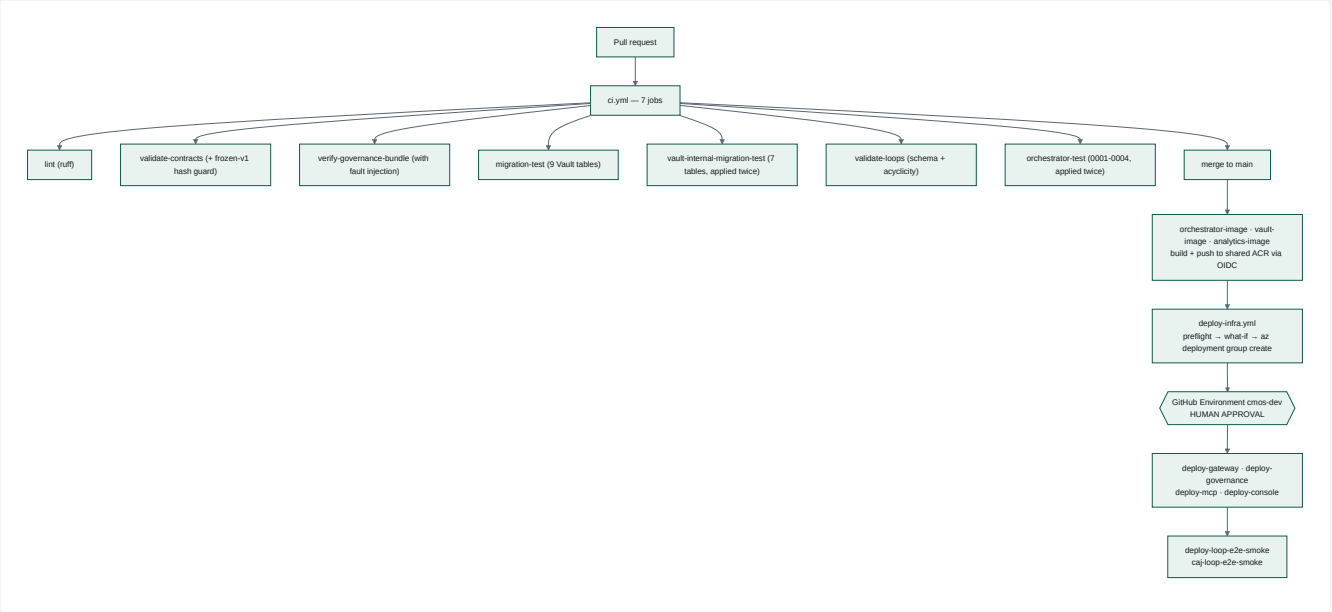
The max_size=12 connection-pool setting in vault/db.py is derived from this table: 3 replicas × 12 = 36 connections against Postgres's confirmed max_connections=50, leaving headroom for migration/query/retention/rollup jobs. The previous value of 20 allowed 60 — already over the limit on its own (finding PERF-2).

1.4 Container Apps Jobs

CATEGORY	JOBS
Migration	`caj-vault-migrate` , `caj-vault-sidecar-migrate` , `caj-orchestrator-migrate` , `caj-governance-migrate` , `caj-analytics-migrate` , `caj-mcp-ops-migrate`
Smoke	`caj-gateway-smoke` , `caj-vault-smoke-test` , `caj-orchestrator-smoke-test` , `caj-governance-smoke` , `caj-mcp-smoke` , `caj-loop-e2e-smoke` , `caj-analytics-buffer-smoke`
Scheduled	`caj-analytics-nightly-ingest` (cron `0 1 * * *`), `caj-vault-retention-expiry`
Operational	`caj-vault-query` , `caj-vault-secret-writer`

Migration jobs exist because Postgres has no public endpoint — **the only way to run DDL is from inside the VNet**.

1.5 CI/CD



13 workflows. Zero client secrets. Everything authenticates via OIDC federated identity (`AZURE_CLIENT_ID` / `AZURE_TENANT_ID` / `AZURE_SUBSCRIPTION_ID`).

1.6 Deployment patterns learned the hard way

Each of these is a compound learning, and each is now a repository-wide rule:

(a) The three-part image bootstrap contract (L-0018, L-0048, L-0060 — independently reproduced on four separate Container Apps):

1. the Bicep `containerImage` param defaults to a **public MCR placeholder**;
2. the deploy workflow's preflight resolves the app's **current live image** via `az containerapp show` — but only if `latestReadyRevisionName` is non-empty, otherwise it re-bootstraps rather than replaying an image reference that has never worked;
3. only the image workflow ever sets a real image.

(b) Never declare `registries[]` on a new `Microsoft.App/jobs` resource (L-0049, L-0061, L-0071). A user-assigned identity cannot be newly attached *and* referenced for ACR pull in the same create.

(c) User-assigned, not system-assigned, when a role assignment is needed in the same deployment (L-0020). A system-assigned `principalId` only resolves once the resource reaches a terminal state — which requires the very role assignment being declared. Circular. A `Microsoft.ManagedIdentity/userAssignedIdentities` resource resolves synchronously and breaks the cycle.

(d) Never resolve an Azure resource with `list --query "[0].x"` (L-0021, L-0036). Once a second resource of that type exists in the RG, list order silently picks the wrong one. A `deploy-gateway` UNAUTHORIZED pull failure was traced to exactly this — the image pushed successfully to an *orphaned* registry the app had no `AcrPull` grant for.

(e) `az containerapp job start` with any Container Argument replaces the whole container (L-0022). Passing `--env-vars` alone fails with `ContainerAppImageRequired` ; adding `--image` back still silently drops the template's `command` and `secretRef` . The fix is a full `--yaml` override per invocation.

(f) Base64-encode every SQL secret (L-0012). Container Apps collapses a literal `$$` in secret values to `$`, corrupting PL/pgSQL dollar-quoting. CI reproduces the exact encoding round-trip before applying, so the failure mode cannot escape to a live deploy.

(g) `CONTRACTS_DIR` / `FUNCTIONS_DIR` (L-0062). A containerised service that reads repo-root files at runtime cannot use a parents-walk — the image has no repository around it. Confirmed live: `loop_load_failed ... No such file or directory: '/contracts/orchestrator/loop-definition.schema.json'`. The fix is an env-var override, evaluated lazily, with the image staging only the files it actually needs.

Part 2 — Configuration Guide

2.1 Configuration philosophy

Four tiers, and the boundaries between them are deliberate:

TIER	WHERE	CHANGE PROCESS
Policy	YAML in git	Pull request + review + deploy
Infrastructure	Bicep + `*.parameters.json`	Pull request + `deploy-infra` + human gate
Runtime	Container Apps env vars / secretRefs	`az containerapp update` or redeploy
Secrets	Key Vault, referenced by secretRef	Operator, via the gated in-VNet path

2.2 Policy files — the platform's control surface

FILE	GOVERNS	EFFECT OF A CHANGE
`services/gatekeeper/policy/autonomy.yaml`	What agents may do	Immediate on redeploy; **validated at startup, fails the service on error**
`services/model-gateway/policy/routing.yaml`	Logical model → tier/provider/model	A model upgrade is **one reviewed line**
`services/model-gateway/policy/budgets.yaml`	Daily USD per function; downgrade path	Cost ceiling and degradation behaviour
`contracts/model-gateway/redaction-rules.yaml`	PII patterns + fixtures	**Frozen** — hash-guarded; changes need a version bump
`docs/permission-register.yaml`	Which clients may be named	Read at runtime by `permission_check.py`
`services/orchestrator/loops/*.yaml`	Workflow DAGs	Validated at startup; a bad loop is logged, not fatal
`functions/_shared/scan-profiles.yaml`	Knowledge sources, per scan profile (replaced `functions/09-*/fetch_sources.yaml`)	Read at runtime; must stay in sync with `MCP_WEB_ALLOWLIST` (`scripts/check_allowlist_sync.py` fails CI when they diverge)
`functions/_shared/scoring-policy.yaml`	What "matters": confidence weights, pillar weights, and the selection cut	Read at runtime by `score-signals` and by the weekly planner. Ships neutral — defaults reproduce the pre-policy behaviour. A policy it cannot honour is refused at load
`functions/*/prompt.md`	Agent behaviour	Staged into the image; requires a rebuild

2.3 Environment variables by service

Orchestrator

VAR	DEFAULT	PURPOSE
`DATABASE_URL`	—	task_state; **absent ⇒ `/status` returns `[]`, no crash**
`SERVICE_BUS_NAMESPACE`	—	**absent ⇒ the local double is used**
`VAULT_API_URL`	—	Vault base URL
`WORKER_POLL_INTERVAL_S`	`1.0`	queue poll cadence
`CONTRACTS_DIR` / `FUNCTIONS_DIR`	repo-relative	**must be set in the container**
`CMOS_DAILY_LOOP_BUDGET_USD`	`5.00`	AC-10 loop budget
`CMOS_GATEWAY_BASE_URL` / `CMOS_GATEKEEPER_BASE_URL` / `CMOS_MCP_WEB_BASE_URL`	—	override live FQDN resolution
`APPLICATIONINSIGHTS_CONNECTION_STRING`	—	absent ⇒ telemetry no-op, never a crash

Model Gateway

VAR	DEFAULT	PURPOSE
`DATABASE_URL`	—	costs + gate_decisions
`ANTHROPIC_API_KEY`	—	via secretRef
`KEY_VAULT_NAME`	—	vault name
`DELIBERATE_FLAG_ENABLED`	`false`	reasoning-hint feature flag
`CONTRACTS_DIR`	repo-relative	**must be set in the container**

Gatekeeper

VAR	DEFAULT	PURPOSE
`DATABASE_URL` / `TEST_DATABASE_URL`	—	**TEST wins, so a test run cannot touch prod**
`AUTONOMY_POLICY_PATH`	`policy/autonomy.yaml`	policy location
`SIGNER_BACKEND`	`local`	`keyvault` **in production**
`KEY_VAULT_URL` , `GATE_SIGNING_KEY_NAME`	— / `gate-token-signing-key`	signing key
`GATE_TOKEN_ISSUER` / `_AUDIENCE` / `_TTL_SECONDS`	`cmos-gatekeeper` / `cmos-publisher` / `900`	token claims
`TEAMS_WEBHOOK_URL`	—	**absent ⇒ inbox-row delivery**
`APPROVAL_BASE_URL`	`https://approval.invalid`	**must be set — the default is a deliberate poison value**
`APPROVAL_LINK_TTL_SECONDS`	`86400`	24h

Publisher

VAR	DEFAULT	PURPOSE
`PUBLISHER_DRY_RUN`	**`true`**	`false` enables live Buffer
`GATE_TOKEN_PUBLIC_KEY_PEM`	—	accepts raw PEM **or** base64
`GATE_TOKEN_ALGORITHMS`	`RS256`	pinned allowlist
`VAULT_API_URL`	—	asset cross-check
`CMOS_MCP_BUFFER_BASE_URL`	—	override

Vault

VAR	DEFAULT	PURPOSE
`DATABASE_URL`	—	**falls back to a Key Vault lookup at first DB use**
`KEY_VAULT_NAME` / `KEY_VAULT_URL`	—	fallback source
`DB_CONNECTION_SECRET_NAME`	`vault-db-connection-string`	secret name
`STORAGE_ACCOUNT_NAME` / `BLOB_CONTAINER_NAME`	— / `vault-assets`	blob storage

Console

VAR	DEFAULT	PURPOSE
`VAULT_API_MODE`	`mock`	`real` + `VAULT_API_BASE_URL` — **config-only cutover, field-verified**
`GATEKEEPER_API_MODE`	`mock`	`real` **blocked — no REST API exists yet**
`APPLICATIONINSIGHTS_WORKSPACE_ID`	—	KQL trace queries
`CONSOLE_SEED_FIXTURES_JSON_B64`	—	local dev seed

MCP servers

VAR	DEFAULT	PURPOSE
`MCP_WEB_LIVE_MODE`	unset	**non-secret flag** — mcp-web has no vendor credential
`MCP_WEB_ALLOWLIST`	`example.com,api.example.com`	**must match** `fetch_sources.yaml` **'s domains**
`RATE_LIMIT_MAX` / `RATE_LIMIT_WINDOW_SEC`	`5` / `1.0`	sliding window
`BUFFER_API_KEY`	—	presence ⇒ live mode
`CANVA_CLIENT_ID` / `CANVA_CLIENT_SECRET`	—	both ⇒ live mode
`MCP_OPS_DB_POOL_MAX_SIZE` / `_TIMEOUT_SECONDS`	`5` / `5.0`	bounds Postgres connections per server

2.4 The dual-mode pattern

Every external integration follows the same rule: **fixture mode is the default and requires nothing; live mode activates purely on the presence of a credential env var. No code or config edit is ever needed.**

COMPONENT	LIVE TRIGGER
mcp-web	`MCP_WEB_LIVE_MODE` truthy
mcp-buffer	`BUFFER_API_KEY` resolvable
mcp-canva	`CANVA_CLIENT_ID` **and** `CANVA_CLIENT_SECRET`
analytics Buffer	`BUFFER_API_KEY` resolvable
Publisher	`PUBLISHER_DRY_RUN=false` *(and never for a proof-circuit asset)*
registry evals	`ANTHROPIC_API_KEY` + `--live`
Teams	`TEAMS_WEBHOOK_URL` present

Learning L-0074 warns about exactly this design: a "goes-live-automatically-once-credentials-exist" path must be *independently verified live*, not assumed to work because the fixture path does.

2.5 Bootstrap sequence for a fresh environment

```
# 0. Prerequisites
az login
az extension add --name containerapp --yes
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.DBforPostgreSQL
az provider register --namespace Microsoft.ServiceBus
az provider register --namespace Microsoft.ContainerRegistry   # NOT in deploy-infra's preflight

# 1. Infrastructure (placeholder images — this is expected and correct)
az deployment group create -g cmos-dev -f infra/main.bicep \
  -p infra/dev.parameters.json -p administratorLoginPassword=<generated>

# 2. Migrations — the ONLY way to run DDL (Postgres has no public endpoint)
for j in caj-vault-migrate caj-vault-sidecar-migrate caj-governance-migrate \
         caj-orchestrator-migrate caj-analytics-migrate caj-mcp-ops-migrate; do
  az containerapp job start -g cmos-dev -n "$j"
done

# 3. Secrets (operator, via the gated in-VNet path — see docs/credentials-runbook.md)
#    ANTHROPIC_API_KEY · buffer-api-key · canva-client-id/-secret
#    teams-webhook-url (optional) · vault-db-connection-string (caj-vault-secret-writer)

# 4. Console auth — one-time, requires directory admin rights
bash scripts/bootstrap-console-auth.sh          # see docs/console-auth-runbook.md
# THEN, in the Portal: Enterprise Application → "Assignment required" = Yes
#   and assign only intended operators. This is the ONLY authorisation control.

# 5. Real images
gh workflow run orchestrator-image.yml
gh workflow run vault-image.yml
gh workflow run analytics-image.yml
gh workflow run deploy-gateway.yml
gh workflow run deploy-governance.yml
gh workflow run deploy-mcp.yml
gh workflow run deploy-console.yml

# 6. Verify
for j in caj-vault-smoke-test caj-orchestrator-smoke-test caj-governance-smoke \
         caj-mcp-smoke caj-gateway-smoke; do
  az containerapp job start -g cmos-dev -n "$j"
done
gh workflow run deploy-loop-e2e-smoke.yml
```

2.6 Operational runbook essentials

Never hardcode an FQDN. Every address is resolved live:

```
ORCH=$(az containerapp show -g cmos-dev -n ca-orchestrator \
  --query properties.configuration.ingress.fqdn -o tsv)
```

Logs and job status:

```
az containerapp logs show -g cmos-dev -n ca-orchestrator
az containerapp job execution list -g cmos-dev -n caj-loop-e2e-smoke
az containerapp job logs show -g cmos-dev -n caj-vault-migrate --container <name>   # --container is REQUIRED (L-0024)
```

Ad-hoc SQL — `caj-vault-query`, and it **must** be invoked with a full `--yaml` override, never `--env-vars QUERY=...` (L-0022).

Emergency stop:

```
curl -X POST -H "Content-Type: application/json" \
  -d '{"active":true,"reason":"incident-2026-08-06"}' \
  https://<console-fqdn>/kill-switch/toggle
```

Propagates to the next gate decision and the next publish attempt — no cache, no TTL.

Trace a run:

```
curl -H "Accept: application/json" https://<console>/tasks/<task_ref>/trace
curl https://<orch>/runs/<task_ref>
```

2.7 Configuration risks

RISK	CONSEQUENCE
`MCP_WEB_ALLOWLIST` and `fetch_sources.yaml` can silently diverge	Ingestion fails with an allowlist rejection nobody expects
`APPROVAL_BASE_URL` defaults to `https://approval.invalid`	Approval links break silently if unset — deliberately poison, but unset is not loud
`SIGNER_BACKEND` defaults to `local`	A misconfigured production deploy signs with an **ephemeral key**; every token then fails verification
`PUBLISHER_DRY_RUN=false` is one env var away	The only guard is a proof-circuit tag on the asset
`TEST_DATABASE_URL` wins over `DATABASE_URL`	Good for tests; catastrophic if it leaks into a production env
`main.bicep` deploys everything together	A governance deploy rotates the Postgres admin password (bug `8277d38`); blast radius is the whole platform
`budgets.yaml.agents` is `{}`	Every function shares one $20/day default — no per-agent ceiling
Prompt changes require an image rebuild	Content changes carry deployment risk

16

Product Vision & Strategy

Grounded entirely in what the code proves the team can build. No market sizing, no assumed demand — the argument is made from implementation evidence.

1. Product vision

> **Every enterprise will run AI agents that do real work. Almost none will be able to prove what those agents were allowed to do, what they actually did, what it cost, or who authorised it. Canvas builds the layer that proves all four.**

The evidence that this is the real vision, rather than a repositioning exercise, is the measured code distribution: of 21,182 lines of non-test Python, **395 lines contain any marketing domain logic** — 1.9%, all of it in five handler functions in a single file. The team did not build a marketing tool and add controls. **They built a control plane and pointed it at marketing to prove it works.**

2. Product strategy

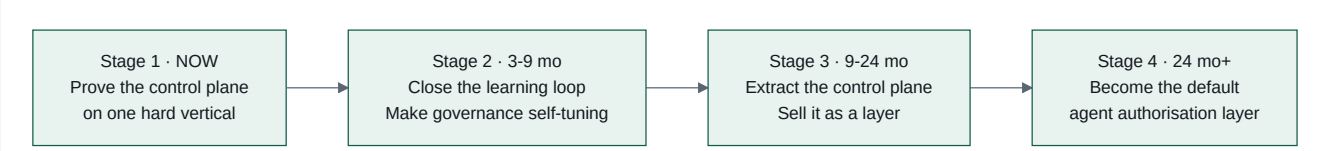
The three-part thesis

(1) Governance is the bottleneck, not capability. Model quality is a commodity that improves monthly without your effort. `routing.yaml` 's own header treats a model upgrade as *"one reviewed line."* What does not commoditise is the ability to answer a GC, a CFO, a CISO and an auditor in the same sentence. This codebase can.

(2) Prove it on a hard vertical first. Marketing content is a genuinely hostile test case: outputs are public, irreversible, brand-critical, legally exposed (client naming, unsupported claims), and cross-border (a US inference provider processing SA data). A control plane that survives *that* survives most enterprise use cases. The platform is already running it live.

(3) Then generalise the layer, not the application. `gatekeeper` , `model-gateway` , `publisher` , `vault` , `registry` and `telemetry-lib` contain **zero marketing logic**. The seam is not aspirational — it is a fact of the current file layout.

Strategic sequence



STAGE	OBJECTIVE	SUCCESS MEASURE
1 · Prove	A live, governed, measured AI pipeline	✅ largely achieved — daily loop live, real Anthropic calls, real metering, real approval cards. Remaining: activate the 20 packages, real publish, enterprise-grade auth
2 · Learn	Governance that tunes itself from its own evidence	An `autonomy.yaml` diff proposed automatically from approval history and merged by a human
3 · Extract	An SDK + control plane usable by any agent framework	A third-party agent routing its tool calls through the gate
4 · Default	The layer enterprises assume they need	Referenced in procurement checklists

3. Why this team can do it — evidence, not assertion

The strongest argument is not the platform. It is the **quality of judgement visible in the code**, which is what a technical investor or an acquiring CTO would actually assess:

EVIDENCE	WHAT IT DEMONSTRATES
The redaction firewall was **too strict to function**, then narrowed four times — each narrowing scoped to one pattern and one named content class, each recorded with its reasoning and its authoriser	Ability to operate a security control under production pressure without weakening it silently
Matched-pattern ids were changed from `fixture:{value}` to `fixture:{group}:{index}` because the original **echoed the client's real name into the audit column**	The kind of finding that only comes from adversarial review, caught and fixed
`write_audit_isolated()` exists because a rejection's own transaction rollback would have destroyed its audit row	Understanding of transactional semantics at the level that matters for compliance
`qa_blocked` given its own transition reason, its own DB CHECK value, and counted as smoke-test *success*	Distinguishing "the AI broke" from "the AI correctly said no" — most platforms conflate these
Migration 0003 root-caused as the true cause after a fix scoped to 0004 "had zero effect for exactly this reason: 0004 was never reached"	Root-cause discipline, not symptom-patching
`NOT_READY_MAX_REQUEUES = 20` tuned against **observed** ~14s production cadence, with the arithmetic against the smoke budget written out	Empirical tuning over theoretical defaults
Function 18-03 deliberately proof-light because positioning.md doesn't claim manufacturing	Epistemic honesty encoded as a testable property
`docs/accepted-risks.md` names four unclosed risks, including one flagged as *"not a user-approved risk acceptance ... flagged here explicitly so it is tracked rather than silently shipped"*	Institutional honesty
79 classified compound learnings with strengthening and recurrence history	A learning organisation, evidenced rather than claimed

A CTO evaluating an acquisition would read `.compound/index.md` and `docs/accepted-risks.md` before reading any code, and would conclude the team is unusually mature.

4. The moat

Ranked by durability:

M1 — Institutional judgement (most durable)

79 learnings across four classes, several marked as recurring three or four times. Learning L-0048's image-bootstrap contract independently reproduced on four separate Container Apps before being written down as a rule. **This is path-dependent knowledge — it cannot be copied, only re-earned through the same incidents.**

M2 — Governance depth

Five independent authorisation layers, all fail-closed, all tested. A competitor can build the same in months — but they will not know *which* controls matter until they have had the incidents. See M1.

M3 — Regional specificity

SA ID and phone regexes in a frozen contract. POPIA s11 lawful-basis modelling. POPIA s72 documented as explicitly unresolved rather than assumed away. `southafricanorth` pinned as a literal. South African English as a brand rule. **For a Johannesburg CFO evaluating US AI tooling, this is not a feature — it is the purchase criterion.** It is also, honestly, a ceiling: Frankfurt would need a rebuild.

M4 — Contract discipline

Ten hash-frozen contracts, CI-enforced, read *at runtime* by the services that must honour them. This makes the platform safely extensible by parallel teams — which is what made the parallel-session build model work at all.

M5 — The compound engineering method

The most valuable and the least productised. See §7.

5. What could kill this

RISK	LIKELIHOOD	SEVERITY	MITIGATION AVAILABLE IN-CODE
Cloud providers bundle governance into their agent platforms	High	High	Azure/AWS will do *model-level* safety. This does *action-level* authorisation with human approval bound to content hashes. Emphasise that distinction, and integrate rather than compete
The market doesn't price governance yet	Medium	High	Lead with cost control (`cost_per_accepted_asset`) — CFOs buy that today. Governance is the upsell
Single-tenant architecture blocks every SaaS model	**Certain**	High	Decide tenancy in Horizon 2, before more production data accumulates (`07-operating-model.md` §D.3)
The 20 unwired agents make demos hollow	**Certain today**	Medium	R1 registry-driven dispatch — 2–3 weeks
Regional moat becomes a regional ceiling	Medium	Medium	The redaction rules are a data contract; a `redaction-rules-eu.yaml` is additive, not a rewrite
Key-person dependency on the agent loop	Medium	High	The 79 learnings *are* the mitigation — they encode the method outside anyone's head. Productise it (§7)
Anthropic dependency	Medium	Medium	The extension point is built and untested. R12 exercises it and simultaneously solves POPIA s72 via in-region Azure OpenAI

6. The strategic decision that matters most

> **Is Canvas Marketing OS a product, or is it the proof for a different product?**

	IF IT'S THE PRODUCT	IF IT'S THE PROOF
Next 6 months	Build marketing depth: calendar UI, more channels, CRM integration, activate all 23 agents	Extract the SDK, add multi-provider, MCP-native integration, publish the governance model
Metric	Content volume, engagement, marketing seats	Agent actions governed, integrations, developers
Buyer	CMO	CTO / CISO / Head of AI
Comparable	Jasper, Writer, Typeface	Credo AI, LLMOps, "Vanta for AI agents"
Multiple	6–10× ARR	15–25× ARR
Risk	Crowded, well-funded, governance moat may not be priced	Category creation is slow and expensive
Code readiness	395 lines (1.9%)	**~20,800 lines (98%)**

The code has already answered this question. The recommendation is a sequenced both: keep Canvas Marketing OS as the flagship reference implementation and the first paying customer, and extract the governance layer as the product. The marketing platform becomes the demo that a governance platform desperately needs — a *live, hostile, public, irreversible* use case where the controls visibly matter.

7. The under-exploited asset

The `.compound/` learning system and the worktree-per-session agent loop that produced this repository are, strategically, the most interesting artefact here — and they are entirely unproductised.

What exists: a spec → plan → build → multi-lens-review → learn loop, with isolation per unit of work, frozen contracts as the coordination protocol between parallel sessions, a documented first-to-land-wins conflict rule, and a classified learning corpus that later specs cite as acceptance criteria.

What it proved: a fully-governed, POPIA-aware, cost-metered, human-gated, production-deployed AI platform on Azure — 917 files, 8 services, 5 schemas, 128 test files, 400 test functions, 13 CI/CD workflows, live in `cmos-dev`.

Why it matters commercially: every enterprise asking *"how do we use AI agents to build software safely at scale?"* is asking exactly the question this loop answers, and this repository is the answer with a working exhibit attached.

[INFERRED] Three options: sell it as a consulting practice ("compound engineering"); license the framework; or fold it into the governance product as *"the method we used to build the platform that governs your agents — and the same governance applies to your engineering agents."* The third is the most coherent, because it makes the product and the method the same story.

8. Vision statement

> **Canvas gives enterprises the confidence to let AI act.**
>
> Not to draft, not to suggest, not to assist — to **act**: to publish, to spend, to commit, to touch systems that matter. We do that by making every agent action a governed transaction: authorised against a policy a human wrote, approved by a named person against the exact bytes they saw, bound by a cryptographic token that can be used once, verified independently at the boundary, metered to the cent, and recorded forever — including every refusal.
>
> We proved it on the hardest thing we had: our own marketing, in public, with a US model provider, under South African privacy law.
>
> **Autonomy you can audit.**

17

Enterprise Vocabulary & Frameworks

Phase 4. For every major thing you have built, the language enterprise software vendors and management consultants use for it — so you can walk into a CIO, CFO or investor conversation and name your own architecture in their words.

Each entry follows the same shape: **What you built → "This is an example of…" → "The enterprise concept is called…" → "McKinsey would call this…" → "Gartner would classify this as…" → "This maps to…"**

---

1. `services/gatekeeper/policy/autonomy.yaml` — levels 0–4

This is an example of externalised authorisation — the decision about *what is allowed* is separated from the code that *does the thing*.

The enterprise concept is called a **Policy Decision Point (PDP)** and a **Policy Enforcement Point (PEP)**. Your PDP is `/gate-check`; your PEPs are the orchestrator's `request-approval` handler and the Publisher. This is the XACML architecture, and it's the same shape as AWS IAM (policy evaluated centrally, enforced at the resource) and Open Policy Agent.

McKinsey would call this a **Delegation of Authority (DoA) matrix**, or in an operating-model review, the *decision rights framework*. Their standard tool is RAPID® (Recommend, Agree, Perform, Input, Decide). Your levels map almost exactly: level 4 = the agent Decides; levels 1–2 = the agent Recommends and a human Decides; level 0 = the decision right does not exist. **You have machine-executable decision rights, which is what every operating- model redesign wants and none can enforce.**

Gartner would classify this as AI TRiSM (AI Trust, Risk and Security Management), specifically the *AI governance* and *runtime enforcement* pillars. Also relevant: their "Guardian Agents" framing (2025).

This maps to ISO/IEC 42001 (AI management systems) §8.3 operational control; the NIST AI RMF **GOVERN** function; the EU AI Act Art. 14 (human oversight); COBIT's EDM (Evaluate/Direct/Monitor).

Say this in a meeting: *"Our autonomy policy is a machine-enforced delegation-of-authority matrix. It's a policy decision point with fail-closed defaults, and every evaluation writes an immutable decision row."*

---

2. `contracts/` — ten hash-frozen files, read at runtime

This is an example of contract-first design with a **breaking-change guard**.

The enterprise concept is called Consumer-Driven Contract Testing and **Interface Versioning Governance**. Your `.frozen-v1.sha256` baseline is a *schema registry with compatibility enforcement* — the same role Confluent Schema Registry plays for Kafka.

The unusual part: `model-gateway/completion.py` reads the `CompletionRequest` schema **out of the frozen OpenAPI file at runtime** and validates against it. That is not documentation-as-code; that is **specification-as-runtime**.

McKinsey would call this an *architectural governance mechanism* enabling **parallel workstream execution** — the mechanism that let a swarm of independent sessions build against each other without integration hell.

Gartner would classify this as API Governance / Contract-First API Management, adjacent to their "Composable Enterprise" thesis (Packaged Business Capabilities with stable interfaces).

This maps to TOGAF's Architecture Contracts (Phase G); Bounded Context integration patterns in Domain-Driven Design (Published Language + Conformist); Postel's Law applied deliberately.

Say this in a meeting: *"Our interfaces are frozen contracts with a CI-enforced hash baseline. Services validate against them at runtime, not just at build time — so a contract drift is a failing request, not a silent divergence."*

---

3. The gate token — RS256, `jti`, `exp`, content-hash-bound

This is an example of a **capability token** (also: bearer capability, object capability) — the token *is* the authority, scoped to exactly one action on exactly one object.

The enterprise concept is called Proof-of-Possession / Sender- Constrained tokens (RFC 8705, DPoP), plus **transaction authorisation** — the pattern behind PSD2 Strong Customer Authentication's "dynamic linking" requirement, where the authorisation code must be cryptographically bound to the *specific amount and payee*, so approving one payment cannot authorise another.

Your `content_hash` binding is dynamic linking for AI actions. That is a precise, powerful analogy, and it comes from banking regulation — which means a regulated buyer already understands it.

McKinsey would call this a **four-eyes principle** (or *maker-checker*) with **non-repudiation**.

Gartner would classify this as fine-grained authorisation / **Externalised Authorisation Management (EAM)** — and increasingly under their "agentic AI identity" research.

This maps to NIST SP 800-63 (authentication assurance); the SWIFT CSP transaction-integrity controls; SOX ITGC change-authorisation controls.

Say this in a meeting: *"Approvals use dynamic linking — the same principle as PSD2. The authorisation token is cryptographically bound to a hash of the exact content approved, and the boundary service independently recomputes that hash from the raw bytes. Approve-A-publish-B is impossible."*

---

4. `services/model-gateway/redaction.py`

This is an example of an **egress data-loss-prevention proxy** with an audit trail.

The enterprise concept is called DLP (Data Loss Prevention) at the egress boundary. In AI-specific language: an **LLM firewall** or **AI gateway**. Your `content_class` mechanism is a **classification-driven policy exemption** — the caller declares a class, and a *gateway-side reviewed mapping* decides what that class relaxes. The caller never names a pattern.

McKinsey would call this a **preventive control** in the three-lines-of- defence model (first line: the control itself; your `gate_decisions` audit is the second line's evidence; the accepted-risks register is third-line input).

Gartner would classify this as AI TRiSM's *"AI runtime inspection and enforcement"* — the same category as Prompt Security, Lakera, Robust Intelligence.

This maps to POPIA s72 (cross-border transfer); GDPR Ch. V; ISO 27001 A.8.12 (data leakage prevention).

The vocabulary that will most impress a privacy officer: your rules file says outright that it *"is not itself a transfer-lawfulness mechanism ... Treat it as one control among several, never as the control."* That is **control-effectiveness honesty**, and it is the difference between a compliance posture and compliance theatre.

Say this in a meeting: *"We run an egress DLP firewall in front of the model provider. Pattern coverage is deliberately acknowledged as incomplete — which is exactly why every block writes an immutable audit row. It's a defence-in-depth control on top of the lawful-basis regime, not a substitute for it."*

---

5. `docs/permission-register.yaml` — default deny, absence ≠ permission

This is an example of an **allowlist with a proven-equivalent default-deny path**.

The enterprise concept is called fail-safe defaults (Saltzer & Schroeder, 1975 — principle 3 of 8). The subtle part you got right and most teams get wrong: *absence* and *explicit denial* must be observationally identical. Your `_self_test()` asserts exactly that equality.

McKinsey would call this a **control by design** rather than a control by detection — the difference between preventing an error and finding it later.

Gartner would classify this as *policy-based content governance*; in a marketing context, **brand safety controls**.

This maps to the principle of least privilege; NIST AC-3 (access enforcement); ISO 27001 A.5.15.

Say this in a meeting: *"Client naming is default-deny with proven equivalence: a name absent from the register blocks identically to one explicitly marked uncleared. We have a self-test that asserts those two paths produce the same outcome, because 'we forgot to add it' must never become 'therefore it's allowed'."*

---

6. `costs` rows + `budgets.yaml` + `cost_per_accepted_asset`

This is an example of chargeback/showback with **graceful degradation**.

The enterprise concept is called FinOps — specifically the emerging **AI FinOps** discipline. Your three-row metering (usd/tokens/ms) is *unit economics instrumentation*; your unbroken FK chain `costs → agent_runs → campaigns` is a **cost allocation model**; your soft-breach tier downgrade is **graceful degradation** or **brownout** (as opposed to a circuit breaker, which fails hard).

`cost_per_accepted_asset` is a cost-per-outcome metric, not a cost-per-unit metric. That distinction is worth a lot in a CFO conversation: token spend is an input measure; cost per *accepted* asset is an output measure, and it is the AI equivalent of **cost per acquisition** or **cost of quality**.

McKinsey would call this zero-based budgeting applied to AI capacity, and the metric itself a **productivity ratio** in a labour-substitution analysis. If they were being expansive: *"the unit economics of synthetic labour."*

Gartner would classify this as Cloud Financial Management extended to AI, and **AI cost governance**.

This maps to activity-based costing (ABC); the FinOps Foundation's Inform → Optimise → Operate phases; ITIL financial management.

Say this in a meeting: *"We meter every model call in three units and attribute it through an unbroken chain to a campaign. Our headline metric is cost per accepted asset — a cost-per-outcome measure, not cost per token. Budget breaches degrade the model tier rather than blocking work."*

---

7. `services/registry` — signed, reproducible, eval-gated agent packages

This is an example of treating prompts as **first-class software artefacts** with a supply chain.

The enterprise concept is called Model Governance and **AI Asset Lifecycle Management**. Your canonical-JSON manifest + detached signature is a **Software Bill of Materials (SBOM)** for AI assets, and the Ed25519 signature is **artefact provenance/attestation** — the same problem SLSA and Sigstore solve for container images.

Your prompt-coupled eval oracle (`tool_check.py` derives its output from `prompt.md` , so deleting a rule fails the test grading it) is **behaviour- coupled regression testing**, and the broken-copy fixture is **mutation testing** applied to prompts. Almost nobody does this.

McKinsey would call this model risk management — and would immediately reach for **SR 11-7**, the US Federal Reserve's model-risk guidance: model inventory, model validation, ongoing monitoring. You have all three primitives.

Gartner would classify this as ModelOps / LLMOps, adjacent to their AI TRiSM *model management* pillar.

This maps to SR 11-7; ISO/IEC 42001 §8.4 (AI system impact assessment); SLSA build provenance levels; the EU AI Act's technical documentation requirements (Annex IV).

Say this in a meeting: *"Prompts are versioned, signed, reproducible artefacts with golden evaluation suites. We have a model inventory, model validation and change control — the three primitives SR 11-7 asks for, applied to generative assets."*

---

8. `governance.kill_switches` — uncached, <5s, global or per-function

This is an example of a **circuit breaker with an operator override** — though strictly it is a *manual* breaker, not an automatic one.

The enterprise concept is called a **kill switch** or **emergency stop (E-stop)**, borrowed from industrial safety (ISO 13850). In IT terms: a **feature flag with a guaranteed propagation bound**.

Your design decision — *no cache at any TTL, because "a cache with any TTL above zero would make the 5s bound a function of cache expiry rather than of the operator's action"* — is the correct one, and it has a name: **strong consistency over availability for a safety-critical read** (the CAP trade-off, made deliberately).

McKinsey would call this a **business continuity control** and, in a risk-appetite discussion, the *tripwire* in a risk-limit framework.

Gartner would classify this as *AI incident response*, part of AI TRiSM's operational pillar.

This maps to the EU AI Act Art. 14(4)(e) — the human overseer's ability to "interrupt the system through a stop button"; NIST AI RMF MANAGE-4.

Say this in a meeting: *"We have a hard emergency stop, global or scoped to a single function, with a guaranteed sub-five-second propagation bound because it's an uncached read on every decision path. It's the Art. 14 stop button, implemented rather than described."*

---

9. `loops/*.yaml` + `uuid5` decomposition

This is an example of declarative workflow orchestration with **deterministic replay**.

The enterprise concept is called a **DAG-based workflow engine** (Airflow, Dagster, Temporal), and your `uuid5(event_id, task_id)` scheme is **deterministic id derivation**, which enables **idempotent replay** and **golden-file testing**.

The interaction between services is **choreography** (services react to events), not **orchestration** (a central conductor calls services). Your orchestrator is a *decomposer and dispatcher*, not a conductor — a meaningful distinction in an architecture review.

McKinsey would call this process standardisation and, in an operating-model context, *straight-through processing (STP)* — the banking term for a transaction that completes without manual intervention. Your level-3/4 functions are STP; your level-1/2 functions are *exception handling*.

Gartner would classify this as Hyperautomation — specifically an *Intelligent Business Process Management Suite (iBPMS)* with AI task workers.

This maps to BPMN 2.0 (your loop YAML is a BPMN process definition in different syntax); the Saga pattern (long-running processes with compensating actions — though you have no compensations); Event-Driven Architecture.

Say this in a meeting: *"Workflows are declarative DAGs with deterministic task-id derivation, so the same trigger always produces the same run. It's straight-through processing with explicit exception routing at the publication gate."*

---

10. Vault taxonomy — 6 mandatory, immutable fields

This is an example of mandatory metadata at the point of creation, with immutability enforcement.

The enterprise concept is called a **data classification scheme** and, collectively, an **information governance framework**. `retention_class` is a **records retention schedule**; `consent_status` is **consent state management**; `evidence_grade` is a **data quality / provenance dimension**.

Making them **immutable post-create** is the key move, and it has a name: **write-once metadata** or *provenance immutability*. It prevents reclassification-after-the-fact, which is exactly the failure mode an auditor looks for.

McKinsey would call this data governance by design and would place your six fields in a **data product** framing (Data Mesh: every data product carries mandatory metadata contracts).

Gartner would classify this as Metadata Management and **Data Governance**, adjacent to their *Active Metadata* and *Data Fabric* research.

This maps to DAMA-DMBOK's Data Governance and Metadata Management knowledge areas; ISO 15489 (records management); the Data Mesh data-product contract.

Say this in a meeting: *"Every object carries six mandatory, immutable governance fields set at creation — vertical, function, campaign, evidence grade, consent status and retention class. Retention and evidentiary weight are properties of the record itself, not of a policy someone applies later."*

---

11. `.compound/learnings/` — 79 classified learnings

This is an example of organisational knowledge capture with **reinforcement signals** (your "strengthened/recurred N times" annotations).

The enterprise concept is called Knowledge Management (Nonaka & Takeuchi's SECI model — you are systematically converting *tacit* incident knowledge into *explicit* codified rules). More narrowly: **blameless post-incident review** with a permanent, searchable corpus.

The recurrence counters are the sophisticated part: a learning marked "3rd recurrence" is telling you the *control is not working*, not that the lesson is popular. That is **control-effectiveness monitoring**.

McKinsey would call this a **learning organisation** (Senge), and operationally a **continuous improvement (kaizen) loop** with a *lessons- learned register*. In a Capability Maturity framing this is **CMMI Level 5 — Optimising** behaviour: quantitative feedback used to systematically improve the process itself.

Gartner would classify this as *Knowledge Graph for Engineering* or DevOps *institutional memory*, and they would be under-selling it.

This maps to ITIL Problem Management (known error database); ISO 9001 §10 (improvement); the Toyota Production System's *yokoten* (horizontal deployment of learning).

Say this in a meeting: *"We run a compound learning system — 79 classified engineering learnings with recurrence tracking. A learning marked as a third recurrence tells us a control isn't working, which is a different signal from a lesson being useful. Later specifications cite prior learnings as acceptance criteria."*

---

12. The worktree-per-session build model

This is an example of isolated, parallelised delivery with **contract-mediated integration**.

The enterprise concept is called trunk-based development with short-lived branches, plus **Team Topologies**' *stream-aligned teams* and *enabling constraints*. Your frozen contracts are the **X-as-a-Service interaction mode** between sessions, and your "first-to-land wins" conflict rule is a **coordination protocol**.

McKinsey would call this an **agile operating model at scale** — specifically, *loosely-coupled, tightly-aligned* squads with clear interfaces and minimal coordination overhead. The frozen contracts are what removes the "integration tax".

Gartner would classify this as *Platform Engineering* — the contracts and CI checks are your **golden path**.

This maps to Conway's Law (deliberately inverted — you designed the interfaces first and let the work organise around them); Team Topologies' cognitive-load management; the Inverse Conway Manoeuvre.

Say this in a meeting: *"We run an inverse Conway manoeuvre — frozen contracts define the seams, and isolated parallel sessions build against them. Integration cost is near zero because the interfaces can't drift; CI enforces a hash baseline."*

---

13. Content-addressed asset storage

This is an example of content addressing with deduplication and reference counting.

The enterprise concept is called Content-Addressable Storage (CAS) — the same principle behind Git objects, IPFS, and Docker layers. Combined with your hash-bound gate token, it gives you **immutable, verifiable artefact identity**.

Gartner would classify this as part of *Digital Asset Management* with integrity assurance.

This maps to WORM (Write Once Read Many) storage for compliance; SEC Rule 17a-4 for records that must be non-rewriteable.

---

14. The dual-surface console (HTML + JSON from one service layer)

This is an example of API-first design with progressive enhancement.

The enterprise concept is called Headless architecture or **API-first**, plus **content negotiation** (HTTP `Accept`). In the AI context, your framing is better: **agent-native**.

Gartner would classify this as *Composable / MACH architecture* (Microservices, API-first, Cloud-native, Headless).

Why it matters strategically: when every human surface has a machine equivalent, an agent can operate the platform. That is a prerequisite for the learning loop in `10-product-roadmap.md` R2 — the feedback service will read the console's JSON, not scrape its HTML.

---

15. `qa_blocked` — a business verdict is not a failure

This is an example of distinguishing **expected exceptions** from **system faults**.

The enterprise concept is called the difference between a **business exception** and a **technical exception** — a distinction that shows up in every serious BPM and integration architecture, and that most systems get wrong by mapping both onto "error".

McKinsey would call this *first-pass yield* vs *defect rate* in a Six Sigma frame. A QA block is a *rework loop*, not a defect in the process.

Why this is genuinely rare: your smoke test counts a legitimate `QA_BLOCKED` verdict as **proof of life**, because reaching a real verdict proves the whole chain worked. That is an unusually mature definition of "green", and it comes from understanding that a system whose safety controls never fire is indistinguishable from one whose safety controls are broken.

Say this in a meeting: *"We separate business verdicts from technical failures at the state-machine level. A QA block has its own transition reason and its own terminal state — and our end-to-end smoke test treats a legitimate block as a pass, because a control that never fires is a control you can't prove works."*

---

16. Quick-reference translation table

YOU BUILT	ENTERPRISE TERM	CONSULTANT TERM	GARTNER CATEGORY
`autonomy.yaml`	Policy Decision Point	Delegation of Authority matrix	AI TRiSM — governance
Gate token + hash binding	Sender-constrained capability token / dynamic linking	Four-eyes with non-repudiation	Externalised Authorisation
Redaction firewall	Egress DLP / LLM firewall	Preventive control, 1st line of defence	AI TRiSM — runtime enforcement
Permission register	Fail-safe defaults	Control by design	Brand safety controls
Costs + budgets	Chargeback + graceful degradation	Zero-based budgeting; productivity ratio	AI FinOps
Registry + evals	AI SBOM + model validation	Model risk management (SR 11-7)	ModelOps / LLMOps
Kill switch	Emergency stop	Business continuity tripwire	AI incident response
Loop YAML DAG	Declarative workflow orchestration	Straight-through processing	Hyperautomation / iBPMS
6-field taxonomy	Data classification + retention schedule	Data governance by design	Metadata Management
`gate_decisions`	Immutable audit log	Evidence for 2nd line of defence	Continuous Controls Monitoring
Frozen contracts	Schema registry with compatibility guard	Architectural governance for parallel workstreams	API Governance
`.compound/learnings`	Known-error database	Learning organisation / CMMI L5	Institutional memory
Worktree-per-session	Trunk-based dev + contract-mediated integration	Loosely-coupled, tightly-aligned squads	Platform Engineering
Content-addressed blobs	Content-Addressable Storage	—	DAM with integrity assurance
Dual-surface console	API-first / headless	—	Composable / MACH
`qa_blocked`	Business vs technical exception	First-pass yield vs defect rate	—
Proof circuit	Canary transaction with poison pill	Production verification testing	Synthetic monitoring
Cascade dead-letter	Fail-fast on unsatisfiable precondition	—	—
`write_audit_isolated`	Out-of-band audit durability	—	—

---

17. The three sentences to memorise

For a CIO:

> *"It's an externalised authorisation layer for AI agents. Policy decision point, fail-closed defaults, sender-constrained capability tokens with dynamic content binding, independent verification at the egress boundary, and an immutable decision ledger that records refusals as first-class events."*

For a CFO:

> *"Every model call is metered in three units and attributed through an unbroken chain to a campaign. Budgets are per-function with graceful degradation rather than hard stops. Our headline metric is cost per accepted asset — a cost-per-outcome measure, not cost per token."*

For an investor:

> *"Ninety-two percent of the code is governance. We built a control plane for AI agents and proved it on the hardest use case we had — public, irreversible, brand-critical marketing content, through a US model provider, under South African privacy law. The marketing platform is the exhibit. The control plane is the product."*

18

The Marketing Code, Analysed

Every line of executable marketing domain logic in the platform lives in one file. This document maps it, compares the five handlers structurally, and identifies what is genuinely marketing-specific versus what is a repeated pattern waiting to be extracted.

Companion extract: `_marketing-code/dispatch-marketing-extract.py` — a read-only, banner-annotated copy of the region with original line numbers preserved, for reading without the surrounding 100 lines of dispatch/gating machinery.

---

1. Where it is

`services/orchestrator/orchestrator/dispatch.py` — 1,138 lines total.

LINES	SECTION	TOTAL	NON-COMMENT	MARKETING-SPECIFIC?
62–232	Constants + client factories	171	~125	**partly** — function ids, proof-circuit tags
235–317	Shared helpers (prompt read, JSON parse, complete+meter)	83	~70	no — generic
320–399	Ingest redaction fallback	80	~77	**yes** — but only because of ingest's content
402–440	`resolve_lineage_result`	39	~34	no — generic DAG memory
443–558	**HANDLER 1 · ingest-signals** (fn 09, haiku)	116	~99	**yes**
561–677	**HANDLER 2 · draft-brief** (no LLM)	117	~86	**yes**
680–856	**HANDLER 3 · qa-review** (fn 02, sonnet)	177	~93	**yes**
859–949	**HANDLER 4 · draft-content** (fn 42, sonnet)	91	~72	**yes**
952–1023	**HANDLER 5 · request-approval** (no LLM)	72	~49	**no** — pure governance
1026–1138	`DISPATCH_TABLE`, pass-through, not-ready/cascade gate	113	~93	no — generic

395 executable lines across the five handlers. The file is **37% comments** — an unusually high ratio, and most of it is root-cause narrative from live incidents, which is the reason this file is hard to read but valuable to keep.

2. The five handlers, compared

Call sequences extracted mechanically, not from reading:

```
ingest-signals          draft-brief           qa-review             draft-content         request-approval
──────────────          ───────────           ─────────             ─────────────         ────────────────
_load_fetch_sources     resolve_lineage       load_permission_check —                     resolve_lineage
build_mcp_web_client    —                     resolve_lineage       —                     —
mcp.call_tool ×4        —                     —                     —                     —
build_vault_client      build_vault_client    build_vault_client    build_vault_client    —
get_or_create_campaign  get_or_create_campaign get_or_create_campaign get_or_create_campaign —
—                       vault.get_signal      get_asset / get_brief —                     —
—                       _render_brief         —                     —                     —
create_agent_run        create_agent_run      create_agent_run      create_agent_run      —
_read_prompt            —                     _read_prompt          _read_prompt          —
emit_task_span          emit_task_span        emit_task_span        emit_task_span        emit_task_span
build_gateway_client    —                     build_gateway_client  build_gateway_client  build_gatekeeper_client
_complete_*             —                     _complete_and_meter   _complete_and_meter   gatekeeper.gate_check
_parse_json_content     —                     _parse_json_content   _parse_json_content   —
vault.create_signal     vault.create_brief ×2 —                     vault.create_asset    —
update_agent_run        update_agent_run      update_agent_run      update_agent_run      —
set_result_ref          set_result_ref        set_result_ref ×2     set_result_ref        set_result_ref
db.transition           db.transition         db.transition ×2      db.transition         db.transition
advance_dependents      advance_dependents    advance_dependents    advance_dependents    advance_dependents
```

Handlers 1, 3 and 4 are the same function with different parameters. Handler 2 is that function minus the LLM call. Handler 5 is a different animal entirely — it makes a governance call and produces no artefact.

The canonical shape (12 steps)

```
resolve input          # lineage result_ref | config YAML | module constants
vault.get_or_create_campaign(name, function_id)
vault.create_agent_run(agent_name, campaign_id, function_id, "running", input_payload)
prompt = _read_prompt(function_dir)
user_content = json.dumps({...})
with emit_task_span(task_type, function_id, task_ref, model, run_id) as span:
    response, cost = _complete_and_meter(gateway, vault, model=..., agent_run_id=...)
    set_span_attribute(span, "cost", cost)
    output = _parse_json_content(response["content"])
    artefact = vault.create_<type>(...)
    vault.update_agent_run(id, "succeeded", output, completed_at)
db.set_result_ref(task_id, {...})
db.transition(task_id, COMPLETED, COMPLETED)
db.advance_dependents(task_id)
```

What actually varies

AXIS	INGEST-SIGNALS	DRAFT-BRIEF	QA-REVIEW	DRAFT-CONTENT
`function_id`	`09-market-intelligence-director`	`brief.compose`	`02-brand-steward-qa`	`42-linkedin-post-writer`
model tier	`claude-haiku`	*none*	`claude-sonnet`	`claude-sonnet`
input source	`fetch_sources.yaml` + mcp-web	lineage → `get_signal`	lineage → `get_brief` / `get_asset`	3 module constants
`user_content` shape	topic + horizon + fetched bodies	*n/a*	{draft_text, client_references, channel}	{pillar, proof_point, campaign}
`content_class`	`public_source_content`	*n/a*	`public_source_content`	*none*
artefact written	`create_signal`	`create_brief` ×2	*none*	`create_asset`
`result_ref` keys	`vault_signal_id, topic`	`brief_id, executive_brief_id`	`pass, violations` | `vault_asset_id`	`vault_asset_id, content_hash`
terminal branch	—	—	**FAILED** / **`qa_blocked`**	—
agent_name	`market-intelligence-director`	`brief-writer`	`brand-steward-qa`	`linkedin-post-writer`

Nine axes of variation. Every one of them is data. Eight could be declared in a manifest; only the qa-review terminal branch is behaviour.

3. What is genuinely irreducible

Not everything here generalises, and it is worth being precise about which parts do not:

(a) qa-review's terminal branch (L830–842). A `pass: false` verdict transitions to `FAILED` with reason `QA_BLOCKED` and deliberately **does not call `advance_dependents`** — so the downstream approval task can never see the asset. That is a distinct control-flow outcome, not a parameter. Any generic handler needs an explicit "verdict handler" concept, not just a schema.

(b) qa-review's lineage-dependent branch (L760–772). It inspects `ancestor_task["task_type"]` to decide whether it is reviewing a brief or a LinkedIn draft, and sets `channel` accordingly — which then changes *which of function 02's rules apply* (`internal-brief` exempts CTA and UTM checks). This is one handler serving two loop positions with different semantics.

(c) ingest-signals' redaction fallback (L320–399). 80 lines of drop-one-source-and-retry logic that exists solely because real news prose trips the `full-name-like` pattern. It is marketing-specific only by accident — the underlying problem is "unstructured third-party text meets a regex firewall", which any ingestion path will hit.

(d) `_render_brief` (L566–597). The only deterministic content generation in the platform. 32 lines, no LLM, byte-identical for identical input. Note the deliberate choice inside it: sources are cited **by domain only**, never the bare URL, so that function 02's customer-facing link rules don't fire on an internal citation.

4. Coupling — what the marketing code reaches for

```
dispatch.py handlers
  ├── orchestrator.clients.gateway_client     → model-gateway  (HTTP)
  ├── orchestrator.clients.vault_client_ext   → vault          (HTTP)
  ├── orchestrator.clients.gatekeeper_client  → gatekeeper     (HTTP)
  ├── orchestrator.clients.mcp_client         → mcp-web        (HTTP)
  ├── orchestrator.db                         → task_state     (Postgres)
  ├── orchestrator.telemetry_wiring           → App Insights
  ├── orchestrator.teams_notify               → Teams webhook
  ├── config.functions_dir()                  → prompt.md files on disk
  └── importlib → functions/02/permission_check.py   (dynamic, L218–232)
```

Every dependency is injected through a **module-level factory** — `build_gateway_client()`, `build_vault_client()`, `build_gatekeeper_client()`, `build_mcp_web_client()` — deliberately, so a test can substitute exactly one without faking an httpx transport chain. That is good design and it is what makes this region testable in isolation.

The one non-obvious coupling: `load_permission_check()` (L218–232) dynamically imports `functions/02-brand-steward-qa/permission_check.py` by path, because a digit-prefixed directory cannot be dotted-imported (learning L-0039). It is reused **by reference, never forked** — so the deterministic uncleared-client check in the orchestrator is literally the same code the eval harness grades. That is the right call and worth preserving through any refactor.

5. Analysis hooks — where to look first

If you are auditing this region, these are the highest-yield places:

#	WHERE	WHAT TO LOOK AT
A1	L707, L786–788	`client_references` is passed as a **hard-coded empty list**. The deterministic uncleared-client check therefore always passes trivially. The mechanism is real and tested; the data feeding it is not wired. (TD-28)
A2	L760–772	Both branches set `content_class = "public_source_content"`, so the `full-name-like` redaction pattern is exempted for *all* qa-review calls. Check whether that is still the intent.
A3	L830–843	The `qa_blocked` path. Confirm `advance_dependents` is genuinely unreachable here — this is the control that stops an unapproved asset reaching the gate.
A4	L923–924	`output["post"]` — direct key access on model output with no `.get()`. A schema-conformant-but-key-missing response raises `KeyError`, which becomes a generic dispatch failure rather than a clear parse error.
A5	L466–486	`ingest_signals_handler` tolerates individual `fetch_url` failures but raises only if **all** fail. With mcp-web in fixture mode (TD-31) all four "succeed" with placeholder text.
A6	L274–317	`_complete_and_meter` swallows cost-lookup failures by design (span cost stays 0.0). Confirm no caller treats `cost == 0.0` as meaningful.
~~A7~~	—	~~`draft_brief_handler` 's ~~`vault_signal_id` ~~lookup depends on ~~`resolve_lineage_result` ~~walking **two** hops past the ~~`score_signals` ~~pass-through.~~ **Resolved:** `score-signals` has a real handler and sets its own `result_ref`, so the walk stops at its immediate predecessor. That `result_ref` is a deliberate superset of `ingest`'s and must stay one — see `score_signals_handler`'s own comment.
A8	L866–876	`DRAFT_CONTENT_PROOF_POINT`, `DRAFT_CONTENT_PILLAR`, `DRAFT_CONTENT_CAMPAIGN_UTM` — the entire input to function 42 is three module constants. This handler cannot produce a second, different post.

6. The extraction argument

The generalisation case is not speculative — it is visible in the shape table above. A registry-driven handler would need this signature:

```
def generic_function_handler(task_id, envelope, db, spec: FunctionSpec) -> None:
    """spec comes from the signed registry manifest, not from Python."""
```

with `FunctionSpec` carrying exactly the nine varying axes:

```
@dataclass(frozen=True)
class FunctionSpec:
    function_id: str
    agent_name: str
    model: str | None               # None => deterministic, no gateway call
    input_source: InputSource       # LINEAGE | CONFIG_YAML | CONSTANTS
    input_builder: str              # name of a registered user_content builder
    content_class: str | None
    artefact: ArtefactSpec | None   # vault method + field mapping
    result_ref_keys: tuple[str, ...]
    verdict: VerdictSpec | None     # the qa-review terminal-branch case
```

That collapses handlers 1, 3 and 4 into one function, keeps handler 2 as the `model=None` case, and leaves handler 5 (`request-approval`) where it is — it is governance, not content production, and should not be forced into the same abstraction.

The payoff is not line reduction — it is that adding an agent stops being a code change. Today, wiring one of the 20 inert packages requires editing `DISPATCH_TABLE` , editing the Dockerfile's staging list, and editing a loop YAML. Under a registry-driven handler it is a manifest entry.

Before doing this, add the guard test that makes the current gap visible: assert every `task_type` in any `loops/*.yaml` either has a `DISPATCH_TABLE` entry or sits on an explicit intentional-pass-through allowlist. That test converts TD-01 from invisible to loud, and it is the regression net the refactor needs.

7. Reading order

For a first pass, read in this order rather than top to bottom:

1. **L1091–1138** `dispatch_task` — the entry point and the not-ready/cascade gate. Understand what must be true before a handler runs.
2. **L402–440** `resolve_lineage_result` — how a handler finds its input. Everything else depends on this.
3. **L878–949** `draft_content_handler` — the shortest complete example of the canonical shape.
4. **L443–558** `ingest_signals_handler` — the same shape with external I/O and the redaction fallback.
5. **L694–856** `qa_review_handler` — the same shape plus a terminal branch and a lineage-dependent rule set. The most complex of the five.
6. **L561–677** `draft_brief_handler` — the deterministic case.
7. **L952–1023** `request_approval_handler` — the governance case.

Skip the comment blocks on the first pass; return to them when something looks arbitrary, because they almost always explain a live incident.

19

Live Verification Log

*A running record of claims in this documentation set that have been checked against the **live external systems themselves**, rather than inferred from the source tree. Each entry records the date, the method, and the outcome.*

Why this exists. The rest of this documentation set was reverse-engineered from source, and `README.md` states the resulting limitation plainly: *"Live Azure state was not inspected. Deployment claims rest on Bicep, on workflow definitions, and on comments recording live verification."*

Connected MCP servers do not remove that limitation — they give no Azure access. What they give is **counterparty verification**: the ability to check the platform's claims about an external system *from that system's side*. Where a constant in the code asserts something about Buffer, Canva or Microsoft, that assertion can now be tested rather than trusted.

Scope discipline. Only read-only calls are used. Nothing in this log writes to, schedules on, or mutates any external account.

Legend

STATUS	MEANING
✅ **Confirmed**	Live system agrees with the code
⚠️ **Diverged**	Live system disagrees — the code or the doc is wrong
❌ **Refuted**	The claim is false
⏳ **Pending**	Identified as checkable, not yet run

2026-08-06 · Buffer

Method: `get_account`, `list_channels` via the Buffer MCP connector, organisation *Canvas Intelligence* (`68e5f2187fe9a5263a3509ab`).

#	CLAIM IN THE CODEBASE	WHERE	LIVE RESULT	STATUS
B1	`BUFFER_ORG_ID = "68e5f2187fe9a5263a3509ab"`	`publisher/app/config.py` L44	account's sole organisation id is `68e5f2187fe9a5263a3509ab`	✅
B2	**"Buffer's free-tier plan caps queued posts at 10"** — flagged in code as *"an ASSUMPTION sourced from the GOAL text, not independently verifiable from any file in this repo (DE-3)"*	`publisher/app/config.py` L29–33	`limits.scheduledPosts: 10`	✅ **assumption promoted to verified fact**
B3	`BUFFER_LINKEDIN_CHANNEL_ID = "68e73facca3a4e6b746d17b4"`	`publisher/app/config.py` L43	channel `68e73facca3a4e6b746d17b4`, `service: linkedin`, `Canvas Intelligence`	✅
B4	Code comment maps `Facebook=68e74731ca3a4e6b746d2469`	`publisher/app/config.py` L40	channel `68e74731ca3a4e6b746d2469`, `service: facebook`	✅
B5	Code comment maps `X=68e745c6ca3a4e6b746d22b2`	`publisher/app/config.py` L41	channel `68e745c6ca3a4e6b746d22b2`, `service: twitter`	✅
B6	The three channel ids in the weekly loop's `channel_ids`	`loops/weekly-content-loop.yaml`	all three exist, all `isDisconnected: false`	✅
B7	Weekly cap of 8 keeps "headroom below the ceiling" of 10	`loops/weekly-content-loop.yaml`	ceiling confirmed at 10; 8 is genuinely below it	✅
B8	Logic App triggers use `South Africa Standard Time`	`infra/modules/scheduling/*.bicep`	account timezone `Africa/Johannesburg`	✅ consistent

B2 is the one that matters

`publisher/app/config.py` carries an unusually candid comment: the free-tier cap of 10 was taken from the GOAL text and explicitly marked as **not independently verifiable from any file in this repo**. It is now verified. The number is right, and the mitigation built around it — a live `list_queue` count check before every create, rather than trusting the constant — was the correct call regardless.

B3–B5 close a known transposition risk

The same file records that *"the GOAL text mistakenly used the X channel id as the LinkedIn id"*, and that all three ids plus the org id were mapped defensively so the error "can never recur". The live account confirms the code's mapping is correct in all three positions: LinkedIn, Facebook and X each resolve to the id the comment assigns them.

That was a documented near-miss with no external confirmation available at the time. It now has one.

2026-08-06 · Buffer — post history (P1, P2)

Method: `list_posts` (read-only, first 100, `hasNextPage: true` — so this is a sample, not the full history), organisation *Canvas Intelligence*.

FIELD	RESULT
posts sampled	100 (more exist)
status	76 `sent`, 24 `scheduled`
`via`	98 `buffer`, 2 `network` — **zero created via API**
createdAt range	2026-01-19 → 2026-07-29
sentAt range	2026-03-04 → **2026-08-05** (the day before this check)
dueAt range	2026-03-04 → 2026-09-17
scheduled, by channel	8 LinkedIn · 8 Facebook · 8 X

P1 — ✅ Confirmed: the platform has never published

Every post in the sample was created through Buffer's own interface (`via: buffer`) or by the network (`via: network`). **None was created through the API**, which is the only route `mcp-buffer` 's `create_draft` can take.

This independently confirms what `00` and `09` previously asserted from `PUBLISHER_DRY_RUN` defaulting true and the proof-circuit's forced dry-run. The claim no longer rests on reading a config default.

P2 — ❌ Refuted, favourably: there is abundant real Buffer data

`09` TD-11 says 3 of 4 analytics sources are fixtures and only Buffer goes live. That stands — and the live one has **76 sent posts across five months** to ingest, not an empty account. The Buffer slice of the KPI rollups would carry real data the moment the nightly job runs against it.

V1 — The governed pipeline and the real marketing workflow are disjoint

This is the most consequential thing the live connection revealed.

Marketing at Canvas Intelligence is happening — actively, on three channels, with a post sent as recently as the day before this check and 24 more scheduled out to September. It is being done **by humans, in the Buffer UI, entirely outside this platform.**

The platform exists to govern marketing publishing. It is not in the path of any actual marketing publishing.

Everything in `14` 's five-layer authorisation chain — autonomy policy, human approval, signed gate token, boundary hash verification, replay ledger — governs a path that has never carried a real post. Meanwhile the path that carries every real post has no autonomy policy, no gate token, no content-hash binding, no `publish_attempts` row, and no kill switch.

That is not a defect in the code. It is a **statement about adoption**, and it reframes the roadmap: `10` R1 (activate the 20 agents) and R9 (multi-channel) are not merely capability work — they are what would bring the real workflow inside the governed one. Until then the control plane is, in the strict sense, unexercised in production.

V2 — The codified brand rules have never been tested against real output

Function 02's rules were run against the 100 real posts. They are stated in `functions/02-brand-steward-qa/prompt.md` as blocking violations.

FN 02 RULE	RESULT AGAINST 100 REAL PUBLISHED POSTS
`link-shortener` — bans `bit.ly` , `lnkd.in` , `tinyurl.com` , `ow.ly` , `buff.ly`	**86 posts would FAIL** — 85 contain `bit.ly` , 1 contains `lnkd.in`
`url-utm` — every URL must be a `canvasintelligence.com` link with 3 UTM params	12 posts carry a Canvas link; only 4 carry any `utm_` → **8 would FAIL**
`sa-english-spelling` — no US variants	`center` ×3, `behavior` ×4 → **would FAIL**
Roof line (fn 42) — must close on `Your Data. Delivered.`	all 6 occurrences read **`Your data. Delivered.`** — lowercase *d*

Read this as a measurement, not an accusation. There are two honest readings and only the CMO can choose between them:

1. **The rules are the to-be state.** They describe the brand discipline the platform is meant to enforce once it is in the path, and current output predates it. Legitimate — but then the rules have never been validated against anything, and the `link-shortener` rule in particular would block 86% of what the brand actually publishes today.
2. **The published content is off-brand.** In which case the platform has been correctly encoding a standard nobody is currently meeting, and the gap is the point.

Either way: **the platform's brand policy and the brand's actual output have never been compared until now.** The `bit.ly` finding is the sharpest — a link shortener is not an accident of tooling here (Buffer's own shortener is `buff.ly` , which appears zero times), so `bit.ly` is a deliberate, systematic choice that the codified policy names as a blocking failure.

The roof-line casing divergence is small but telling: function 42's prompt mandates `Your Data. Delivered.` and every real post writes `Your data. Delivered.` One of the two is wrong, and it is a one-character fix in whichever place is wrong.

Recommended action: run function 02's `safety_suite.py` over an export of real published posts as a one-off calibration exercise, before the platform is put in the publishing path. It is the cheapest possible validation of the brand rules, it needs no new code, and it would have surfaced all four of these divergences.

2026-08-06 · Microsoft documentation (P3, P4)

Method note, stated up front because it affects how much weight these carry: the Microsoft Learn MCP connector returned `MCP error -32003: MCP tool call requires approval` on every attempt, and direct fetches of `learn.microsoft.com` return **HTTP 403** through this environment's proxy — for both `WebFetch` and `curl` . Both answers below were therefore obtained by **web search over the Microsoft Learn corpus**, which returned the relevant Learn pages and their wording, rather than by fetching those pages directly.

That is a weaker citation than reading the page. It is recorded as such. The underlying pages are named so anyone with unproxied access can confirm in a minute:

- `learn.microsoft.com/en-us/azure/azure-resource-manager/templates/deployment-modes`
- `learn.microsoft.com/en-us/azure/key-vault/keys/about-keys-details`

P3 — ✅ Confirmed: incremental mode is incremental *per resource*, not *per property*

The question behind TD-31: when a Bicep/ARM deploy re-applies `infra/modules/mcp/container-app.bicep` , does its `env` list **replace** whatever is on the live Container App, or merge with it?

Microsoft's `deployment-modes` documentation addresses this directly, and names the exact mistake:

> A common misunderstanding is to think properties that aren't specified in the template are left unchanged. If you don't specify certain properties, Resource Manager interprets the deployment as overwriting those values. Properties that aren't included in the template are reset to the default values.

and, on redeployment specifically:

> When redeploying an existing resource in incremental mode, all properties are reapplied. The properties aren't incrementally added.

The distinction that matters: **incremental mode is incremental at the resource level** — resources absent from the template are left alone — **but the body of a resource that *is* in the template is a full replacement.** The template "always contains the final state of the resource. It can't represent a partial update."

Applied to this repo, `infra/modules/mcp/container-app.bicep` L116:

```
env: concat(envVars, keyVaultSecretEnv)
```

`env` is an array property whose value is computed entirely from template inputs. On redeploy it is set to exactly that computed list. Any variable set on the live app by any other means — portal, `az containerapp update`, a hand edit — is **not** in `envVars`, is therefore not in the computed list, and is therefore dropped.

And `MCP_WEB_LIVE_MODE` is not in `envVars`. Re-verified this session:

```
grep -rn "MCP_WEB_LIVE_MODE" --include=*.bicep --include=*.yml \
  --include=*.yaml --include=*.py --include=*.sh .
```

returns **only** function-package `tools.yaml` files describing the variable's *effect*. It appears in no Bicep file, no workflow, and no script. It exists solely as documentation of a variable nothing declares.

TD-31's mechanism is confirmed. The finding no longer rests on one person's reading of an IaC template. What remains unverifiable from here is the *current state* — whether the variable is set on `ca-mcp-web` right now — which needs `az` access. So the correct phrasing of TD-31 is unchanged and now properly grounded: *if* it is set, the next full infra deploy silently removes it, and knowledge intake reverts to a synthetic fixture while `caj-mcp-smoke` continues to pass.

P4 — ✅ Confirmed: Key Vault has no Ed25519 key type, at any tier

The claim under test appears in three places in the codebase:

WHERE	WHAT IT SAYS
`services/publisher/app/verifier.py` L17–19	"RS256 only. The contract also allows ES256/PS256/EdDSA, but EdDSA is unavailable on a standard-tier Key Vault (no Ed25519 key type at any SKU)"
`services/publisher/app/config.py` L7–8	"See app/verifier.py for why RS256 and not EdDSA (this Key Vault SKU has no Ed25519 key type at all)"
`.compound/index.md` L-0031	"Azure Key Vault standard tier cannot create/sign/verify EdDSA (Ed25519) keys — RSA (RS256/PS256) and EC P-256/P-384/P-521/P-256K (ES256/ES384/ES512) only"

Microsoft's supported-curve list is exactly the four the codebase names: **P-256, P-256K (SECP256K1), P-384, P-521**, alongside RSA. Ed25519 is absent. Azure CLI rejects it explicitly — `az keyvault key create --curve Ed25519` returns *"Unsupported curve: Ed25519. Supported curves are P-256, P-384, P-521, P-256K, SECP256K1"* — and the corresponding `azure-cli` issue records that this holds on a **premium** vault and on **Managed HSM** too, not only on standard tier.

So the code's reasoning is correct, and slightly *understated*: this is not a SKU limitation that a tier upgrade would lift. **L-0031 and TD-25's remediation path are both validated**, and option (a) in `docs/accepted-risks.md` — switch `signing.py` / `verify_signature.py` to ES256 with the production key held in Key Vault — is the right default.

What P4 also surfaced

Two things worth recording, neither of which I expected going in.

First, the platform runs two signing schemes, and the split is principled. Gate tokens are RS256 (`publisher/app/verifier.py`) because Key Vault holds the key. The registry artefact is Ed25519 (`services/registry/signing.py`, `SIGNATURE_ALGORITHM = "Ed25519"`, deterministic per RFC 8032) because the build holds the key itself, in software, via `cryptography`. That is not inconsistency — **the algorithm choice follows key custody**, which is the correct way round. `14` should say so explicitly; at present it documents the two schemes separately without naming the reason they differ.

Second, a doc-drift item. `services/registry/signing.py`'s module docstring (L7–11) explains the `keyvault://` fail-loud path with only the *networking* reason:

> Key Vault is not reachable from this scope (public network access is Disabled, no in-VNet runner)

But `docs/accepted-risks.md` carries the L-0031 correction added 2026-07-31, which establishes a **second and harder** reason: even with a network path, Key Vault could not hold this key at all, because the key is Ed25519. The module docstring predates that correction and was not updated with it.

This matters because the docstring is what an engineer reads first, and it implies the follow-up is blocked on infrastructure (get an in-VNet runner) when it is actually blocked on an algorithm decision that requires a code change. `accepted-risks.md` says this precisely — *"this is a config swap only if option (a) is chosen; if the algorithm changes, `signing.py` / `verify_signature.py` need the matching code change first"* — and the docstring's own promise, *"moving to a production signing key is a configuration swap, never a code change"*, is now known to be false for the recommended path.

Fix: three lines in the `signing.py` docstring, pointing at L-0031. No behaviour change. It is the cheapest item in this entire document set.

Pending checks — identified, not yet run

Ordered by what they would change in this documentation set. P3 and P4 are complete; the remainder are lower-value and unattempted.

#	QUESTION	SOURCE TO QUERY	WOULD RESOLVE
P5	Do any Canva **brand templates** exist?	Canva `search-brand-templates`, `list-brand-kits`	Function 45's carousel path and `bulk_create_from_csv` are template-locked — `template_id` is required. No templates would be a hard blocker on the weekly loop that nothing in the repo records.
P6	Do any Fireflies transcripts exist?	Fireflies `fireflies_get_transcripts`	Function 26 harvests advocacy from transcripts. The integration is unbuilt (`12` I18); if there is also no input, the function is inert on both sides.
P7	Do the four ingestion URLs still resolve?	Microsoft Learn (for the Fabric page); web fetch for the RSS feeds	`fetch_sources.yaml` 's own header asks for exactly this: *"re-verify this liveness periodically, since a renamed/retired page silently narrows AC-24's guarantee rather than failing loudly."*
P8	Is Semrush a realistic future signal source?	Semrush `domain_overview`	`contracts/vault-api.yaml` 's own example payload cites `source: "semrush"`, but no integration exists (`12` I18).

P5 is now the most valuable of these: a missing brand template would be a hard blocker on the weekly loop that nothing in the repo records.

What this cannot fix

The connectors verify **counterparties**, not the platform. They cannot:

- inspect live Azure state (`cmos-dev` resource group, Container App revisions, actual env vars) — so **TD-31 cannot be confirmed as currently-broken or currently-fine**. P3 verified its *mechanism* against Microsoft's documentation; the *current state* of `ca-mcp-web` remains unknown from here;
- read the Postgres instance, so no claim about live data volumes or row states can be checked;
- read Application Insights, so no telemetry or trace claim can be checked;
- confirm whether `MCP_WEB_LIVE_MODE` is set on `ca-mcp-web` right now — the single most consequential open question in this documentation set.

Those all require `az` CLI access to the subscription. Until then, the limitation stated in `README.md` stands unchanged.

Operational note

Two constraints shaped how much of this log could be filled in, both worth knowing before anyone tries to extend it:

1. **The MCP connectors flap.** Servers were observed disconnecting and reconnecting repeatedly, and alternating between friendly names and hashed ids. Any batch of checks should tolerate a mid-run drop and resume rather than assume a stable session.
2. **`learn.microsoft.com` is not directly reachable from this environment.** `WebFetch` and `curl` both return HTTP 403 through the proxy, and the Microsoft Learn MCP connector returned `requires approval` on every call. P3 and P4 were answered by search over the Learn corpus instead — good enough to settle both questions, but a weaker citation than reading the page, and labelled as such above.

What was recorded elsewhere as a result

FROM	BECAME
P3	Confirmation paragraph added to `09` TD-31
P4	`09` TD-25 corrected — the limitation is **not** tier-specific
P4 side-finding	`09` TD-33 (new) — `signing.py` docstring predates L-0031
V2	`09` TD-32 (new, Priority 2) — brand rules uncalibrated against real output
V1	No debt item. It is an adoption finding, not a defect — see `07` and `10` R1